

GovAsset Platform — Complete Presentation Guide

Read everything below and you will understand your system fully. All answers use simple language — no unnecessary technical words.

Table of Contents

#	Part	What it explains
1	Django Request Flow	Step-by-step: what happens from typing a URL to getting a response
2	Multi-tenancy	What multi-tenancy means (apartment analogy)
3	django-tenants	How django-tenants creates separate schemas per ministry, with code evidence
4	Keycloak SSO Flow	The 14-step login flow + Keycloak purpose + sessions + session expiry (3 timers)
5	Keycloak vs SimpleJWT	When each is used, why mobile doesn't use Keycloak, why web doesn't use JWT
6	0.0.0.0:8000 Explained	Why we use it, what happens if we don't
7	All Code Components	Decorators, serializers, middleware, permissions, exception handler, pagination, helper functions explained
8	Security Features	All 15 security features + dual brute force protection (Keycloak + Django), Keycloak admin console settings, IP capture in audit logs, IntegerField vs ForeignKey explained
9	Database Design	Public schema vs tenant schemas, how they "join" (they don't — that's the security point), what each holds, the one controlled bridge (user authentication), why not ministry_id filter
10	Other Technologies	DRF, CORS, mozilla-django-oidc, drf-yasg, SimpleJWT, decouple, etc.
11	Deployment to Cloud	What you need for production, do you use 0.0.0.0?
12	JWT Tokens	Access vs refresh token, 3 parts of JWT, expiration, how to change
13	Integration With Other Groups	How they call our API, what we share, example code, all endpoints listed
14	Why Each Ministry Needs a Domain	Why not just use /ministry/1/ in the URL
15	Presentation Tips	Common panel questions, demo sequence, how to answer
16	Quick Reference	Every key file and what it does
17	Complete Request-to-Response Flow	Web browser flow + mobile API flow in full detail
18	Glossary	Simple definitions of every technical term
19	Remember This for the Panel	Key sentences to memorize
20	CSS Quick Reference	How to change colors, text size, backgrounds — inline CSS & Bootstrap Icons — plus asset expiry color coding (red/amber/blue/green meanings)
21	Database Deep Dive	All tables, columns, why shared vs private schemas, why PostgreSQL auto-starts — plus live schema count (6 schemas, 50 tables), every table's purpose, Django vs our tables explained, the SHARED_APPS vs TENANT_APPS settings split, and what setup_demo_data touches per schema
22	Making the System Live	Production setup, domains, code changes, NGINX proxy, load balancers, X-Forwarded-For IP headers explained
23	Dev Setup: Web + Mobile	How two developers work together + which fixes affect mobile vs web only

#	Part	What it explains
24	Complete File-by-File Explanation	Every folder and every <code>.py</code> file explained, with key classes and code evidence
25	Important Commands & Setup Reference	Full step-by-step: PostgreSQL, find IP, start Keycloak, start Django, connect phone, register domain, Flutter rebuild, demo data (flush vs setup_demo_data explained with code), troubleshooting, test accounts
26	The Full 10-Group Project	Complete project structure, Group 1's role as security/auth foundation, dependency matrix (all 9 groups depend on us), dissertation titles, where our API fits
27	Tonight & Tomorrow Checklist	What to do tonight, tomorrow morning, 30 mins before panel, what to bring, what NOT to do
28	Defense Strategies	What If scenarios — crash, freeze, missing features, technology choices, no tests, performance, forgetting your lines
29	Confidence Tips	Posture, eye contact, voice, recovery from stumbling, the 3-bullet rule, owning weaknesses, the panel's real questions, final pep talk
30	Pre-Presentation Testing Checklist	9-phase test run with code references for live demos — run this before the panel enters
31	How Django Connects to Keycloak	Full SSO link chain (button → URL → view → Keycloak), 4-step redirect, complete OIDC flow diagram, production choices: our Keycloak vs government Keycloak (broker vs replace), what stays the same regardless of identity provider
32	Why We Built Group 2's Asset Register	Corrected defense: proof-of-concept (can't validate security without data), audit trail scope requires transactions, clean removal boundary (keep vs hand over table), what to say to the panel
33	Production Scope Cleanup	What stays vs what is removed (with file-by-file breakdown), 1,940 lines removed = 40% of codebase, why system still works, production architecture diagram
34	The Complete Integrated System	Shared identity, independent systems — each group runs their own separate app, integration is at the identity level (Keycloak), not the UI level. What you actually give other groups (server address, test credentials, verify-token endpoint). User experience walkthrough. Why not all-in-one-Django. Real-world asset journey.
35	Swagger API Documentation	What Swagger is, where defined in code (settings.py:40, urls.py:56-79, 146-156), URL to access (<code>/api/docs/</code>), how it auto-generates docs, full endpoint table, step-by-step for other groups, Authorize button explained, common misconceptions, demo instructions
36	Why Keycloak vs Normal Auth + Sidebar Links	Two scenarios (one Django project vs separate servers), why sidebar links alone don't solve SSO, comparison table (Django auth vs Keycloak), user experience walkthrough with and without Keycloak, government standard argument, one-liner answer for panel
37	Beginner's Guide: Integration & Keycloak Flow	Step-by-step for beginners: (A) How integration works — full merge vs separate servers, (B) Tracing keycloak_login_url across 10 steps with diagram, (C) Every Keycloak setting in settings.py explained in plain language with table, (D) Quick cheat sheet for panel

PART 1: DJANGO REQUEST FLOW — What happens from URL to response?

In this part:

- [11-step request flow](#)
- [Our project's specific flow \(Keycloak\)](#)

11-step request flow

When someone types a URL or clicks a link, here is the exact path:

```
Browser types: http://moh.localhost:8000/assets/
```

```
Step 1: Browser sends the URL to your computer (localhost)
Step 2: Django receives it and looks at the URL path: /assets/
Step 3: django-tenants middleware checks: "What domain is this?" → moh.localhost
Step 4: django-tenants looks up "moh" in the Domain table → finds Ministry of Health
Step 5: django-tenants switches the database to moh_schema
Step 6: Django matches /assets/ against urlpatterns in urls.py
Step 7: Finds asset_list_view (or AssetListCreateAPIView if API)
Step 8: The view function runs – it queries the database
Step 9: The database only sees moh_schema's tables (data isolation!)
Step 10: The view returns an HTML page or JSON response
Step 11: Django sends the response back to the browser
```

Key point: django-tenants steps 3-5 happen on EVERY request, BEFORE your view code runs. This is why `TenantMainMiddleware` must be the first middleware in the list.

Our project's specific flow:

```
User → Login Page → Clicks "Sign in with Government SSO"
→ Django redirects to Keycloak login page (separate website)
→ User types username/password on Keycloak
→ Keycloak redirects back to Django with a code
→ Django exchanges code for tokens (server talks to server)
→ Django verifies the token signature (proves it's really Keycloak)
→ Django finds or creates the user in its database
→ User is redirected to /dashboard/
```

PART 2: MULTI-TENANCY — What does it mean?

In this part:

- [What multi-tenancy means](#)
- [Wrong approach \(ministry_id column\)](#)
- [Our approach \(PostgreSQL schemas\)](#)

What multi-tenancy means

Multi-tenancy means one system serves many customers, and each customer's data is completely separate.

Think of an apartment building:

- The building = one Django application
- Each apartment = one ministry's data
- Each apartment has its own locked door (PostgreSQL schema)
- A person in apartment 3 cannot walk into apartment 5

Our system: Instead of apartments, we have ministries (MOH, MOF, etc.). Each ministry gets their own locked room in the database. Ministry of Health staff ONLY see Health data. Ministry of Finance staff ONLY see Finance data.

NOT multi-tenant (wrong approach):

```
One big table with a "ministry_id" column
→ Risk: A bug might accidentally show Finance data to Health users
→ Risk: A programmer forgets to filter by ministry_id
```

OUR approach (correct):

Separate PostgreSQL schemas

- Each ministry has its own table called `assets_asset`
- If a query doesn't filter by ministry, it STILL only sees one schema
- Data isolation is ENFORCED BY THE DATABASE, not by our code

PART 3: django-tenants — How exactly does it work?

In this part:

- [How django-tenants works \(3 key mechanisms\)](#)
- [Evidence in our code](#)

How django-tenants works

django-tenants is a library that:

- 1. Changes the database engine** — Instead of the normal PostgreSQL driver, it uses `django_tenants.postgresql_backend`. This custom driver adds `SET search_path = moh_schema, public;` before EVERY query.
- 2. Identifies the tenant from the URL** — `TenantMainMiddleware` reads the domain (e.g., `moh.localhost`), looks it up in the `tenants_domain` table, finds which Ministry it belongs to, and sets the schema name.
- 3. Creates schemas automatically** — When a Super Admin creates a new Ministry, django-tenants runs `CREATE SCHEMA moh_schema` in PostgreSQL and creates all the tables inside it (`assets_asset`, `organizations_orgunit`, etc.).

Evidence in our code:

settings.py:

```
# Custom PostgreSQL backend (line 121)
DATABASES = {
    'default': {
        'ENGINE': 'django_tenants.postgresql_backend', # ← This is key
    }
}

# Router tells Django to use this backend for all apps (line 130)
DATABASE_ROUTERS = ('django_tenants.routers.TenantSyncRouter',)

# Which apps are shared (one copy, public schema)
SHARED_APPS = ['django_tenants', 'tenants', 'authentication', ...]

# Which apps are per-tenant (separate copy per ministry)
TENANT_APPS = ['organizations', 'assets']
```

tenants/models.py:

```
class Ministry(TenantMixin):
    # TenantMixin gives us schema_name field automatically
    auto_create_schema = True # When saved, PostgreSQL creates a new schema
```

Every view that queries tenant data:

```
from django_tenants.utils import schema_context

# Switch to the ministry's private schema
with schema_context(user.ministry_schema):
```

```
# This query ONLY sees that ministry's data
assets = Asset.objects.all()
```

PART 4: KEYCLOAK SSO — The full login flow (simple version)

Keycloak is a separate program that handles logging in. Django does NOT handle passwords — Keycloak does.

The flow with 3 characters:

```
Character 1: THE USER (opens browser)
Character 2: DJANGO (our app)
Character 3: KEYCLOAK (the identity server)

Step 1: User visits http://moh.localhost:8000/login/
Step 2: Django shows the login page with a "Sign in with Government SSO" button
Step 3: User clicks the button → browser goes to /oidc/authenticate/
Step 4: Django says "I don't handle passwords. Go to Keycloak."
         Redirects to: http://localhost:8180/realms/govasset/protocol/openid-connect/auth
Step 5: Keycloak shows its own login page (looks different from Django)
Step 6: User types username and password
Step 7: Keycloak checks: "Is this password correct?" If yes, continues
Step 8: Keycloak creates a temporary CODE and sends the browser back to Django
         URL: /oidc/callback/?code=xyz123
Step 9: Django takes the code and calls Keycloak secretly (server-to-server):
         "Hey Keycloak, here's the code. Give me the real tokens."
Step 10: Keycloak verifies the code and sends back: access_token + id_token
Step 11: Django checks the id_token's SIGNATURE (like checking a passport seal)
         - It fetches Keycloak's public key
         - Verifies the signature matches (proves Keycloak signed it)
         - Checks expiration date
Step 12: If everything checks out, Django looks up the user in its database
Step 13: If user exists → login successful → redirect to /dashboard/
Step 14: If user doesn't exist → create PendingAccess → show error message
```

How Django knows the user is real (not a hacker):

Three checks, like a passport check at the airport:

```
Check 1 – THE SIGNATURE (cryptographic):
  Keycloak signs the token with its PRIVATE key (like a government stamp)
  Django verifies with Keycloak's PUBLIC key (like checking the stamp is real)
  If someone forged a token, the signature won't match → REJECTED

Check 2 – THE PASSPORT DETAILS (claims):
  Django checks:
  - Is the token expired? (exp field)
  - Was this token meant for OUR app? (aud field)
  - Did Keycloak really issue this? (iss field)
  If any fails → REJECTED

Check 3 – THE DATABASE LOOKUP (application):
  Django looks in its own database: "Is this user registered?"
  If no matching user found → REJECTED (shows pending access message)
```

What is the real purpose of Keycloak? (Why do we have it at all?)

This is the most important question. Here is the answer:

Keycloak exists so that one username + password works across MULTIPLE government systems, not just ours.

Imagine the government has 20 different systems:

- Our asset management system
- A human resources system
- A budget system
- A procurement system
- A document management system

Without Keycloak, a user would need **20 different usernames and passwords** — one for each system. With Keycloak, they log in ONCE and access ALL 20 systems.

Analogy: Keycloak is like a government ID card. The ID card is issued by the central government (Keycloak). It works at the health ministry (our system), the finance ministry (another system), the education ministry (another system). You don't need a separate ID card for each ministry.

OUR APPROACH (with Keycloak):

```
User has ONE account in Keycloak

Our system:    "Is this user real? Let me check
                Keycloak's signature on their token"

HR system:     "Same user? Let me check Keycloak
                too. Yes, same person."

Budget system: "Same user. Same ID. Same credentials."
```

ALTERNATIVE (without Keycloak):

```
User has SEPARATE accounts in every system

Our system:    username: moh_admin, password: abc123
HR system:     username: aminah,   password: xyz789
Budget system: username: ahassan,   password: qwe456

User forgets 3 passwords. Writes them on sticky notes.
Sticky note falls off. Security risk.
```

Key point: Keycloak is NOT just for us. It is the CENTRAL identity system for ALL government online services. Our Django app is just ONE of many systems that use it.

How Django and Keycloak work together — the relationship

KEYCLOAK'S JOB:

```
Store passwords
Check passwords
Issue tokens
Handle "forgot pwd"
Handle "new user"

Users from ALL
government systems
live here
```

DJANGO'S JOB:

```
Store user profiles
Store roles
Store ministry
Store permissions
Store audit logs
Handle API tokens

Only our system's
users live here
```

↑
THEY TALK
VIA OIDC PROTOCOL
↑

```
EXAMPLE: User "amina@moh.go.tz"
In Keycloak:    username + password (the "who are you?" part)
In Django:      role = MINISTRY_ADMIN, ministry = MOH (the "what can you do?" part)
```

Why both? Why not just Django or just Keycloak?

If we ONLY used Keycloak: Keycloak doesn't know what ministry the user belongs to, what their role is, what assets they manage. Keycloak only knows "this username, this password, this email."

If we ONLY used Django: Each government system would have its own login. User needs 20 accounts. No centralized identity.

Does Django also store user credentials (passwords)?

The answer depends on how the user was created:

For web-only users (logged in via Keycloak): NO. Django never sees the password. The user types their password on Keycloak's page, not Django's page. Django receives a token FROM Keycloak that says "this user is authenticated," and stores an **unusable password** in its database — a placeholder like `!aBcDeF12345`.

This means:

- The user CAN log in through the web browser (Keycloak handles password checking)
- The user CANNOT log in through the mobile API (no valid Django password to check against)

For mobile/API users (logged in via direct API): YES. Django receives the password from the mobile app and checks it against its database. The password is stored securely (hashed using Django's password hasher).

This means:

- The user CAN log in through the mobile API (Django checks the stored password)
- The user can ALSO log in through the web (if they also have a Keycloak account)

Key insight — A user needs TWO different credentials for web + mobile:

EXAMPLE: User "amina@moh.go.tz"	
KEYCLOAK PASSWORD (for web browser)	DJANGO PASSWORD (for mobile app)
password: "M0H@2024"	password: "Am!naM0bile99"
She types on Keycloak login page	She types in Flutter app login screen
Stored in Keycloak's own database	Stored in Django's database (hashed, never in plain text)

These are **SEPARATE** passwords stored in **SEPARATE** databases. Changing one does not change the other.

How does a user get a Django password?

1. When a Super Admin creates the user directly in Django (via the user management page), they set a password
2. When a PendingAccess request is approved, the admin sets a password at that time
3. The password is sent to Django, hashed, and stored in the `password` field of `authentication_customuser` table

What happens if a Keycloak-only user tries to log in via the mobile app? They fail. Django has no valid stored password for them (only an unusable placeholder). The mobile app shows "Invalid credentials." This is correct behavior — the admin must set a separate Django password if the user needs mobile access.

What if a user exists in Keycloak but NOT in our Django database?

Step 1: User goes to our login page, clicks SSO, types their Keycloak username/password correctly. Keycloak says "Yes, this person is real."

Step 2: Keycloak sends the user back to Django with a valid token. Django checks the token and verifies: "Yes, Keycloak really signed this. This user is real."

Step 3: Django looks in its OWN database for this user. If the user exists → login success. If the user does NOT exist:

- Django creates a PendingAccess record
- Django shows the user a page saying:
 - "Your account is not yet registered. Contact your administrator."
- The Super Admin sees a pending approval request
- The Super Admin can approve or reject
- If approved: Django creates the actual user in its database (with role, ministry, and a Django password for mobile access)
- The user can now log in via web (Keycloak) AND mobile (Django password)

Code evidence (authentication/oidc_backend.py):

```
def create_user(self, request, user_claims):
    # This runs when Keycloak says "user is real"
    # but Django has no matching user
    ...
    PendingAccess.objects.create(
        email=email,
        keycloak_id=sub,
        first_name=first_name,
        last_name=last_name,
    )
    return None # ← Returning None means "user not allowed yet"
```

How does someone get created in Keycloak alone? (Without Django knowing)

An IT officer at a ministry can create a user in Keycloak directly without involving Django at all:

```
IT Officer goes to http://localhost:8180 → govasset realm → Users → Add User
→ Sets username: "amina", email: "amina@moh.go.tz"
→ Sets a temporary password
→ Tells Amina: "Your account is ready. Go to the system and log in."
  ↓
Amina opens our web app → Clicks "Sign in with Government SSO"
→ Keycloak says: "Yes, I know you. Password is correct."
→ Keycloak sends Amina to Django with a valid token
→ Django checks its database... "I don't know this person."
→ PendingAccess record is created
→ Amina sees: "Your account is awaiting approval. Contact your administrator."
→ Super Admin sees the pending request → approves it
→ Super Admin assigns Amina's role and ministry
→ Super Admin sets Amina's Django password (for mobile access)
→ Amina can now log in via web AND mobile
```

Important: Keycloak alone is NOT enough to access our system. The user must also exist in Django with a role and ministry assignment. PendingAccess is the bridge between "Keycloak knows you" (identity) and "Django knows you" (authorization).

Can someone be created in Django alone? (Without Keycloak)

YES. A Super Admin can create a user directly in Django using the user management page. This creates the user only in Django. The user can then log in via the mobile app (API login with Django password) but NOT via the web (no Keycloak account).

If the user also needs web access, the Super Admin must ALSO create them in Keycloak. Our code in `authentication/keycloak_admin.py` helps automate this — when you create a user in Django, it can optionally create a matching account in Keycloak too.

Summary: Two creation paths

```

PATH 1: Keycloak → Django
  User created in Keycloak by IT officer
    → User tries web login
    → PendingAccess created
    → Super Admin approves
    → Django user created with unusable password
    → User can log in via web ONLY
    → (Mobile access requires admin to set separate Django password)

PATH 2: Django → Keycloak
  Super Admin creates user in Django
    → Django creates matching account in Keycloak automatically
    → User gets BOTH:
      - Django password (for mobile API login)
      - Keycloak account (for web browser SSO)
    → User can log in via web AND mobile

```

The PendingAccess flow (Path 1) is designed for the real-world scenario where an IT officer creates users in the central identity system (Keycloak) without coordinating with our team. PendingAccess ensures no user gets access to our system until a Super Admin explicitly approves them and assigns a role.

Does Django manage sessions?

YES. Django manages TWO kinds of sessions:

Kind	What it is	Where it's stored
Django session cookie	"This browser is logged in as user X"	<code>django_session</code> table (public schema)
Keycloak session	"This user has an active Keycloak login"	Inside Keycloak's database

When you log in via the web:

1. Keycloak creates a session (you are logged into Keycloak)
2. Django creates a session (you are logged into Django)
3. Both are active at the same time

When you log out:

1. Django destroys its session (you're logged out of Django)
2. Django tells Keycloak to destroy its session too (you're logged out of Keycloak)
3. Both sessions are gone

Session expiry — Why you get asked to sign in again

You noticed that if you leave the system idle for a while, or switch to another tab for a long time, you come back and it asks you to sign in again. This is called **session expiry** and it is one of your security features — not a bug.

There are actually **three separate timeout systems** running in your project simultaneously. Each one works differently depending on whether you are on the web or mobile.

System 1 — Django Session Timeout (Web Platform)

When you log into the web platform, Django creates a **session** — a record stored in your database that says "this browser is logged in as user X."

This session has a time limit. In `config/settings.py`:

```
SESSION_COOKIE_AGE = 28800      # 8 hours (in seconds)
SESSION_COOKIE_HTTPONLY = True  # JavaScript cannot steal this cookie
```

28800 seconds = 8 hours. After 8 hours of inactivity, the session expires and Django makes you log in again.

Did you build this or did it come automatically? You configured it. Django's session system exists by default, but the 8-hour limit is something you deliberately set. A typical Django default is 2 weeks — you shortened it to 8 hours because a government platform should not stay logged in for 2 weeks on an unattended computer.

How to check:

```
Open config/settings.py → search for SESSION_COOKIE_AGE
Divide by 3600 to get hours: 28800 ÷ 3600 = 8 hours
```

How to change it (e.g., to 4 hours):

```
SESSION_COOKIE_AGE = 14400      # 4 hours (4 × 60 × 60)
```

System 2 — Keycloak Session Timeout (Web Login)

Keycloak has its own separate session. When a user logs in through Keycloak's page, Keycloak creates its own session remembering "this browser authenticated as user X."

This matters for **SSO (Single Sign-On)** — if a user has a valid Keycloak session and navigates to another system connected to the same Keycloak, they don't need to type their password again. But if the Keycloak session expires, even a valid Django session would force a re-authentication through Keycloak.

Where is this configured? In the Keycloak admin panel at <http://localhost:8180>:

```
Login as admin → govasset realm → Realm Settings → Sessions tab
```

You will see settings like:

- **SSO Session Idle** — how long a session can be idle before Keycloak expires it
- **SSO Session Max** — the absolute maximum lifetime regardless of activity

Did you build this? No — Keycloak manages this entirely on its own. You only interact with it through the Keycloak admin panel.

System 3 — JWT Token Expiry (Mobile App and API)

The mobile app does NOT use Django sessions or Keycloak sessions. It uses **JWT tokens**. These tokens have their own expiry built directly into them.

In `config/settings.py`:

```
SIMPLE_JWT = {
    'ACCESS_TOKEN_LIFETIME': timedelta(minutes=30),  # Token dies after 30 min
    'REFRESH_TOKEN_LIFETIME': timedelta(days=1),    # Refresh lasts 1 day
}
```

Access token — lasts 30 minutes. After 30 minutes, any API call the mobile app makes gets a 401 Unauthorized response. The app uses the refresh token to silently get a new access token without making the user type their password again.

Refresh token — lasts 1 day. After 1 day without any app activity, the refresh token expires. The mobile app returns to the login screen and the user must type their password again.

Did you build this? You configured it. The `djangoestframework-simplejwt` library handles the token creation — but the 30-minute and 1-day limits are values you set.

The complete picture — Three timers running at once

WEB PLATFORM USER

Opens browser → logs in through Keycloak → Django creates a session

Timer 1: Django session → 8 hours idle → must re-authenticate to Django

Timer 2: Keycloak session → set in Keycloak admin → must re-enter password

MOBILE APP USER

Opens app → logs in through Django API → receives JWT tokens

Timer 3: Access token → 30 minutes → app silently refreshes

Timer 4: Refresh token → 1 day → must log in again

What you are probably seeing:

- **After a few hours away** → Django session expired (Timer 1, 8 hours)
- **Asked to type password on Keycloak's page specifically** → Keycloak session expired (Timer 2)
- **Mobile app shows login screen after a day** → Refresh token expired
- **Quickly after switching tabs** → Not a timeout — likely your browser lost the cookie, or you accidentally logged out

Forgot password — How it works with Keycloak

Question: Does the "forgot password" feature still work now that Keycloak handles passwords?

Answer: Yes, but it works differently — and this is actually more secure.

Before Keycloak: Django's own "forgot password" flow would send an email with a reset link, and the user would reset the password in Django's database.

After Keycloak: The password doesn't live in Django's database anymore. Django's password field is empty (set to an unusable value) for Keycloak-authenticated users. The password lives entirely in Keycloak.

So "forgot password" now goes through Keycloak:

```
User clicks "Forgot Password" on Keycloak's login page
↓
Keycloak sends a reset link to their email
↓
User clicks the link, sets a new password in Keycloak
↓
Done — they can log in again with the new password
```

Django is not involved in this process. This is **more secure** because password reset logic is handled by a dedicated, tested security tool (Keycloak) rather than our own code.

Does our Django "forgot password" URL still work? It might appear to work (send an email), but it would reset a password that isn't actually used anymore. If your login page has a "forgot password" link that goes to Django's own reset page, you should remove that link or redirect it to Keycloak's login page where the real forgot-password button lives.

Wrong password logout — Double protection

Question: If someone types the wrong password 5 times, does the lockout still work with Keycloak?

Answer: You actually have TWO separate lockout systems protecting you:

Lockout System 1 — Keycloak's own lockout (for web browser users):

Keycloak itself watches for wrong passwords. After 5 wrong attempts, Keycloak locks that account for 15 minutes. This happens entirely inside Keycloak — Django doesn't even know about it.

```
Web user types wrong password 5 times on Keycloak's page
↓
Keycloak locks the account for 15 minutes
↓
Web user sees on Keycloak's page: "Account temporarily disabled"
↓
Django was never contacted – Keycloak handled everything
```

Lockout System 2 — Django's own lockout (for API/mobile users):

Your `authentication/models.py` has a `LoginAttempt` model that records every login attempt:

```
# authentication/models.py
class LoginAttempt(models.Model):
    username = models.CharField(max_length=150)
    ip_address = models.GenericIPAddressField()
    attempted_at = models.DateTimeField(auto_now_add=True)
    was_successful = models.BooleanField(default=False)
```

Your API login view checks this before allowing login:

```
# authentication/api_views.py (simplified)
class LoginAPIView(APIView):
    def post(self, request):
        username = request.data.get('username')
        ip = request.META.get('REMOTE_ADDR')

        # Count recent failed attempts from this IP
        recent_failures = LoginAttempt.objects.filter(
            username=username,
            was_successful=False,
            attempted_at__gte=timezone.now() - timedelta(minutes=15)
        ).count()

        if recent_failures >= 5:
            return Response(
                {'error': 'Too many failed attempts. Try again in 15 minutes.'},
                status=429
            )

        # Proceed with login
        user = authenticate(username=username, password=password)

        # Record this attempt
        LoginAttempt.objects.create(
            username=username,
            ip_address=ip,
            was_successful=(user is not None)
        )
```

Summary — Who protects whom:

Who is trying to log in	Which lockout protects them
Web browser user (through Keycloak page)	Keycloak's own built-in lockout
Mobile app user (through your API)	Your Django <code>LoginAttempt</code> lockout
Another group's system (through your API)	Your Django <code>LoginAttempt</code> lockout

You have double protection. Both paths are covered.

PART 5: What is the difference between Keycloak and SimpleJWT?

People get confused. Here is the simple answer:

	Keycloak	SimpleJWT
Used for	Logging in (web browser)	API calls (mobile app)
Who handles it	Keycloak server (separate program)	Django itself
What it produces	A Django session (cookie in browser)	A JWT token (text string)
When is it used	When user clicks "Sign in"	When mobile app calls API
Password involved?	Yes, user types password on Keycloak	Yes, on first login only

Think of it this way:

- **Keycloak** = The security guard at the gate who checks your ID
- **SimpleJWT** = The visitor badge you get AFTER the guard lets you in

The mobile app (Flutter) does NOT use Keycloak. Instead:

1. Mobile app calls `POST /api/auth/login/` with username + password
2. Django checks the password against its database (same as Keycloak)
3. Django returns a JWT token (valid 30 minutes)
4. Mobile app sends this token with every API call
5. When token expires, mobile app calls `POST /api/auth/refresh/`

Why doesn't the mobile app use Keycloak?

Keycloak uses a **browser redirect flow** — it sends the user to a Keycloak login page, the user types their password there, then it redirects back to Django.

This does NOT work on a mobile app because:

```

WEB BROWSER FLOW (works with Keycloak):
  Browser → Django → redirect to Keycloak → user types password → redirect back
  ↑                                     ↑
  Browser can follow redirects automatically.
  The "Sign in with Google" button you see on websites works this way.

MOBILE APP FLOW (cannot use Keycloak):
  Flutter app → Django → "Go to Keycloak" → ???
  ↑
  A mobile app is not a web browser. It cannot display Keycloak's
  login page. It cannot follow web redirects.

Option A: Open a web browser inside the app (WebView)
  → Ugly, confusing, users don't trust it
  → Keycloak session doesn't persist properly
  → Many technical problems

Option B: Mobile app sends username + password directly to Django
  → Simple, works every time

```

```
→ Django checks the password and returns a JWT token
→ This is what we do ✓
```

The real reason: Keycloak (and all OIDC providers) were designed for web browsers. Mobile apps use a different authentication pattern called "direct login" where the app sends credentials to an API and gets back a token. This is industry standard — even Google, Facebook, and Twitter have separate "mobile login" APIs that don't use browser redirects.

Why doesn't the web browser use SimpleJWT (like the mobile app)?

Three reasons:

Reason	Explanation
1. Security	With SimpleJWT, the user types their password directly into Django's login page. If someone steals the password, they have full access. With Keycloak, Django never sees the password — Keycloak handles it securely.
2. Single Sign-On	If the web used SimpleJWT, users would need a separate login for EVERY government system (asset management, HR, budget, etc.) — 20 different passwords. With Keycloak SSO, one login works everywhere.
3. Government policy	The government mandates that ALL online services use Keycloak for authentication. We cannot build our own login page that bypasses Keycloak. This is a security policy, not a technical choice.

So why does the mobile app get an exception?

Because there is no other way. Mobile apps cannot use Keycloak's browser redirect flow (explained above). The government understands this and allows mobile apps to use direct API login. But the mobile app's login is STILL connected to Keycloak indirectly:

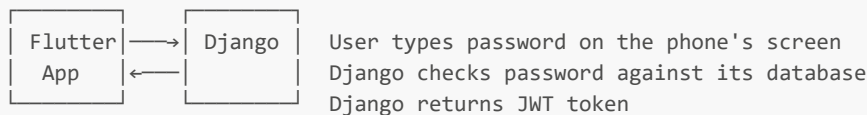
```
Mobile app login → Django checks password → Django stores users that
                  came FROM Keycloak originally
                  ↑
The user first had to exist in Keycloak and be approved in Django
via the web flow. The mobile app login is a CONVENIENCE for
already-approved users.
```

Summary: Two authentication paths side by side

PATH 1: WEB BROWSER (uses Keycloak)



PATH 2: MOBILE APP (uses SimpleJWT)



PART 6: Why 0.0.0.0:8000 instead of just `runserver`?

Normal runserver:

```
python manage.py runserver
→ Django listens ONLY on 127.0.0.1:8000
→ 127.0.0.1 = "this computer only"
→ Your phone CANNOT reach this
→ moh.localhost CANNOT reach this
```

0.0.0.0:8000:

```
python manage.py runserver 0.0.0.0:8000
→ Django listens on ALL network interfaces
→ 0.0.0.0 = "everywhere"
→ Same WiFi network? Your phone CAN reach this
→ moh.localhost on same computer CAN reach this
```

Analogy:

- **127.0.0.1** = a phone that only calls itself
- **0.0.0.0** = a phone that can receive calls from anyone

Why 127.0.0.1? — What it means when you see it

When you open your browser on your laptop and go to <http://localhost:8000>, the request NEVER leaves your machine. It travels from one program (your browser) to another program (Django) entirely inside the same computer, through what is called a **loopback interface**.

Every computer in the world reserves the address **127.0.0.1** to mean **"myself."** It is not a real network address — it is a shortcut that means "stay inside this machine."

So when Django reads `request.META['REMOTE_ADDR']` and the browser is on the same laptop, the answer is always **127.0.0.1** — because the request came from the machine itself.

When your phone sends a request over the hotspot, the request travels through a real network — from your phone's WiFi radio, through the hotspot signal, to your laptop's WiFi adapter. At that point `REMOTE_ADDR` is a real network address like **192.168.43.118** — your phone's actual address on that small private network.

The rule — Who sees what IP:

Who sent the request	What IP you see in request.META
Browser on the same laptop as Django	127.0.0.1 (loopback — never leaves the machine)
Phone over hotspot	192.168.43.x (phone's real network address)
Another laptop on the same WiFi	That laptop's IP
Real internet user (in production)	Their real public IP

Why this matters for security: In a real government deployment, every audit log entry would show the real public IP of whoever performed the action. Investigators can trace a public IP back to a physical location or an organization if needed. During development, you see **127.0.0.1** which is correct — the request is from your own machine.

Do you need 0.0.0.0 in production (cloud)?

NO. In production, you use a proper web server like Nginx or Apache. The server listens on **0.0.0.0:80** (that's what web servers do by default). Django sits behind the web server and only listens on localhost (127.0.0.1:8000) — the web server forwards requests to it.

So: **0.0.0.0:8000** is ONLY for development when you need your phone or tablet to test.

What if you don't use 0.0.0.0?

Then your mobile phone connected to the same WiFi cannot access the website. You type `http://192.168.100.18:8000` on your phone and nothing happens, because Django only listens on `127.0.0.1` (which means itself only).

PART 7: All Your Code Components — Explained Simply

What is `mozilla_django_oidc`?

It is a Django library that does the Keycloak handshake for us. Without it, we would have to write hundreds of lines of code to handle the OIDC protocol (redirects, code exchange, token verification). The library provides:

- `/oidc/authenticate/` — Redirects user to Keycloak login page
- `/oidc/callback/` — Handles the return from Keycloak, exchanges code for tokens
- `/oidc/logout/` — Logs user out of both Django and Keycloak

What is `rest_framework` (DRF)?

Django REST Framework is a library that makes it easy to build APIs (Application Programming Interfaces). An API is how the Flutter mobile app talks to our Django backend.

Without DRF: You would have to manually parse JSON from requests, manually convert database data to JSON, manually handle errors, etc.

With DRF: You define a serializer (a class that says "here are the fields") and a view (a class that says "what happens on GET, POST, etc."), and DRF handles everything.

What is `corsheaders`?

CORS = Cross-Origin Resource Sharing.

Problem: By default, a web browser on `flutterapp.com` cannot call an API on `moh.localhost:8000`. The browser blocks it for security.

Solution: CORS headers. We tell Django: "Hey, it's OK if the Flutter app calls our API from a different address."

Our settings:

```
CORS_ALLOW_ALL_ORIGINS = True # Allow Flutter from any address
CORS_ALLOW_CREDENTIALS = True # Allow cookies to be sent
```

What is `drf_yasg`?

It automatically creates a Swagger documentation page at `/api/docs/`. This is a nice web page that shows ALL your API endpoints, with buttons to test them. Other groups (Group 2-10) use this page to understand how to call our API.

You don't write documentation — `drf_yasg` reads your code and generates it.

What is `python-decouple`?

It reads secret values from a `.env` file. This keeps passwords out of your code.

```
# Instead of writing the password in settings.py:
SECRET_KEY = 'my-secret-key-here' # BAD — visible in code

# We do this:
SECRET_KEY = config('SECRET_KEY') # GOOD — read from .env file
```

Your `.env` file has:

```
SECRET_KEY=my-secret-key-here
```



```
DB_PASSWORD=my-db-password
```

The `.env` file is NEVER committed to GitHub (it's in `.gitignore`).

What is `psycopg2-binary`?

It is the driver that lets Python talk to PostgreSQL. Think of it like a translator — Django speaks Python, PostgreSQL speaks SQL, `psycopg2` translates between them.

What is Pillow?

A library that handles image files. When someone uploads a photo of an asset (like a laptop), Pillow processes the image — resizing, format conversion, etc.

What are DECORATORS?

A decorator is a wrapper around a function. It runs code BEFORE your function runs.

Analogy: A security guard at the door of a building. Before you enter (before your function runs), the guard checks your ID.

```
@login_required_custom    # ← This is a decorator (security guard)
@role_required('SUPER_ADMIN') # ← Another security guard
def ministry_list_view(request):
    # This code only runs if BOTH guards say OK
    ...
```

Our custom decorators:

Decorator	What it does	Where it's used
<code>@login_required_custom</code>	Blocks unauthenticated users → redirect to login	ALL views
<code>@role_required('MINISTRY_ADMIN')</code>	Blocks wrong roles → redirect to dashboard	Admin views
<code>@ministry_isolation_check</code>	Blocks users without a ministry schema	Ministry views

Code (authentication/decorators.py):

```
def login_required_custom(view_func):
    @wraps(view_func)
    def wrapper(request, *args, **kwargs):
        if not request.user.is_authenticated:
            messages.warning(request, "Please log in.")
            return redirect('login')
        return view_func(request, *args, **kwargs)
    return wrapper
```

What are SERIALIZERS?

A serializer converts between two formats:

1. **Python objects** → **JSON** (for the API response)
2. **JSON** → **Python objects** (from the API request)

Example: When Flutter asks for an asset, Django:

1. Gets the Asset object from the database (Python object)
2. Passes it through `AssetSerializer`
3. DRF converts it to JSON automatically
4. Flutter receives: `{"id": 1, "name": "Dell Laptop", "asset_number": "MOH-ICT-0001"}`

Our serializers (authentication/api_serializers.py):

- `AssetSerializer` — Converts assets to JSON for the mobile app
- `AuditLogSerializer` — Converts audit logs
- `OrgUnitSerializer` — Converts organization units
- `UserProfileSerializer` — Converts user profiles (never exposes passwords!)
- `CustomTokenObtainPairSerializer` — Adds role, ministry_schema to JWT tokens

What is MIDDLEWARE?

Middleware is code that runs on EVERY request, BEFORE and AFTER your view.

Think of it as a pipeline:

```
Request comes in
→ Middleware 1 (TenantMainMiddleware: sets schema)
→ Middleware 2 (SecurityMiddleware: security headers)
→ Middleware 3 (SessionMiddleware: loads session)
→ Middleware 4 (CorsMiddleware: handles CORS)
→ Middleware 5 (AuthMiddleware: loads logged-in user)
→ Middleware 6 (SchemaMiddleware: OUR custom one)
→ YOUR VIEW FUNCTION
→ Back through all middlewares
Response goes out
```

Our custom middleware (authentication/middleware.py):

```
class SchemaMiddleware(MiddlewareMixin):
    def process_request(self, request):
        # Skip public pages
        # For authenticated users, save their schema name on the request
        if hasattr(request, 'user') and request.user.is_authenticated:
            request.schema_name = request.user.ministry_schema or 'public'
            request.user_role = request.user.role
```

This means ANY view can access `request.schema_name` — no need to look it up from the database each time.

What is the EXCEPTION HANDLER?

When an API error happens (e.g., user not authenticated, permission denied, not found), DRF returns errors in different formats. The Flutter app needs a CONSISTENT format.

Our custom handler (authentication/api_exception_handler.py) catches all errors and returns them in one shape:

```
{
  "error": True,
  "message": "Authentication credentials were not provided.",
  "code": "authentication_required",
  "status": 401
}
```

Flutter always knows: check `error`, read `message`, handle by `code`.

What is PAGINATION?

When you have 1000 assets and the user views the list, you don't send all 1000 at once — you send 20 at a time (one page).

Our pagination helper (authentication/pagination.py):

```
def paginate_queryset(queryset, request, per_page=20):
    paginator = Paginator(queryset, per_page)
    page = paginator.page(request.GET.get('page', 1))
    return page, paginator
```

Usage in views:

```
page, paginator = paginate_queryset(assets, request, per_page=25)
# page.object_list = the 25 items for this page
# paginator.count = total number of items
# paginator.num_pages = how many pages total
```

What are PERMISSION CLASSES?

DRF has a built-in system for checking who can access API endpoints. We created custom permission classes:

Where (authentication/api_permissions.py):

```
class HasMinistrySchema(BasePermission):
    """Block users who aren't assigned to any ministry."""
    def has_permission(self, request, view):
        if request.user.role == 'SUPER_ADMIN':
            return True
        return bool(request.user.ministry_schema)

class CanManageAssets(BasePermission):
    """Auditors can read only. Others can write."""
    def has_permission(self, request, view):
        if request.method in ['GET', 'HEAD', 'OPTIONS']:
            return True # Anyone can read
        return request.user.role in ['SUPER_ADMIN', 'MINISTRY_ADMIN', ...]
```

Usage in API views:

```
class AssetListCreateAPIView(APIView):
    permission_classes = [
        IsAuthenticated, # Must be logged in
        HasMinistrySchema, # Must have a ministry
        CanManageAssets, # Must have permission
    ]
```

What are HELPER FUNCTIONS?

The most important thing to know: A helper function is NOT a special kind of function. It is just a normal Python function. Python treats it exactly the same as any other function. The name "helper" only describes its **purpose** — it helps other parts of your code do their job.

So what makes a function a "helper"?

A helper function is a small function that does ONE small, reusable job. Think of it as a **tool** you keep in your toolbox and grab whenever you need it.

	Main function	Helper function
Job	Does the main task (e.g., login a user, create an asset)	Does a small supporting task (e.g., get the user's IP)
Size	Can be 20-100+ lines	Usually 3-15 lines

	Main function	Helper function
Used by	The browser/API calls it directly	Other functions call it
Reused?	Usually called once per URL	Called from many places

Analogy:

- Main function = a chef cooking a full meal
- Helper function = a knife that the chef uses to chop, slice, and dice across many meals

The rule — When to create a helper function:

If you write the **same code more than once** in different places, and that code performs **one clear task**, make it a helper function.

For example, instead of writing this in 3 different views:

```
x_forwarded_for = request.META.get("HTTP_X_FORWARDED_FOR")
if x_forwarded_for:
    ip = x_forwarded_for.split(",")[0].strip()
else:
    ip = request.META.get("REMOTE_ADDR")
```

You write it **ONCE** as a helper and call it from everywhere:

```
ip = _get_client_ip(request) # One line – clean!
```

Why does this matter? If you later need to change how the IP is determined (e.g., add support for a new header), you update **ONE** function instead of searching through many files. This is called **Don't Repeat Yourself (DRY)** — a core principle of good code.

Where is `request.META['REMOTE_ADDR']` in our project?

The answer is: **It's not in our project**. It is provided by **Django** automatically.

When a browser sends a request, Django creates an `HttpRequest` object for you:

```
def my_view(request):    # ← Django CREATES this 'request' object
    print(request)       #     You never created it yourself
```

Inside that object, Django stores a dictionary called `META` containing everything about the request — the sender's IP, what browser they used, what domain they visited, etc. Your code just reads from it:

```
request.META['REMOTE_ADDR']    # Returns: "192.168.1.100"
```

You never create `request.META` — Django does it for you automatically on every request.

Example in our code — `_get_client_ip()`:

```
def _get_client_ip(request):
    """Get the real IP address of the user making the request."""
    x_forwarded_for = request.META.get('HTTP_X_FORWARDED_FOR')
    if x_forwarded_for:
        ip = x_forwarded_for.split(',')[0].strip()
    else:
        ip = request.META.get('REMOTE_ADDR')
    return ip
```

Why make this a helper function? Because 3 different places need the user's IP:

1. The audit log (records who did what from what IP)
2. The brute force lockout (counts failed logins per IP)
3. The dashboard (shows login history)

Without a helper, you write the same 6 lines in 3 places. With a helper, you write it ONCE and call it.

Why does it start with an underscore _?

The underscore is a Python **convention** (not a rule enforced by Python). It tells other programmers:

"This function is for internal use only. Don't use it outside this file unless you have a good reason."

Python does NOT stop you from calling `_get_client_ip(request)` from anywhere. The underscore is just a polite warning to other developers.

How to spot a helper function in code:

- It has a descriptive name like `_get_client_ip`, `paginate_queryset`, `format_date`
- It is usually small (3-15 lines)
- It does ONE thing and does it well
- It often starts with underscore `_` (internal use convention)
- It is defined near the top of a file, right after the imports

Another example in our code — `paginate_queryset()`:

```
def paginate_queryset(queryset, request, per_page=20):
    paginator = Paginator(queryset, per_page)
    page = paginator.page(request.GET.get('page', 1))
    return page, paginator
```

Every view that shows a list (assets, users, audit logs) calls this instead of writing pagination logic from scratch. That's the whole point of a helper — write once, use everywhere.

PART 8: SECURITY FEATURES — Complete Code Evidence

Here is EVERY security feature in the system with exact file paths and line numbers where the code lives.

1. Database-level Isolation

What it does: Users of one ministry CANNOT see another ministry's data — even if there is a bug in our code. PostgreSQL schemas enforce this at the database engine level.

How it works: django-tenants switches the PostgreSQL `search_path` on every connection. When a request from `moh_admin` comes in, the database connection points exclusively at `moh_schema`. Every query runs inside `moh_schema` only — it is physically impossible to accidentally touch `mof_schema`.

Evidence	File	Line
'ENGINE': 'django_tenants.postgresql_backend' — custom database driver that sets <code>search_path</code> per request	<code>config/settings.py</code>	101-110
<code>TenantMainMiddleware</code> — inspects the hostname, looks up the matching <code>Domain</code> record, switches to correct schema before any view runs	<code>config/settings.py</code> (MIDDLEWARE)	65
<code>TenantSyncRouter</code> — ensures shared apps migrate to public schema and tenant apps migrate to each ministry schema separately	<code>config/settings.py</code>	111-113

2. Role-based Access Control (RBAC)

What it does: Each user has one of 5 roles (SUPER_ADMIN, MINISTRY_ADMIN, AGENCY_MANAGER, FACILITY_CLERK, AUDITOR). Views check the role before running — if the user's role is not allowed, they are redirected away.

Where the decorator is defined:

Evidence	File	Line
<code>role_required(*allowed_roles)</code> — checks <code>request.user.role not in allowed_roles</code> , redirects to dashboard with error if unauthorized	<code>authentication/decorators.py</code>	19-35
<code>login_required_custom</code> — blocks unauthenticated users	<code>authentication/decorators.py</code>	8-16
<code>ministry_isolation_check</code> — blocks users without a <code>ministry_schema</code> ; Super Admin bypasses	<code>authentication/decorators.py</code>	38-56

Every view that uses `@role_required`:

View Function	Roles Allowed	File	Line
<code>asset_create_view</code>	MINISTRY_ADMIN, AGENCY_MANAGER, FACILITY_CLERK	<code>assets/views.py</code>	186
<code>asset_edit_view</code>	MINISTRY_ADMIN, AGENCY_MANAGER, FACILITY_CLERK	<code>assets/views.py</code>	374
<code>asset_delete_view</code>	MINISTRY_ADMIN only	<code>assets/views.py</code>	516
<code>user_list_view</code>	SUPER_ADMIN, MINISTRY_ADMIN, AGENCY_MANAGER	<code>authentication/user_views.py</code>	12
<code>user_create_view</code>	SUPER_ADMIN, MINISTRY_ADMIN, AGENCY_MANAGER	<code>authentication/user_views.py</code>	42
<code>user_edit_view</code>	SUPER_ADMIN, MINISTRY_ADMIN, AGENCY_MANAGER	<code>authentication/user_views.py</code>	179
<code>user_toggle_active_view</code>	SUPER_ADMIN, MINISTRY_ADMIN	<code>authentication/user_views.py</code>	332
<code>user_reset_password_view</code>	SUPER_ADMIN, MINISTRY_ADMIN	<code>authentication/user_views.py</code>	440
<code>ministry_list_view</code>	SUPER_ADMIN only	<code>tenants/views.py</code>	11
<code>ministry_create_view</code>	SUPER_ADMIN only	<code>tenants/views.py</code>	51
<code>ministry_detail_view</code>	SUPER_ADMIN only	<code>tenants/views.py</code>	157
<code>ministry_toggle_active_view</code>	SUPER_ADMIN only	<code>tenants/views.py</code>	203
<code>org_unit_list_view</code>	SUPER_ADMIN, MINISTRY_ADMIN, AGENCY_MANAGER	<code>organizations/views.py</code>	25
<code>org_unit_create_view</code>	MINISTRY_ADMIN	<code>organizations/views.py</code>	95
<code>org_unit_edit_view</code>	MINISTRY_ADMIN	<code>organizations/views.py</code>	216
<code>org_unit_delete_view</code>	MINISTRY_ADMIN	<code>organizations/views.py</code>	309
<code>audit_log_view</code>	SUPER_ADMIN, MINISTRY_ADMIN, AUDITOR	<code>organizations/views.py</code>	372
<code>audit_log_detail_view</code>	SUPER_ADMIN, MINISTRY_ADMIN, AUDITOR	<code>organizations/views.py</code>	497
<code>master_data_list_view</code>	MINISTRY_ADMIN	<code>organizations/master_data_views.py</code>	21
<code>master_data_create_view</code>	MINISTRY_ADMIN	<code>organizations/master_data_views.py</code>	67
<code>master_data_edit_view</code>	MINISTRY_ADMIN	<code>organizations/master_data_views.py</code>	172
<code>master_data_delete_view</code>	MINISTRY_ADMIN	<code>organizations/master_data_views.py</code>	261

View Function	Roles Allowed	File	Line
master_data_seed_view	MINISTRY_ADMIN	organizations/master_data_views.py	305
asset_category_list_view	MINISTRY_ADMIN	organizations/master_data_views.py	388
asset_category_create_view	MINISTRY_ADMIN	organizations/master_data_views.py	414
asset_category_edit_view	MINISTRY_ADMIN	organizations/master_data_views.py	534
pending_access_list_view	SUPER_ADMIN, MINISTRY_ADMIN	authentication/pending_access_views.py	12
pending_access_review_view	SUPER_ADMIN, MINISTRY_ADMIN	authentication/pending_access_views.py	39
pending_access_clear_view	SUPER_ADMIN only	authentication/pending_access_views.py	142

3. API Permission Classes

What it does: Same as roles but for API endpoints. DRF permission classes that check the user's role before allowing access to each API view.

Where the permission classes are defined:

Evidence	File	Lines
IsSuperAdmin — request.user.role == 'SUPER_ADMIN'	authentication/api_permissions.py	6-11
IsMinistryAdmin — role in ['SUPER_ADMIN', 'MINISTRY_ADMIN']	authentication/api_permissions.py	14-19
IsAgencyManagerOrAbove — role in ['SUPER_ADMIN', 'MINISTRY_ADMIN', 'AGENCY_MANAGER']	authentication/api_permissions.py	22-27
CanManageAssets — read for all authenticated; write requires asset roles	authentication/api_permissions.py	30-39
CanDeleteAssets — DELETE only for SUPER_ADMIN or MINISTRY_ADMIN	authentication/api_permissions.py	42-51
CanViewAuditLogs — role in ['SUPER_ADMIN', 'MINISTRY_ADMIN', 'AUDITOR']	authentication/api_permissions.py	54-59
HasMinistrySchema — blocks users without ministry_schema; Super Admin exempt	authentication/api_permissions.py	62-71

4. Brute Force Protection — Two Independent Systems

What it does: Two separate brute-force lockout systems protect different login paths. Together they cover every way someone can attempt to log in.

System A — Keycloak's built-in brute force (protects the web SSO login page):

Configured in the Keycloak admin console (not in our Python code), at this path:

```
http://localhost:8180 → admin/admin123 → govasset realm → Realm Settings → Security Defenses → Brute Force Detection
```

Setting	Recommended Value	What It Means
Max Login Failures	5	How many wrong passwords before Keycloak locks the account
Wait Increment	60 seconds	Initial wait time after first lockout
Max Wait	900 seconds (15 min)	Maximum lockout duration

Setting	Recommended Value	What It Means
Failure Reset Time	43200 seconds (12 hrs)	How long before the failed count resets to zero

System B — Django's LoginAttempt model (protects the API login endpoint):

Evidence	File	Lines
LoginAttempt model — <code>MAX_ATTEMPTS = 5</code> , <code>LOCKOUT_MINUTES = 15</code>	<code>authentication/models.py</code>	122-164
<code>_is_locked_out()</code> — queries LoginAttempt, returns <code>(True, minutes_remaining)</code> if locked	<code>authentication/api_views.py</code>	128-144
<code>_record_failed_attempt()</code> — increments attempts, sets <code>locked_until</code> when <code>MAX_ATTEMPTS (5)</code> reached	<code>authentication/api_views.py</code>	146-167
Web view helpers — <code>_get_login_attempt()</code> , <code>_record_failed_attempt()</code> , <code>_is_locked_out()</code>	<code>authentication/views.py</code>	89-143

How they interact:

Entry Point	Protected By	What Happens at 5 Failures
Web browser SSO login (Keycloak page)	Keycloak brute force	Keycloak blocks the account for 15 minutes
API login (<code>POST /api/auth/login/</code>)	Django LoginAttempt	Django returns 429 "Account locked" for 15 minutes

Say this to the panel:

"We have two independent brute-force protection systems. Keycloak protects the web SSO login with configurable attempt limits in the Keycloak admin console at Realm Settings → Security Defenses. Our own Django LoginAttempt model protects the API login endpoint used by the mobile app and integrating groups, blocking after 5 failed attempts for 15 minutes. These two systems cover every possible login path independently."

5. Tamper-proof Audit Log

What it does: Once an audit record is created, it CANNOT be edited or deleted — even by a Super Admin. The `save()` and `delete()` methods are overridden to raise `PermissionError`.

Evidence	File	Lines
<code>AuditLog.save()</code> override — <code>if self.pk: raise PermissionError("AuditLog records are immutable and cannot be modified.")</code>	<code>organizations/models.py</code>	149-155
<code>AuditLog.delete()</code> override — <code>raise PermissionError("AuditLog records cannot be deleted.")</code>	<code>organizations/models.py</code>	157-159
<code>SuperAdminAuditLog.save()</code> override — same pattern for platform-level actions	<code>authentication/models.py</code>	209-215
<code>SuperAdminAuditLog.delete()</code> override — raises <code>PermissionError</code>	<code>authentication/models.py</code>	217-220

Every audit record captures the user's IP address. This is how we know not just WHO did something, but FROM WHERE.

6. Session Security

What it does: Session cookie expires when the browser closes, cannot be read by JavaScript, and is only sent over HTTPS in production. This prevents session hijacking via XSS and man-in-the-middle attacks.

Evidence	File	Line
<code>SESSION_EXPIRE_AT_BROWSER_CLOSE = True</code> — session cookie deleted when browser closes	<code>config/settings.py</code>	157

Evidence	File	Line
<code>SESSION_COOKIE_AGE = 28800</code> — 8-hour inactivity timeout (28800 seconds)	<code>config/settings.py</code>	161
<code>SESSION_COOKIE_HTTPONLY = True</code> — JavaScript cannot read the session cookie	<code>config/settings.py</code>	163-165
<code>SESSION_COOKIE_SECURE = config(...)</code> — only sent over HTTPS in production	<code>config/settings.py</code>	167-169
<code>CSRF_COOKIE_HTTPONLY = True</code> — JavaScript cannot read the CSRF cookie	<code>config/settings.py</code>	172

7. JWT Token Security

What it does: Short expiry (30 minutes), rotated refresh tokens, old tokens blacklisted. Even if a JWT is stolen, it can only be used for 30 minutes. Refresh tokens cannot be reused after rotation.

Evidence	File	Line
<code>'ACCESS_TOKEN_LIFETIME': timedelta(minutes=30)</code> — access token expires in 30 minutes	<code>config/settings.py</code>	236
<code>'REFRESH_TOKEN_LIFETIME': timedelta(days=1)</code> — refresh token expires in 1 day	<code>config/settings.py</code>	237
<code>'ROTATE_REFRESH_TOKENS': True</code> — a NEW refresh token is issued on every refresh	<code>config/settings.py</code>	238
<code>'BLACKLIST_AFTER_ROTATION': True</code> — old refresh tokens are blacklisted and cannot be reused	<code>config/settings.py</code>	239
<code>'ALGORITHM': 'HS256'</code> — HMAC-SHA256 signing algorithm	<code>config/settings.py</code>	240
<code>'SIGNING_KEY': SECRET_KEY</code> — uses Django's SECRET_KEY for signing	<code>config/settings.py</code>	241
<code>'AUTH_HEADER_TYPES': ('Bearer',)</code> — standard <code>Authorization: Bearer <token></code> format	<code>config/settings.py</code>	242

8. Clickjacking Protection

What it does: Prevents our pages from being loaded inside an iframe on another website. Stops attackers from tricking users into clicking buttons on an invisible overlay.

Evidence	File	Line
<code>X_FRAME_OPTIONS = 'DENY'</code> — blocks all iframe embedding	<code>config/settings.py</code>	179

9. Content Sniffing Protection

What it does: Tells browsers not to guess (sniff) MIME types. Prevents content-type confusion attacks where an attacker uploads a malicious file disguised as an image.

Evidence	File	Line
<code>SECURE_CONTENT_TYPE_NOSNIFF = True</code>	<code>config/settings.py</code>	182

10. Cross-schema Protection (IntegerField)

What it does: User IDs are stored as `IntegerField` instead of `ForeignKey` because PostgreSQL cannot create foreign key constraints across schemas. Users live in the public schema, but data records (audit logs, org units) live in each ministry's private schema. Using `IntegerField` prevents cross-schema FK errors and keeps the schemas truly independent.

Evidence	File	Line
<code>performed_by_id = models.IntegerField(null=True)</code> in AuditLog (tenant schema → references user in public schema)	<code>organizations/models.py</code>	107-109
<code>manager_id = models.IntegerField(null=True)</code> in OrgUnit	<code>organizations/models.py</code>	32-34
<code>reviewed_by_id = models.IntegerField(null=True)</code> in PendingAccess	<code>authentication/models.py</code>	97-99
<code>performed_by_id = models.IntegerField(null=True)</code> in SuperAdminAuditLog	<code>authentication/models.py</code>	189

11. Pending Access Workflow

What it does: New users cannot just log in — an admin must approve them first. When a user authenticates through Keycloak but has no profile in Django, a `PendingAccess` record is created. The `create_user()` method returns `None` (instead of creating the user), which blocks the login.

Evidence	File	Lines
<code>PendingAccess</code> model with fields: username, email, keycloak_id, status, reviewed_by_id, etc.	<code>authentication/models.py</code>	59-119
<code>create_user()</code> returns <code>None</code> — overrides mozilla-django-oidc default, creates <code>PendingAccess</code> record instead of auto-creating user	<code>authentication/oidc_backend.py</code>	41-76
<code>pending_access_list_view</code> — lists pending requests	<code>authentication/pending_access_views.py</code>	11-35
<code>pending_access_review_view</code> — approve/reject with audit logging to both <code>SuperAdminAuditLog</code> and <code>tenant AuditLog</code>	<code>authentication/pending_access_views.py</code>	38-138

The flow:

```
User logs in via Keycloak → Django checks database → Not found?
  → PendingAccess created → User sees "awaiting approval"
  → Super Admin approves → Django creates user profile
  → User can now log in
```

12. IP Tracking in Audit Logs

What it does: Every audit record captures the user's real IP address with proxy awareness. The `_get_client_ip()` function checks `HTTP_X_FORWARDED_FOR` first (for when the site is behind Nginx or Cloudflare), then falls back to `REMOTE_ADDR`.

Evidence	File	Lines
<code>_get_client_ip()</code> — checks <code>HTTP_X_FORWARDED_FOR</code> , falls back to <code>REMOTE_ADDR</code>	<code>authentication/api_views.py</code>	122-126
Same helper in web views	<code>authentication/views.py</code>	146-151
Same helper in asset web views	<code>assets/views.py</code>	15-22
Same helper in asset API views	<code>assets/api_views.py</code>	20-25
Same helper in org unit views	<code>organizations/views.py</code>	14-21
Same helper in master data views	<code>organizations/master_data_views.py</code>	10-17
IP captured on asset create (web)	<code>assets/views.py</code>	571
IP captured on asset create (API)	<code>assets/api_views.py</code>	269
IP captured on asset update (API)	<code>assets/api_views.py</code>	457
IP captured on asset delete (API)	<code>assets/api_views.py</code>	520
IP captured on org unit create	<code>organizations/views.py</code>	192
IP captured on user create/edit	<code>authentication/user_views.py</code>	531
IP captured on pending access review	<code>authentication/pending_access_views.py</code>	85, 114
IP captured on login	<code>authentication/api_views.py</code>	109
IP captured on logout	<code>authentication/api_views.py</code>	297

13. Password Validation

What it does: Four validators ensure users choose strong passwords — minimum 8 characters, not a common password, not purely numeric, not similar to their username/email/name.

Evidence	File	Lines
<code>UserAttributeSimilarityValidator</code> — password cannot resemble username, email, first_name, last_name	<code>config/settings.py</code>	120-122
<code>MinimumLengthValidator</code> — minimum 8 characters (<code>min_length: 8</code>)	<code>config/settings.py</code>	124-126
<code>CommonPasswordValidator</code> — blocks top 20,000 most common passwords	<code>config/settings.py</code>	127-129
<code>NumericPasswordValidator</code> — blocks purely numeric passwords	<code>config/settings.py</code>	130-132

14. LoginAttempt Brute-force Tracking

What it does: Every login attempt — success or failure — is permanently recorded in the database. Django counts failed attempts from the same username within the last 15 minutes and blocks after 5 failures.

Evidence	File	Line
<code>LoginAttempt</code> model — <code>username</code> , <code>ip_address</code> , <code>attempts</code> , <code>last_attempt</code> , <code>locked_until</code> fields	<code>authentication/models.py</code>	128-136
<code>MAX_ATTEMPTS = 5</code> — constant	<code>authentication/models.py</code>	125
<code>LOCKOUT_MINUTES = 15</code> — constant	<code>authentication/models.py</code>	126
<code>is_locked</code> property — checks if <code>locked_until</code> is in the future	<code>authentication/models.py</code>	148-154
<code>minutes_remaining</code> property — shows how long until unlock	<code>authentication/models.py</code>	157-164
<code>_is_locked_out()</code> — queries the model, returns lock status	<code>authentication/api_views.py</code>	128-144
<code>_record_failed_attempt()</code> — creates/updates <code>LoginAttempt</code> , sets <code>locked_until</code> at 5 failures	<code>authentication/api_views.py</code>	146-167

15. Warning for Default Secret Key

What it does: The `SECRET_KEY` is loaded from an environment variable (not hardcoded). If someone accidentally runs production with the default insecure key (`django-insecure-...`), Django prints a warning in the terminal.

Evidence	File	Line
<code>SECRET_KEY = config('SECRET_KEY')</code> — loaded from <code>.env</code> file, never hardcoded	<code>config/settings.py</code>	11
Warning check: <code>if not DEBUG and 'django-insecure' in SECRET_KEY: print("WARNING...")</code>	<code>config/settings.py</code>	287-293
Warning check: <code>if 'test' in sys.argv and DEBUG: print("WARNING...")</code>	<code>config/settings.py</code>	295-301

Things that might look like issues (but are actually correct)

The panel may notice these things and think they are security problems. Here is the honest explanation for each:

Thing that might look like an issue	The real situation
Django's password fields are empty/null for Keycloak users	This is correct — passwords live in Keycloak, not Django. Django's password is set to an unusable value
PendingAccess records exist with no role assigned	These are users Keycloak knows about but Django hasn't approved yet. Normal and expected
LoginAttempt records keep growing	This is correct — they are a permanent security record and should never be deleted
Web "forgot password" might still point to Django's reset page	This is a real issue to fix — redirect it to Keycloak's forgot-password page instead
<code>/api/auth/login/</code> can be called without a token	This is correct — you obviously cannot require a token BEFORE logging in
Some endpoints return 403 (Forbidden) instead of 404	This is intentional — returning 404 would reveal whether the URL exists; 403 is more secure
Multiple failed login attempts from the same IP	Both Keycloak and Django track and block these after 5 failures

PART 9: DATABASE DESIGN — How our tables are organized

Public schema (shared across ALL ministries):

```
tenants_ministry    → List of all ministries (MOH, MOF, etc.)
tenants_domain      → Domain to ministry mapping (moh.localhost → MOH)
authentication_customuser → All users from all ministries
django_session      → Login sessions
...
```

Each ministry schema (one per ministry):

```
assets_asset        → Assets belonging to this ministry
assets_assetcategory → Asset categories (ICT, Furniture, etc.)
organizations_orgunit → Org hierarchy (Ministry → Agency → Facility)
organizations_masterdata → Dropdown options (funding sources, etc.)
organizations_auditlog → Audit trail for this ministry
...
```

How Do Public and Private Schemas "Join"?

Question: "How do public and private schemas join? What is the point of each?"

They do NOT join in the traditional SQL sense. That is actually the entire security point.

django-tenants switches the PostgreSQL `search_path` on every connection — meaning when a request from `moh_admin` comes in, the database connection is pointed **exclusively at moh_schema**. Every query from that point runs inside `moh_schema` only. It is physically impossible for that query to accidentally touch `mof_schema` because the database itself does not know `mof_schema` exists during that connection.

What the public schema holds:

The public schema is the **shared lobby** of your database. It contains tables that must exist across ALL ministries or belong to the platform itself, not to any one ministry:

```
public schema contains:
├─ tenants_ministry    (the list of all tenant ministries)
├─ tenants_domain      (the web addresses mapped to each ministry)
```

```

├─ authentication_customuser (ALL users across all ministries – so login can work)
├─ authentication_pendingaccess (access requests before users are assigned to a ministry)
├─ authentication_loginattempt (brute-force tracking across all users)
├─ django_session (login sessions)
└─ ... (other shared tables)

```

What each private schema holds:

Each ministry's private schema contains tables whose data must **never be shared with or visible to other ministries**:

```

moh_schema contains:
├─ assets_asset (Ministry of Health's assets ONLY)
├─ assets_assetcategory (MOH's categories ONLY)
├─ organizations_orgunit (MOH's hierarchy ONLY)
├─ organizations_auditlog (MOH's action history ONLY)
├─ organizations_masterdata (MOH's reference data ONLY)
└─ ...

mof_schema contains:
├─ EXACT SAME TABLE NAMES but COMPLETELY DIFFERENT DATA
└─ ...

```

Where public and private DO interact:

There is ONE case where public and private data interact: **user authentication**.

```

When someone logs in:
1. Django looks up their user record in the PUBLIC schema's CustomUser table
   → finds their role and ministry_schema (e.g., "moh_schema")
2. Django switches the connection to their ministry's PRIVATE schema
3. All subsequent data access runs inside that private schema only

```

This is the controlled, intentional bridge between public and private.

Why not put everything in one table with a `ministry_id` filter?

If everything were in one table with a `ministry_id` column, a single coding mistake — forgetting to add `.filter(ministry_id=current_ministry)` to one query — would expose ALL ministries' data to each other. Schema isolation means even a buggy query cannot cross the boundary because the other schemas literally do not exist in the current connection's view.

Say this to the panel:

"The public schema holds platform-wide identity data — users, domains, and ministry records — because login must work before we know which ministry a user belongs to. Each ministry's private schema holds their operational data — assets, audit logs, categories — because that data must never be accessible to another ministry. Django-tenants enforces this by switching the PostgreSQL search path at the database connection level on every request, which means it is not a software filter that can be accidentally bypassed — it is enforced by the database engine itself."

PART 10: OTHER TECHNOLOGIES — What they are and where we use them

Django REST Framework (DRF)

What: A library that makes building APIs easy. **Where:** Every `api_views.py` file uses DRF's `APIView`, `Response`, and `status` classes.

Why: Writing JSON APIs by hand is tedious. DRF handles JSON conversion, error handling, authentication, and permissions automatically.

django-cors-headers

What: Allows the Flutter mobile app (running on a phone) to call our Django API (running on a laptop). **Where:** `MIDDLEWARE` includes `corsheaders.middleware.CorsMiddleware`. Settings include `CORS_ALLOW_ALL_ORIGINS`. **Why:** Without CORS, the mobile browser would block API calls because they come from different addresses.

mozilla-django-oidc

What: Handles the Keycloak OIDC handshake (redirects, code exchange, token verification). **Where:** `urlpatterns` includes `path('oidc/', include('mozilla_django_oidc.urls'))`. Settings include `OIDC_OP_*_ENDPOINT` URLs. **Why:** Writing OIDC from scratch is extremely complex and error-prone. Mozilla (the Firefox company) maintains this library.

drf-yasg

What: Generates Swagger API documentation automatically. **Where:** `/api/docs/` in the browser shows the documentation page. **Why:** Other groups (Group 2-10) need to understand our API. Instead of writing documentation manually, drf-yasg reads our code and creates an interactive page.

djangorestframework-simplejwt

What: Creates and verifies JWT tokens for the mobile app. **Where:** `SIMPLE_JWT` settings in settings.py. Used in `LoginAPIView`, `RefreshTokenAPIView`, etc. **Why:** JWT is the standard way for mobile apps to authenticate. SimpleJWT handles token creation, refresh, blacklisting, and verification.

python-decouple

What: Reads secrets from `.env` file. **Where:** `SECRET_KEY = config('SECRET_KEY')` in settings.py. **Why:** You never hardcode passwords or secret keys in your code. If someone views your code on GitHub, they can't steal your secrets.

requests

What: A library for making HTTP calls from Python. **Where:** Used to call Keycloak Admin API (creating users in Keycloak from Django). **Why:** Django needs to communicate with Keycloak server-to-server.

pytest-django

What: A testing framework. **Where:** Currently not used extensively (test files are placeholders). **Why:** Allows writing automated tests that verify the system works correctly.

PART 11: DEPLOYMENT TO CLOUD — What, where, how

What is "the cloud"?

The cloud = someone else's computer that runs 24/7.

Instead of running `python manage.py runserver` on your laptop (which you turn off at night), you rent a server from a company like:

- **AWS** (Amazon Web Services)
- **DigitalOcean** (\$5/month basic server)
- **Linode** or **Vultr**

What you need for production:

```
[User's Browser]
  ↓
[Domain Name: moh.mof.go.tz]
  ↓
[DNS — points domain to server IP]
  ↓
[NGINX or Apache — web server]
  (handles HTTPS certificates, static files, security)
  ↓
[Gunicorn or uWSGI — runs Django]
```

↓
[PostgreSQL Database]

Do you use 0.0.0.0 in production?

NO. In production:

- NGINX listens on **0.0.0.0:80** (HTTP) and **0.0.0.0:443** (HTTPS with SSL)
- Django runs behind NGINX, listening **ONLY** on **127.0.0.1:8000**
- NGINX forwards (proxies) requests from the internet to Django

The 0.0.0.0:8000 thing is **ONLY** for your development testing when you access Django from your phone.

What about domains?

In production:

```
https://moh.mof.go.tz    → Routes to Ministry of Health's schema
https://mof.mof.go.tz    → Routes to Ministry of Finance's schema
https://admin.mof.go.tz  → Routes to public schema (Super Admin)
```

Each domain gets an SSL certificate (for HTTPS — the padlock icon).

PART 12: JWT TOKENS — Access token, refresh token, how they work

What is a JWT token?

JWT stands for **JSON Web Token**. It is a string of text that proves who you are. Think of it like a **digital ID card** that the mobile app carries around.

The mobile app logs in once, gets this ID card, and shows it with every request so Django knows: "Oh, this is user X, they are a Ministry Admin for MOH."

What does a JWT look like?

```
eyJhbGciOiJIUzI1NiJ9.eyJ1c2VyX2lkIjoxfQ.s5xLm8vF9kZn6w
|-----|-----|-----|
| PART 1 | PART 2 | PART 3 |
| HEADER | PAYLOAD | SIGNATURE |
```

Three parts separated by dots:

Part	Name	What it contains	Can you read it?
First part	Header	What type of token and how it was signed (e.g., HS256)	Yes (base64 decoded)
Second part	Payload	The actual data: user_id, role, ministry_schema, expiration time	Yes (base64 decoded)
Third part	Signature	A cryptographic seal that proves the token was issued by Django and not forged	No (requires secret key)

Important: Parts 1 and 2 are just encoded (base64), NOT encrypted. Anyone can decode them and see the contents. But they **CANNOT** change them because the signature (part 3) would break.

How to SEE inside a JWT token:

Go to jwt.io and paste your token. It will show you:

```
// HEADER
{
  "alg": "HS256",
  "typ": "JWT"
}

// PAYLOAD
{
  "user_id": 1,
  "role": "MINISTRY_ADMIN",
  "ministry_schema": "moh_schema",
  "exp": 1712345678,    // ← Expiration time (Unix timestamp)
  "iat": 1712343878    // ← Issued at time
}
```

Access token vs Refresh token

We have TWO tokens, not one:

```
ACCESS TOKEN (short life)
  Purpose: Proves who you are for API calls
  Valid for: 30 minutes
  Used in: Every API request header
  What if stolen? Only useful for 30 minutes

REFRESH TOKEN (longer life)
  Purpose: Gets you a NEW access token when the old one expires
  Valid for: 1 day
  Used in: POST /api/auth/refresh/
  What if stolen? Can be blacklisted (invalidated)
```

The flow:

```
Mobile app logs in
→ Gets access token (30 min) + refresh token (1 day)
→ Uses access token for 30 minutes of API calls
→ Token expires (30 min passed)
→ Mobile app calls: POST /api/auth/refresh/ with refresh token
→ Django gives a NEW access token (another 30 min)
→ Sometimes also gives a NEW refresh token (ROTATE_REFRESH_TOKENS = True)
→ Old refresh token is blacklisted (cannot be used again)
→ Cycle continues until 1 day passes
→ After 1 day, user must log in again
```

Code evidence — Where we configure token lifetimes:

```
# config/settings.py — LINE 235-248
from datetime import timedelta

SIMPLE_JWT = {
    'ACCESS_TOKEN_LIFETIME': timedelta(minutes=30),    # ← 30 minutes
    'REFRESH_TOKEN_LIFETIME': timedelta(days=1),      # ← 1 day
    'ROTATE_REFRESH_TOKENS': True,                   # ← Give new refresh token each time
    'BLACKLIST_AFTER_ROTATION': True,                 # ← Old one stops working
    'ALGORITHM': 'HS256',                             # ← How we sign it
    'SIGNING_KEY': SECRET_KEY,                       # ← Uses Django's secret key
```



```
'AUTH_HEADER_TYPES': ('Bearer',), # ← Token prefix in header
}
```

How to change the token duration (if panel asks):

```
# Change 30 minutes to 2 hours:
'ACCESS_TOKEN_LIFETIME': timedelta(hours=2)

# Change 1 day to 7 days:
'REFRESH_TOKEN_LIFETIME': timedelta(days=7)
```

Why 30 minutes? Why not 1 year?

Short access tokens = more secure. If someone steals your token, they can only use it for 30 minutes. After that, it's useless. The refresh token lasts longer but can be blacklisted if stolen.

Code evidence — Where the token is created with extra data:

```
# authentication/api_serializers.py – LINE 7-18
class CustomTokenObtainPairSerializer(TokenObtainPairSerializer):
    @classmethod
    def get_token(cls, user):
        token = super().get_token(user)
        # These extra fields are embedded INSIDE the token
        token['role'] = user.role
        token['ministry_schema'] = user.ministry_schema or ''
        token['full_name'] = user.get_full_name() or user.username
        return token
```

This means when the mobile app decodes the token, it can read the user's role and ministry without making an extra API call.

Code evidence — Mobile app login endpoint:

```
# authentication/api_urls.py – LINE 18
path('auth/login/', LoginAPIView.as_view(), name='api_login'),
# POST /api/auth/login/ → returns { access, refresh, user }
```

Code evidence — How the login API creates the token

When the mobile app or other group calls `POST /api/auth/login/`, this code runs:

```
# authentication/api_views.py
class LoginAPIView(APIView):
    permission_classes = [AllowAny] # No token needed to log in

    def post(self, request):
        username = request.data.get('username')
        password = request.data.get('password')

        user = authenticate(username=username, password=password)
        if user:
            # Create the token with extra info baked in
            refresh = RefreshToken.for_user(user)
            refresh['role'] = user.role # Role goes INTO the token
            refresh['ministry_schema'] = user.ministry_schema # Ministry too

            return Response({
```

```

        'access': str(refresh.access_token),
        'refresh': str(refresh),
        'user': {
            'username': user.username,
            'role': user.role,
            'ministry_schema': user.ministry_schema
        }
    })
    return Response({'error': 'Invalid credentials'}, status=401)

```

What this means: When a user logs in, Django:

1. Checks the username/password (authenticate)
2. Creates a JWT token with `RefreshToken.for_user(user)`
3. Adds `role` and `ministry_schema` as extra fields INSIDE the token
4. Returns the token AND the user info in one response

The Flutter app or other group never needs to make a second call to find out the user's role — it's already in the token.

PART 13: INTEGRATION WITH OTHER GROUPS — How they use our API

Our role

We are Group 1. Groups 2-10 depend on us. They need to call our API to:

- Verify users (is this token valid? what role do they have?)
- Get asset data for their systems
- Get organization data

What do we share with them?

We share our **API endpoint URLs** and a **sample token**. We do NOT share our source code or database.

Scenario 1: They just need to check if a user is real

Their system receives a token from a mobile user. They call our API to verify it.

What we give them:

API Endpoint: POST `http://moh.platform.go.tz/api/auth/verify-token/`

Headers:

Authorization: Bearer <the_token_their_user_provided>
Content-Type: application/json

Response (if valid):

```

{
  "valid": true,
  "role": "MINISTRY_ADMIN",
  "ministry_schema": "moh_schema",
  "full_name": "Amina Hassan",
  "email": "amina@moh.go.tz"
}

```

Response (if invalid/expired):

```

{
  "valid": false,
  "message": "Token is invalid or expired"
}

```

They don't need our database. They don't need Django. They just need to make an HTTP request to our URL.

Scenario 2: They want asset data for reporting

What we give them:

```
API Endpoint: GET http://moh.platform.go.tz/api/assets/?status=ACTIVE

Headers:
  Authorization: Bearer <valid_token>

Response:
{
  "count": 150,
  "results": [
    {"id": 1, "asset_number": "MOH-ICT-0001", "name": "Dell Laptop", ...},
    ...
  ]
}
```

Do we give them a token or do they get their own?

They get their own token. Here's how:

1. We create a user account for their system (like "group2_service_account")
2. We give them: username + password
3. Their system calls: POST /api/auth/login/ with those credentials
4. Our API returns: { access_token, refresh_token, user_info }
5. They use the access_token in all subsequent API calls
6. When token expires, they call /api/auth/refresh/ to get a new one

The simple analogy:

The **API** is the door. The **token** is the key to that door. You share the **door** (API URL) with other groups. They get their **own key** (token) by logging in themselves.

You do NOT hand them a token directly. You never say "here is your token." That would be like giving someone your house key — they'd have YOUR identity, not their own.

Here is exactly what you give them (one time, via WhatsApp or email):

1. Our server address: http://172.16.20.232:8000
2. Our API docs link: http://172.16.20.232:8000/api/docs/
3. A test account: username: moh_admin
password: Admin@123

And here is what THEY do with it:

```
Step 1 – They call your login API themselves:
POST http://172.16.20.232:8000/api/auth/login/
{"username": "moh_admin", "password": "Admin@123"}

Step 2 – Your system gives THEM their own token:
{"access": "eyJ...their token...", "user": {"role": "MINISTRY_ADMIN"}}

Step 3 – They use that token on every request:
GET http://172.16.20.232:8000/api/assets/
Header: Authorization: Bearer eyJ...their token...
```

So the flow is: You give them the door address → they knock (login) → the system gives them their own key → they use that key on every future knock.

Do other groups go to the Keycloak login page?

No. This is important to understand.

Keycloak's login page is only for **web browser users** — real ministry staff using the website. The journey for web users is:

Web browser user → gets redirected to Keycloak login page → types password there

Other groups are **developers building systems**, not humans typing passwords in a browser. They use our **API login endpoint** directly (the `POST /api/auth/login/` shown above). That endpoint checks the username and password internally and gives back a JWT token. Keycloak's web page never appears.

Think of it this way:

- Keycloak's page = the front door of a government office (for real staff walking in)
- API login endpoint = a back door for systems talking to systems (no human needed)

Both doors lead to the same building — but other groups use the back door.

What if they want a public API (no token required)?

You already have this ability. Look at your views — some use `AllowAny`, some use `IsAuthenticated`:

```
# PUBLIC – no token needed (currently only the login endpoint):
class LoginAPIView(APIView):
    permission_classes = [AllowAny]

# PROTECTED – token required:
class AssetListCreateAPIView(APIView):
    permission_classes = [IsAuthenticated]
```

If another group asks you to make a specific endpoint public — for example, a read-only list of asset categories — you change one line:

```
class AssetCategoryListAPIView(APIView):
    permission_classes = [AllowAny] # ← Change this line
```

Should you make things public? Only read-only, non-sensitive data. Never user data, never audit logs, never financial data. The rule is: if the data could embarrass the government or reveal security information, it must stay protected.

What if they use different technology?

It doesn't matter. Our API speaks HTTP + JSON. Every programming language can do this:

Their technology	How they call our API
Python	<code>requests.get(url, headers={'Authorization': 'Bearer ...'})</code>
PHP	<code>curl</code> or <code>file_get_contents</code>
Java	<code>URLConnection</code> or <code>OkHttp</code>
JavaScript	<code>fetch(url, {headers: {'Authorization': 'Bearer ...'}})</code>
C# .NET	<code>HttpClient</code>
Any language	HTTP is universal — every language supports it

Sample Python code they would write:

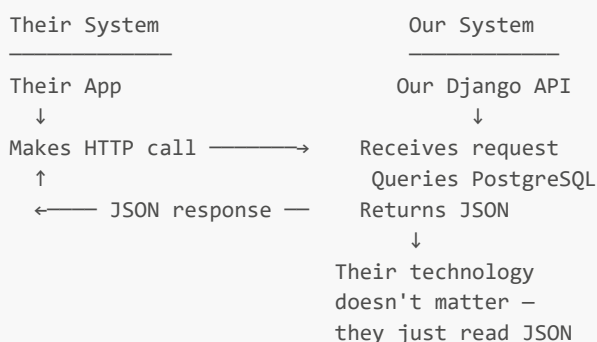
```
import requests

# Step 1: Login to get a token
login_response = requests.post(
    'http://moh.platform.go.tz/api/auth/login/',
    json={'username': 'group2_user', 'password': 'given_password'}
)
token = login_response.json()['access']

# Step 2: Use the token to get asset data
headers = {'Authorization': f'Bearer {token}'}
assets_response = requests.get(
    'http://moh.platform.go.tz/api/assets/',
    headers=headers
)
assets = assets_response.json()
print(f"Total assets: {assets['count']}")
```

What if they don't use PostgreSQL?

They don't need to. Our database is inside our system. They never connect to our database. They only connect to our API (which is an HTTP endpoint). Our API queries our PostgreSQL database and returns JSON. They receive JSON. Their database can be anything — MySQL, MongoDB, SQL Server, or even Excel.



What if they need a new feature or data?

They tell us: "We need to get all assets that expired this month." We add a new API endpoint or a filter parameter. We update the Swagger documentation at </api/docs/>. They check the documentation and start using the new endpoint.

What happens if we create a user but they don't add it?

They will call </api/auth/verify-token/> with that user's token and get back: `{"valid": false, "message": "User not found"}`. They will know the user doesn't exist in our system.

What happens if they add a user but we don't?

The user will authenticate at Keycloak successfully, but when Keycloak redirects to Django, Django will say: "This user is not registered in our system." A PendingAccess record will be created and the user will see: "Your account is not yet registered. Contact your administrator."

Where is our API documentation?

```
http://localhost:8000/api/docs/
```

This is a Swagger UI page that shows EVERY endpoint with:

- The URL path (e.g., /api/assets/)
- What HTTP method to use (GET, POST, etc.)
- What data to send (JSON format)
- What you get back (JSON format)
- A "Try it out" button to test the API live

We tell other groups: "Go to this URL. All our APIs are documented there."

How is this page automatically created? In your `settings.py` and `config/urls.py`, you have `drf-yasg` installed, which reads your code and builds the documentation page:

```
# config/settings.py - drf_yasg is in INSTALLED_APPS
INSTALLED_APPS = [
    ...
    'drf_yasg', # ← This is what creates the automatic documentation
]

# config/urls.py - The Swagger page configuration
from drf_yasg.views import get_schema_view
from drf_yasg import openapi

schema_view = get_schema_view(
    openapi.Info(
        title="GovAsset Platform API",
        default_version='v1',
        description="REST API for the Government Asset Management Platform",
    ),
    public=True, # Anyone can view the docs (no login required)
)

urlpatterns += [
    path('api/docs/', schema_view.with_ui('swagger')),
]
```

Every time you add a new API endpoint, Swagger automatically adds it to the documentation page. You never write documentation manually.

This means: If another group asks "Can you add an API for X?" you:

1. Add the new endpoint to `api_urls.py` and `api_views.py`
2. Restart the server
3. The new endpoint appears on the Swagger page automatically
4. Tell them: "Check the docs again, it's there now"

What credentials do we give other groups for testing?

```
API Base URL: http://192.168.100.18:8000/api/
Username: test_group_user
Password: Test@123
```

They use these to log in and get a token. Simple.

Who gives the token to other groups? Django or Keycloak?

Django gives the token. Here is why:

```
Token type: JWT (JSON Web Token)
Issued by: Django (using SimpleJWT library)
```

Token contains: user_id, role, ministry_schema, expiration time
Signed with: Django's SECRET_KEY

Keycloak also issues tokens, but those Keycloak tokens are ONLY for web browser login. The tokens we give other groups for API access are Django tokens, created by SimpleJWT.

Important: Keycloak tokens and Django JWT tokens are DIFFERENT. They have different formats, different signatures, and different purposes:

KEYCLOAK TOKEN (for web login)

- Used between: Browser → Django → Keycloak
- Validates: "This user logged in through Keycloak"
- Format: OIDC id_token (longer, more complex)

DJANGO JWT TOKEN (for API access)

- Used between: Mobile app → Django, Other groups → Django
- Validates: "This user has permission to call our API"
- Format: SimpleJWT access_token (shorter, simpler)

What does an actual API endpoint look like? (Real example from our code)

When another group's developer asks "What URL do I call?" — here is exactly what we give them:

BASE URL (during development):
`http://192.168.100.18:8000/api/`

BASE URL (in production):
`https://moh.platform.go.tz/api/`

ENDPOINT: Get list of assets
`GET /api/assets/`

ENDPOINT: Get one specific asset
`GET /api/assets/{id}/`
Example: `GET /api/assets/5/`

ENDPOINT: Create a new asset
`POST /api/assets/`

ENDPOINT: Update an asset
`PUT /api/assets/{id}/`

ENDPOINT: Delete an asset
`DELETE /api/assets/{id}/`

But we don't give them all endpoints individually! We give them ONE URL:

"Go to `http://192.168.100.18:8000/api/docs/` — all our APIs are listed there with a 'Try it out' button."

The Swagger page (`/api/docs/`) automatically lists EVERY endpoint that exists. If we add a new endpoint tomorrow, it appears on the Swagger page automatically. The other group does NOT need us to send them a new document.

What attributes/fields do other groups need to send? (How do they know?)

Again, Swagger tells them. Here is what they see on the Swagger page for "Create Asset":

`POST /api/assets/`

Request body (JSON):

```
{
  "name": "Dell Latitude 5420",           // Required – the asset name
  "asset_number": "MOH-ICT-0042",        // Required – unique ID
  "category": 3,                         // Required – category ID from dropdown
  "status": "ACTIVE",                    // Required – one of: ACTIVE, UNDER_MAINTENANCE,
  DECOMMISSIONED, DISPOSED
  "description": "Laptop for ICT officer", // Optional
  "serial_number": "SN123456789",         // Optional
  "purchase_date": "2024-01-15",          // Optional – format: YYYY-MM-DD
  "purchase_cost": "2500000.00",          // Optional – decimal number as string
  "location": "MOH HQ, Room 305",         // Optional
  "assigned_to": "amina@moh.go.tz",       // Optional – user email
  "org_unit": 12                          // Optional – org unit ID
}
```

The Swagger page also shows:

- Which fields are required vs optional
- What data type each field is (string, number, date)
- What the valid values are for dropdown fields
- Example values
- The exact JSON format

How do other groups access our API during development?

During development, we are all on the same college/local WiFi. Here is the setup:

```
OUR LAPTOP (Group 1):
  IP: 192.168.100.18
  Running: python manage.py runserver 0.0.0.0:8000
  API available at: http://192.168.100.18:8000/api/

OTHER GROUP'S LAPTOP (Group 2-10):
  Also on the same WiFi
  They open their browser/postman/terminal
  They call: GET http://192.168.100.18:8000/api/assets/
  They get back: JSON data
```

What if our IP changes? We tell them: "My IP changed, here is the new one: <http://192.168.100.XXX:8000/api/>"

What if we are on different WiFi? We cannot access each other. We would need to either:

1. Both connect to the same WiFi (college network)
2. Use a tool like ngrok (gives us a public URL that forwards to our laptop)
3. Deploy to a cloud server (production mode)

Complete example: A group wants to integrate with us from scratch

Here is the ENTIRE process from start to finish:

```
STEP 1: We meet with the other group
  Them: "We want to display our ministry's assets in our dashboard."
  Us: "OK. Here is our API base URL and your test credentials."

STEP 2: We give them:
  API Base URL: http://192.168.100.18:8000/api/
  Username: group2_service_account
  Password: Test@123
  Swagger Docs: http://192.168.100.18:8000/api/docs/
```


STEP 3: They test the API manually (in their browser or Postman)
 They open: `http://192.168.100.18:8000/api/docs/`
 They click "Try it out" on `GET /api/assets/`
 They see the JSON response with actual data
 They understand the format

STEP 4: They write code in THEIR system
 Let's say they use PHP (their system is built in PHP).

They write this code:

```
```php
<?php
// Step 1: Login to get token
$login = http_post('http://192.168.100.18:8000/api/auth/login/', [
 'username' => 'group2_service_account',
 'password' => 'Test@123'
]);
$token = json_decode($login)->access;

// Step 2: Get assets
$assets = http_get('http://192.168.100.18:8000/api/assets/', [
 'Authorization: Bearer ' . $token
]);
$data = json_decode($assets);

// Step 3: Display in their dashboard
foreach ($data->results as $asset) {
 echo $asset->name . " - " . $asset->status;
}
?>
```

STEP 5: They test. It works. They are integrated.

STEP 6 (later): We deploy to production. We tell them: "The new URL is `https://moh.platform.go.tz/api/`" They update one line in their code. Everything still works.

### Do we also give them access to Keycloak?

**\*\*No.\*\*** Other groups do NOT need Keycloak access. Here is why:

For API access, other groups use: Username + Password → Django → Django JWT token

They NEVER go through Keycloak for API access. They NEVER need to log into Keycloak admin. They NEVER need Keycloak credentials.

Keycloak is ONLY for web browser SSO login. API access uses Django's own JWT authentication.

### What if our system is down? How does it affect other groups?

Their System → calls our API → no response (our server is down) → Their dashboard shows "Service unavailable" → Their users see a timeout error → They cannot get fresh data until our system is back up → They CAN use their last cached data (if they saved it)

This is why we need production deployment (24/7 server). During development, we only run when we are testing.

### How does the mobile (Flutter) app use our API?

The Flutter app is ALSO an "other group" – it just happens to be built by our team, not another group. The Flutter app:

1. Has our API base URL hardcoded: `http://192.168.100.18:8000/api/`
2. Shows a login screen: username + password fields
3. Calls `POST /api/auth/login/` with those credentials
4. Gets back a JWT token (access + refresh)
5. Stores the token on the phone (secure storage)
6. Adds Authorization: Bearer to EVERY request
7. When token expires, silently refreshes it
8. If refresh fails, shows login screen again

The Flutter app NEVER opens a browser to Keycloak. It uses the direct API login (same as other groups would).

### Complete list of ALL API endpoints other groups can use

These are defined in ``authentication/api_urls.py`` and ``assets/api_urls.py``:

Method	URL	Purpose
POST	<code>`/api/auth/login/`</code>	Get JWT token (username + password)
POST	<code>`/api/auth/refresh/`</code>	Get new access token (using refresh token)
POST	<code>`/api/auth/logout/`</code>	Blacklist token (log out)
POST	<code>`/api/auth/verify-token/`</code>	Check if a token is valid
GET	<code>`/api/auth/profile/`</code>	Get current user's profile
GET/POST	<code>`/api/assets/`</code>	List all assets / Create new asset
GET/PUT/DELETE	<code>`/api/assets/{id}/`</code>	Get / Update / Delete one asset
GET	<code>`/api/org-units/`</code>	List organization units
GET	<code>`/api/dashboard/stats/`</code>	Dashboard statistics
GET	<code>`/api/audit-log/`</code>	View audit log entries

All these are documented at ``/api/docs/`` with example requests and responses.

### Where is the API configuration in our code?

The API routing is configured in TWO files:

**\*\*File 1 – ``authentication/api_urls.py`` (lists every API endpoint):\*\***

```
```python
# This file maps URLs to their handler views
urlpatterns = [
    # Authentication
    path('auth/login/', LoginAPIView.as_view()),
    path('auth/logout/', LogoutAPIView.as_view()),
    path('auth/refresh/', RefreshTokenAPIView.as_view()),
    path('auth/verify-token/', VerifyTokenAPIView.as_view()),
    path('auth/me/', MeAPIView.as_view()),

    # Assets
    path('assets/', AssetListCreateAPIView.as_view()),
    path('assets/<int:asset_id>', AssetDetailAPIView.as_view()),
    path('assets/categories/', AssetCategoryListAPIView.as_view()),

    # Organization
    path('org-units/', OrgUnitListAPIView.as_view()),

    # Audit logs
    path('audit-logs/', AuditLogListAPIView.as_view()),
```

```
# Dashboard statistics
path('dashboard/stats/', DashboardStatsAPIView.as_view()),
]
```

File 2 — `config/urls.py` (connects the API to the main project):

```
urlpatterns = [
    # ... all the web page URLs ...

    # This ONE line attaches ALL API endpoints:
    path('api/', include('authentication.api_urls')),

    # Swagger documentation
    path('api/docs/', schema_view.with_ui('swagger')),
]
```

The line `path('api/', include(...))` puts `api/` in front of every URL in `api_urls.py`. So `auth/login/` becomes `/api/auth/login/`. That is why all your APIs start with `/api/`.

How other groups see this: They never look at these files. They just go to `/api/docs/` and see everything listed automatically.

PART 14: Why does each ministry need its own domain?

Question: "Why can't we use one domain like `platform.go.tz/ministry/1/`?"

Answer: Because django-tenants works by reading the DOMAIN from the URL, not the PATH.

How django-tenants identifies the tenant:

```
URL: https://moh.platform.go.tz/assets/
           ↑
    django-tenants reads this part
    Looks up "moh.platform.go.tz" in Domain table
    Finds: this belongs to Ministry of Health
    Switches database to moh_schema
```

Why not use URL path instead? (e.g., `/ministry/1/`)

```
# This would NOT work with django-tenants:
URL: https://platform.go.tz/ministry/1/assets/
           ↑
    django-tenants doesn't look here
    It ONLY looks at the domain name
```

django-tenants is designed specifically for the "each tenant has its own domain" model. This is actually the standard for SaaS products:

- `company1.salesforce.com`
- `company2.slack.com`
- `moh.github.com`

But we also handle IP access:

For mobile API access, users hit our server by IP address (not domain). We set:

```
SHOW_PUBLIC_IF_NO_TENANT_FOUND = True
```

When Django can't find a matching domain (because the user connected via IP), it falls back to the public schema. This is why the mobile app works even without domains.

PART 15: PRESENTATION TIPS — How to present well

Before the panel:

1. **Run the project** — Make sure `python manage.py runserver 0.0.0.0:8000` works. Show it running.
2. **Have test data** — Create a test ministry, test user, test assets so you can click through the UI.
3. **Open Keycloak admin** — Show Keycloak's admin page (<http://localhost:8180>) with users listed there.
4. **Open PostgreSQL** — Show the database schemas: `\dn` in `psql` shows `moh_schema`, `mof_schema`, etc.

Common panel questions and how to answer:

Q: "What problem does your system solve?"

"Currently, government ministries track assets on paper or Excel. There is no central system. When an asset moves between ministries, there is no record. Our system gives each ministry its own secure space to track assets, with a shared super admin who can see everything."

Q: "Why PostgreSQL and not MySQL?"

"Because only PostgreSQL supports schemas. We need schemas to give each ministry its own private set of tables. MySQL doesn't have this feature."

Q: "How is data kept separate between ministries?"

"Each ministry has its own PostgreSQL schema — basically its own set of tables. When a user from Ministry of Health logs in, the system only connects to MOH's schema. Even if there's a bug in the code, the database won't show another ministry's data."

Q: "What if one ministry has millions of assets?"

"Each schema is independent. MOH having 2 million assets doesn't slow down MOF's queries. Each ministry's tables only contain their own data."

Q: "How does the mobile app authenticate?"

"The mobile app calls our API with username and password. Django returns a JWT token that expires in 30 minutes. The mobile app sends this token with every request. When it expires, they use a refresh token to get a new one."

Q: "What happens if Keycloak is down?"

"Users who are already logged in can continue. New users cannot log in. The mobile app's existing tokens continue to work for 30 minutes. After that, they cannot refresh."

Q: "What is the most important security feature?"

"Database-level isolation. Each ministry's data lives in a separate PostgreSQL schema. Even if someone finds a bug in our code, the database itself prevents cross-ministry access."

Q: "How do you know who changed an asset?"

"Every action — create, update, delete, login, logout — is recorded in the AuditLog. The record cannot be modified or deleted. It captures who did it, what they changed, the old values, and the new values."

Q: "How do you add a new ministry?"

"The Super Admin fills a form with the ministry name, schema name, and domain. When they click submit, Django creates a new PostgreSQL schema, runs all migrations inside it, creates the root organization unit, and sets up the domain mapping. The whole process takes about 2 seconds."

If you don't know an answer:

"That's a good question. I have not tested that scenario yet. Based on how the system is designed, I believe it would work like this: [your best guess]. But I would need to verify that before giving a confident answer."

This is honest and shows understanding without lying.

For the demo:

1. Start with the **login page** — show the SSO button
2. Log in as **Super Admin** — show ministry list
3. Show **onboarding a new ministry** — explain schemas
4. Show **asset list** — demonstrate search, filter, pagination
5. Show **audit log** — demonstrate tamper-proof records
6. Show **Swagger API docs** (/api/docs/) — demonstrate the API
7. Show **PostgreSQL** — run `\dn` to show schemas

Browser cache issue — CSS not loading (important!)

What happened to you: Your browser saved (cached) the CSS file locally from a previous visit. Even after restarting your PC, the browser still used the SAVED CSS from its cache instead of fetching the fresh CSS from the server. This is why Ctrl+F5 (hard refresh) fixed it — it tells the browser to IGNORE the cache and download everything fresh.

Why restarting the PC didn't work: The browser cache is stored on your hard drive. Restarting the PC does NOT clear the browser cache. Only clearing the cache or a hard refresh removes it.

How to fix it during a presentation (if CSS breaks):

1. **Quick fix:** Press **Ctrl+Shift+R** (Windows) or **Cmd+Shift+R** (Mac) on the page
2. **Alternative:** Open your browser's Developer Tools (F12) → right-click the refresh button → "Empty Cache and Hard Reload"
3. **Nuclear option:** Chrome → Settings → Privacy and security → Clear browsing data → Check "Cached images and files" → Clear data

Why this happens more with CSS than HTML:

- HTML pages are usually served with **no-cache** headers (always check for updates)
- CSS files are often served with cache headers that say "keep this for days/weeks"
- When you change the CSS, the browser doesn't know — it keeps using the old saved version

To prevent this in the future: When you update the CSS file, you can also restart the Django server (Ctrl+C then `python manage.py runserver`). Django's development server automatically adds cache-busting headers in debug mode, which helps — but sometimes the browser's cache still wins.

PART 16: QUICK REFERENCE — Key files and what they do

File	What it does	Why it exists
<code>config/settings.py</code>	All configuration — apps, database, security, auth	The brain of the project
<code>config/urls.py</code>	Maps URLs to view functions	The roadmap — matches URLs to code
<code>tenants/models.py</code>	Ministry and Domain models	Defines what a tenant (ministry) is
<code>tenants/views.py</code>	Web pages for managing ministries	Super Admin manages ministries here
<code>authentication/models.py</code>	CustomUser, LoginAttempt, PendingAccess	User accounts beyond Django's defaults
<code>authentication/views.py</code>	Login/logout pages	The web login page
<code>authentication/api_views.py</code>	API login, refresh, verify token	Mobile app authentication
<code>authentication/api_urls.py</code>	API URL routes	Maps API URLs to API views

File	What it does	Why it exists
<code>authentication/api_serializers.py</code>	Converts data to JSON for API	Translates database to API format
<code>authentication/api_permissions.py</code>	Who can access which API endpoint	Security for APIs
<code>authentication/api_exception_handler.py</code>	Consistent error format for API	Makes errors predictable
<code>authentication/decorators.py</code>	Role checking for web views	Security for web pages
<code>authentication/middleware.py</code>	Sets schema on every request	Ensures correct data isolation
<code>authentication/oidc_backend.py</code>	Bridges Keycloak to Django users	The Keycloak connection
<code>authentication/pagination.py</code>	Splits lists into pages	Handles "next page" logic
<code>assets/models.py</code>	Asset and AssetCategory models	The core data — what is an asset?
<code>assets/api_views.py</code>	Asset CRUD API endpoints	Mobile app manages assets here
<code>assets/views.py</code>	Asset web pages	Browser manages assets here
<code>organizations/models.py</code>	OrgUnit, MasterData, AuditLog	Org hierarchy, dropdowns, audit trail
<code>organizations/views.py</code>	Org hierarchy web pages	Browser views org tree
<code>organizations/api_views.py</code>	Org tree, audit log, dashboard API	Mobile app reads org data
<code>organizations/master_data_views.py</code>	Dropdown management pages	Admin configures dropdown options
<code>templates/shared/base.html</code>	Main layout with sidebar	Every page uses this template

PART 17: COMPLETE REQUEST-TO-RESPONSE FLOW

Web browser flow (full example):

1. User types: `http://moh.localhost:8000/assets/`

↑
2. Operating system asks DNS: "What is moh.localhost?"
 → /etc/hosts file says: `127.0.0.1 moh.localhost`
 → So the request goes to your own computer (localhost)
3. Django's runserver receives the request on port 8000
4. Django goes through MIDDLEWARE in order:
 - a) TenantMainMiddleware:
 "The domain is moh.localhost. Let me find the tenant."
 → Queries `tenancy_domain` table: "Who owns moh.localhost?"
 → Finds: Ministry of Health (`moh_schema`)
 → Sets connection to `moh_schema`
 - b) SecurityMiddleware:
 Adds security headers to the response later
 - c) SessionMiddleware:
 Loads the session for this browser
 - d) CorsMiddleware:
 Checks if this is a cross-origin request
 - e) CommonMiddleware:
 Handles URL redirects (e.g., `/assets` → `/assets/`)
 - f) AuthenticationMiddleware:

```

    "Is this user logged in?"
    Loads the user from the session

g) Our SchemaMiddleware:
    "What is this user's ministry_schema?"
    Sets request.schema_name = 'moh_schema'

h) SessionRefresh (mozilla-django-oidc):
    Checks if Keycloak session is still valid

5. URL dispatch:
    Django matches /assets/ against urlpatterns:
    → Finds: path('assets/', asset_list_view, name='asset_list')

6. Decorators check (before view runs):
    @login_required_custom: "Is user logged in? Yes → continue"
    @role_required('MINISTRY_ADMIN'): "What is user's role? Yes → continue"

7. View function runs:
    with schema_context('moh_schema'):
        assets = Asset.objects.all() → Only MOH's assets
    Returns HTML page with list of assets

8. Response goes back through middleware (reverse order)

9. Browser receives HTML and renders it

```

Mobile API flow (full example):

```

1. Flutter app sends: GET http://192.168.100.18:8000/api/assets/
   Header: Authorization: Bearer eyJhbGciOiJI...

2. Django's middleware processes the request (same as above)

3. URL dispatch matches: /api/assets/
   → Finds: AssetListCreateAPIView

4. Permission classes check:
   IsAuthenticated: "Is the JWT token valid? Yes → continue"
   HasMinistrySchema: "Does user have a ministry? Yes → continue"
   CanManageAssets: "GET is read-only, all roles can read → continue"

5. View runs:
   with schema_context('moh_schema'):
       assets = Asset.objects.all()
       serializer = AssetSerializer(assets, many=True)
       return Response(serializer.data) → JSON response

6. Flutter receives JSON and shows it in the mobile app

```

PART 18: GLOSSARY — Simple definitions

Term	Simple meaning
Schema	A private folder inside the PostgreSQL database. Each ministry has one.
Tenant	One customer in a multi-tenant system. For us, one ministry = one tenant.
OIDC	OpenID Connect — a standard way for one website to let another website handle login. Google, Facebook, and Microsoft use this.
JWT	JSON Web Token — a string of text that proves who you are. Like a digital ID card with an expiration date.

Term	Simple meaning
SSO	Single Sign-On — log in once, access many systems. Keycloak is our SSO provider.
DRF	Django REST Framework — makes building APIs easy.
CORS	Cross-Origin Resource Sharing — tells browsers it's OK to let the Flutter app call our API.
Middleware	Code that runs on every request before your view. Like airport security — everyone goes through it.
Decorator	A wrapper around a function that checks something first (like "is the user logged in?").
Serializer	Converts database data to JSON (for API) and JSON back to database data.
Mixin	A pre-built class you add to your model to give it extra features automatically. TenantMixin gives us schema_name for free.
REST API	A way for programs to talk to each other over the internet. Our Flutter app uses the REST API to get and update data.
Claims	Information inside a JWT token about the user (username, role, email, etc.).
Realm	In Keycloak: a collection of users, clients, and configurations. Ours is called "govasset".
Search path	PostgreSQL setting that tells it which schemas to look in when running queries. django-tenants sets this automatically.

PART 19: REMEMBER THIS FOR THE PANEL

The panel wants to know:

1. **You understand your own project** — Not just how to use it, but WHY you made each choice
2. **You can explain technical concepts simply** — If your supervisor asks "what is multi-tenancy?", you can explain it without reading from a script
3. **You know the weaknesses** — Be honest about what's not perfect (no tests, no rate limiting)
4. **You can demo it live** — Practice clicking through the UI without freezing

Key sentences to memorize:

"Each ministry gets its own PostgreSQL schema — a private set of database tables. This means Ministry of Health data is physically separate from Ministry of Finance data, enforced by the database, not just by our code."

"Keycloak handles the password checking. Our Django app never sees the user's password. Keycloak tells us: 'This user is who they say they are.' Then we check our database: 'Does this user exist in our system?'"

"JWT tokens expire after 30 minutes with a 1-day refresh token. This limits damage if a token is stolen — the thief can only use it for 30 minutes."

"Our audit log is append-only. Once a record is created, it cannot be changed or deleted. This is required for government compliance and asset tracking accountability."

The one-paragraph answer — If the panel asks "How does your security work end to end?"

Memorize this paragraph. It covers authentication, tokens, multi-tenancy, and audit in one confident answer:

"Our system has two types of users: web browser users and API users. Web users log in through Keycloak, which handles password checking and brute-force lockout. API users, including our mobile app and other groups' systems, log in through our own API endpoint, which has its own five-attempt lockout using our LoginAttempt model. Both types of login produce a JWT token containing the user's identity, role, and ministry. Every subsequent request attaches that token, and our permission classes check it before any data is touched. The multi-tenancy middleware switches the database to the correct ministry's private schema before any query runs. Every action is permanently recorded in an audit log that cannot be modified or deleted by anyone, including the Super Admin. This is all configured in settings.py, oidc_backend.py, api_views.py, and the models in tenants and organizations."

This answer covers Keycloak, JWT, brute-force, multi-tenancy, and audit — everything they are likely to ask about.

PART 20: CSS QUICK REFERENCE — What to change when the panel asks

The panel may ask: "Change this text to red" or "Make the background blue". Here is how to do it fast.

Quickest way: Inline CSS (highest priority)

Inline CSS beats any file. Add `style="..."` to any HTML tag:

```
<!-- Change text color -->
<h1 style="color:red;">This text will be red</h1>
<p style="color:blue;">Blue text</p>
<span style="color:green;">Green text</span>

<!-- Change background color -->
<div style="background-color:yellow;">Yellow background</div>
<td style="background:#fff3cd;">Light yellow cell</td>

<!-- Change text size -->
<p style="font-size:18px;">Bigger text</p>
<p style="font-size:14px;">Smaller text</p>
<h1 style="font-size:32px;">Custom heading size</h1>

<!-- Change font weight (boldness) -->
<p style="font-weight:bold;">Bold text</p>
<p style="font-weight:normal;">Normal weight</p>

<!-- Change padding (space inside) -->
<div style="padding:20px;">More space inside</div>
<div style="padding:10px 15px;">10 top/bottom, 15 left/right</div>

<!-- Change margin (space outside) -->
<div style="margin-bottom:20px;">Push things below down</div>

<!-- Change border -->
<div style="border:2px solid red;">Red border</div>
<div style="border:1px dashed #ccc;">Dashed grey border</div>

<!-- Change width -->
<div style="width:50%;">Half width</div>
<div style="max-width:800px;">Max 800px wide</div>

<!-- Multiple styles together -->
<div style="color:red; background:yellow; font-size:18px; padding:15px; border:1px solid orange;">
  All at once
</div>
```

Our project's CSS variable names (used everywhere)

We defined these in `static/css/style.css`. Use them in inline styles:

```
<!-- Text colors -->
style="color:var(--accent);"          /* Blue accent color */
style="color:var(--danger);"          /* Red for errors */
style="color:var(--success);"         /* Green for success */
style="color:var(--warning);"         /* Orange/amber for warnings */
style="color:var(--text-muted);"      /* Grey for secondary text */
style="color:var(--text-primary);"    /* Default text color */

<!-- Background colors -->
style="background:var(--bg-page);"    /* Page background */
style="background:var(--success-bg);" /* Light green bg */
style="background:var(--danger-bg);"  /* Light red bg */
style="background:var(--warning-bg);"  /* Light yellow bg */
style="background:var(--accent);color:#fff;" /* Blue bg + white text */
```

```

<!-- Border colors -->
style="border-color:var(--danger-border);"
style="border-color:var(--success-border);"

<!-- Border radius (rounded corners) -->
style="border-radius:var(--radius-sm);" /* Small rounding */
style="border-radius:var(--radius-md);" /* Medium rounding */
style="border-radius:var(--radius-lg);" /* Large rounding */
style="border-radius:var(--radius-xl);" /* Extra large */

<!-- Shadows -->
style="box-shadow:var(--shadow-sm);" /* Small shadow */
style="box-shadow:var(--shadow-md);" /* Medium shadow */
style="box-shadow:var(--shadow-lg);" /* Large shadow */

```

Common color names vs our project colors

Panel says...	Use this inline CSS
"Make this red"	<code>style="color:red;"</code> or <code>style="color:var(--danger);"</code>
"Make background yellow"	<code>style="background:yellow;"</code> or <code>style="background:var(--warning-bg);"</code>
"Make this green"	<code>style="color:green;"</code> or <code>style="color:var(--success);"</code>
"Make this blue"	<code>style="color:blue;"</code> or <code>style="color:var(--accent);"</code>
"Make text bigger"	<code>style="font-size:20px;"</code>
"Make text smaller"	<code>style="font-size:12px;"</code>
"Make this bold"	<code>style="font-weight:bold;"</code>
"Add spacing"	<code>style="padding:15px;"</code> or <code>style="margin:10px 0;"</code>
"Add border"	<code>style="border:1px solid #ccc;"</code>

Custom CSS classes we have (defined in style.css)

Do NOT use Bootstrap classes like `text-danger` or `bg-primary` — Bootstrap CSS is NOT loaded. Instead, use our custom classes:

Class	What it does
<code>.btn</code>	Makes a link/button look like a styled button
<code>.btn-primary</code>	Blue button
<code>.btn-outline</code>	Button with border (no fill)
<code>.btn-danger-outline</code>	Red outlined button
<code>.card</code>	White box with shadow and rounded corners
<code>.badge</code>	Small colored label
<code>.badge-active</code>	Green badge (asset is active)
<code>.badge-maint</code>	Orange badge (under maintenance)
<code>.badge-decomm</code>	Grey badge (decommissioned)
<code>.data-table</code>	Full-width table with borders
<code>.alert-danger</code>	Red message box
<code>.alert-success</code>	Green message box
<code>.alert-warning</code>	Yellow message box
<code>.alert-info</code>	Blue message box

Class	What it does
<code>.form-control</code>	Input field styling
<code>.form-select</code>	Dropdown styling
<code>.page-header</code>	Title area at top of page
<code>.filter-bar</code>	Search/filter row
<code>.empty-state</code>	"No data" message box
<code>.stat-card</code>	Dashboard statistic box
<code>.sidebar</code>	Left navigation panel

Bootstrap Icons (yes, we have these)

We load Bootstrap Icons (not Bootstrap CSS). Use them like this:

```

<i class="bi bi-plus-circle"></i>      <!-- Plus icon -->
<i class="bi bi-pencil"></i>           <!-- Edit/pencil icon -->
<i class="bi bi-trash"></i>           <!-- Delete/trash icon -->
<i class="bi bi-search"></i>          <!-- Search icon -->
<i class="bi bi-download"></i>        <!-- Download icon -->
<i class="bi bi-arrow-left"></i>      <!-- Left arrow -->
<i class="bi bi-check-circle"></i>    <!-- Check mark -->
<i class="bi bi-exclamation-triangle"></i> <!-- Warning triangle -->
<i class="bi bi-gear"></i>            <!-- Settings gear -->
<i class="bi bi-person"></i>          <!-- User icon -->
<i class="bi bi-box"></i>            <!-- Box/asset icon -->
<i class="bi bi-building"></i>        <!-- Building/ministry icon -->
<i class="bi bi-shield-lock"></i>    <!-- Security/lock icon -->

```

Where files are located

What you want to change	File to edit
Global styles (all pages)	<code>static/css/style.css</code>
A single page's layout	The <code>.html</code> template file in <code>templates/</code>
A single element	Use inline <code>style="..."</code> on that HTML tag

If they ask you to find what CSS class controls something:

Open the page in a browser → Right-click the element → Inspect (or F12) → Look at the "Styles" tab → It shows you which CSS file and line number controls it.

If they ask you to add a new page:

1. Create `.html` file in the correct `templates/` folder
2. Use existing classes: `class="page-header"`, `class="card"`, `class="btn btn-primary"`
3. Use inline styles for unique styling
4. All templates extend `base.html` which loads `style.css` automatically

Asset Expiry Color Coding — What Color and Why

The system uses 4 colors to signal asset expiry status at a glance. The logic lives in two places: the model properties (`assets/models.py:172-191`) and the dashboard grouping (`authentication/dashboard_views.py:94-101`).

Status	Color	CSS Variable	When It Shows	Example
Expired	 Red	--danger: #dc2626	asset_expiry_date < today (past the date)	Fire extinguisher that expired 3 months ago
Expiring soon	 Amber	--warning: #d97706	0–30 days remaining	Software license expiring in 2 weeks
Expiring later	 Blue	--info: #0284c7	31–90 days remaining	Warranty expiring in 2 months
All clear	 Green	--success: #16a34a	Zero expiry warnings	No assets expiring at all

How the code decides (assets/models.py):

```
is_expired      → asset_expiry_date < today      → RED
expires_soon    → 0 <= days_left <= 90          → AMBER
```

The dashboard (dashboard_views.py:94-101) then splits expires_soon into two buckets for display:

```
days_left < 0    → expired_assets    → RED card header
0–30 days        → expiring_soon     → AMBER card header
31–90 days       → expiring_later    → BLUE card header
0 warnings       → (none)            → GREEN "All clear" banner
```

Where the colors are defined (static/css/style.css:29-43):

```
--success: #16a34a; /* green */
--warning: #d97706; /* amber */
--danger:  #dc2626; /* red */
--info:    #0284c7; /* blue */
```

Where the colors appear in templates:

Location	Template	Lines
Dashboard expired card	templates/dashboard/dashboard.html	83-134
Dashboard expiring-soon card	templates/dashboard/dashboard.html	136-188
Dashboard expiring-later card	templates/dashboard/dashboard.html	190-238
Dashboard "All clear" banner	templates/dashboard/dashboard.html	240-251
Asset detail header badge	templates/assets/asset_detail.html	18-28
Asset detail alert banner	templates/assets/asset_detail.html	51-63
Asset detail expiry date cell	templates/assets/asset_detail.html	238-249
Status badge classes	static/css/style.css	386-397

To change a color (if the panel asks): Edit static/css/style.css lines 29-43. For example, change --danger: #dc2626 to --danger: #ff0000 to make all red elements pure red.

How warranty gets its own color: Warranty expiry does NOT use color coding. It shows as plain text with a small "Expired" note next to it if the warranty has passed (asset_detail.html:231-233 checks warranty_is_active).

Why do we never start PostgreSQL?

PostgreSQL runs as a **Windows service** — it starts automatically when Windows starts. You don't need to run it manually.

Check it: Open "Services" (Windows search → type "Services")
 → Look for "postgresql-x64-16" (or similar name)
 → Status should be "Running"
 → Startup type: "Automatic" (starts when Windows boots)

If it's not running: Right-click → Start

When you run `python manage.py runserver`, Django automatically connects to PostgreSQL using the settings in `config/settings.py`. You never need to type `pg_ctl start` or anything like that.

How many tables do we have? And their purpose?

We have **10 main tables** across our system:

#	Table name	Schema	Purpose
1	<code>tenants_ministry</code>	Public	List of all ministries (MOH, MOF, MOE...)
2	<code>tenants_domain</code>	Public	Maps domains to ministries (moh.localhost → MOH)
3	<code>authentication_customuser</code>	Public	All users from ALL ministries
4	<code>authentication_loginattempt</code>	Public	Failed login records (brute force tracking)
5	<code>authentication_pendingaccess</code>	Public	Users waiting for admin approval
6	<code>django_session</code>	Public	Browser session storage
7	<code>assets_asset</code>	Per-ministry	All assets for that ministry
8	<code>assets_assetcategory</code>	Per-ministry	Asset categories (ICT, Furniture, Vehicles...)
9	<code>organizations_orgunit</code>	Per-ministry	Organization hierarchy (Ministry → Agency → Facility)
10	<code>organizations_auditlog</code>	Per-ministry	Every action recorded (cannot be deleted)
+	<code>organizations_masterdata</code>	Per-ministry	Dropdown options (funding sources, etc.)

Wait — tables 1-6 are in the PUBLIC schema (one copy for everyone) Tables 7-10+ are in EACH MINISTRY'S PRIVATE schema (separate copy per ministry)

If we have 5 ministries, we have: 6 public tables + (5 × 4 private tables) = 26 tables total.

What columns does each table have?

1. `tenants_ministry` (who are our ministries?)

`id` → Auto number (1, 2, 3...)
`name` → "Ministry of Health"
`schema_name` → "moh_schema" (used by PostgreSQL)
`created_at` → When this ministry was added

2. `tenants_domain` (how URLs map to ministries)

`id` → Auto number
`domain` → "moh.localhost" (the URL people type)
`tenant_id` → Points to which Ministry this domain belongs to
`is_primary` → Is this the main domain for this ministry?

3. authentication_customuser (all users in the system)

```

id           → Auto number
username     → Login name (e.g., "moh_admin")
email        → Email address
password     → Hashed password (never the real password!)
role         → SUPER_ADMIN, MINISTRY_ADMIN, AGENCY_MANAGER, FACILITY_CLERK, AUDITOR
ministry_schema → "moh_schema" (which ministry they belong to)
keycloak_id  → Their ID in Keycloak (links the two systems)
is_active    → True/False (can they log in?)
date_joined  → When they registered

```

4. assets_asset (the main data — what we track)

```

id           → Auto number
name         → "Dell Latitude 5420"
asset_number → "MOH-ICT-0042" (unique barcode number)
category     → Which category (points to assetcategory table)
status       → ACTIVE, UNDER_MAINTENANCE, DECOMMISSIONED, DISPOSED
description  → "Laptop for ICT officer"
serial_number → Manufacturer's serial number
purchase_date → When it was bought
purchase_cost → How much it cost
location     → Where it is physically
assigned_to  → Who is using it (email)
org_unit     → Which organization unit it belongs to
created_at   → When this record was added
updated_at   → When it was last changed

```

Why do some tables live in the public schema and others in private schemas?

This is the most important database design decision. Here is the simple rule:

```

PUBLIC SCHEMA = Things that are shared across ALL ministries
- Users (a user belongs to one ministry but the user table is shared)
- Ministries (the list of all ministries)
- Domains (the URL-to-ministry mapping)
- Login attempts (tracking failed logins globally)
- Pending access (approval queue for new users)
- Sessions (browser login sessions)

PRIVATE SCHEMA = Things that belong to ONE ministry only
- Assets (MOH's laptops are not MOF's laptops)
- Categories (each ministry has their own categories)
- Org units (each ministry has their own structure)
- Audit logs (each ministry's actions are private)

```

The key reason: Data isolation is enforced by the database.

```

WRONG APPROACH (one big table with ministry_id):
assets_asset table:
  id  name          ministry_id
  1   Dell Laptop   1   (MOH)
  2   Office Chair  2   (MOF)
  3   Hospital Bed  1   (MOH)

Query: Asset.objects.all()
→ Returns ALL 3 rows from all ministries

```

- BUG! A programmer forgot to filter by `ministry_id`
- MOH sees MOF's data. Security breach.

OUR APPROACH (separate schemas):

MOH schema → `assets_asset` table:

```
id  name
1   Dell Laptop
3   Hospital Bed
```

MOF schema → `assets_asset` table:

```
id  name
2   Office Chair
```

Query: `Asset.objects.all()` (while in MOH schema)

- Returns only 2 rows (MOH's data)
- Cannot see MOF's data even if we forget to filter
- The DATABASE ITSELF prevents cross-ministry access

Analogy: Think of the public schema as the building's LOBBY — everyone shares it (entrance, directory, security desk). Each ministry's private schema is their OWN OFFICE — other ministries cannot walk in.

Why is the users table in the public schema?

Because a user's identity crosses ministries:

Super Admin → manages ALL ministries → needs to see everything

Ministry Admin → manages ONE ministry → sees only their schema

But ALL users are listed in one table so we can:

1. Check "does this username already exist?"
2. Super Admin can see all users across all ministries
3. Prevent duplicate accounts

If users were in private schemas, two ministries could accidentally create the user "admin" — and we wouldn't know.

But wait — if users are in the public schema, can MOH users see MOF users?

No. The `CustomUser` model has a `ministry_schema` field. When we query users, we filter by schema:

```
# In code, when listing users for a ministry admin:
users = CustomUser.objects.filter(ministry_schema='moh_schema')
# This only returns MOH users, even though the table is shared
```

This is called **application-level filtering** (the code filters). It is less secure than schema-level isolation, which is why we only use it for the users table (which needs to be shared).

How Many Schemas Do We Actually Have?

From the live database (verified via `information_schema`), we have **6 schemas** inside the single `govasset` database:

Schema	Type	Contents	Tables
<code>public</code>	Shared	All cross-ministry data	15 tables
<code>moh_schema</code>	Tenant	Ministry of Health	7 tables
<code>mof_schema</code>	Tenant	Ministry of Finance	7 tables
<code>moe_schema</code>	Tenant	Ministry of Education	7 tables
<code>mos_schema</code>	Tenant	Ministry of Sports	7 tables

Schema	Type	Contents	Tables
<code>moa_schema</code>	Tenant	Ministry of Account	7 tables

Total: 1 database, 6 schemas, 50 tables (15 public + 5 ministries × 7 each).

Each new ministry you add creates a new schema with the same 7 tables — so if you had 50 ministries, you would have 1 public schema + 50 tenant schemas = 51 schemas.

How Many Tables in Each Schema — and What Is Each One For?

The database has two kinds of tables: **Django system tables** (created automatically by Django, not by us) and **our application tables** (defined in our models).

Public Schema — 15 Tables (Shared by ALL Ministries)

Django system tables (created by Django, not by us):

#	Table	Created By	Purpose
1	<code>django_migrations</code>	Django	Tracks which migrations have run — Django uses this to know what to apply next
2	<code>django_content_type</code>	Django	Registry of all model types in the project — Django's way of knowing "what models exist"
3	<code>django_session</code>	Django	Stores browser session data — keeps users logged in between page loads
4	<code>django_admin_log</code>	Django admin	Logs every action performed in the Django admin panel (creates edits deletes)
5	<code>auth_group</code>	Django auth	Groups of permissions — can be used to assign permissions to multiple users at once
6	<code>auth_group_permissions</code>	Django auth	Links groups to their specific permissions (many-to-many relationship)
7	<code>auth_permission</code>	Django auth	Every possible permission in the system — Django generates one per model (e.g. "Can add asset", "Can change user")

Our application tables (defined in our models):

#	Table	Defined In	Purpose
8	<code>tenants_ministry</code>	<code>tenants/models.py:6-24</code>	Lists all ministries — the <code>TenantMixin</code> model that django-tenants uses to identify each tenant
9	<code>tenants_domain</code>	<code>tenants/models.py:26-28</code>	Maps domain names (e.g. <code>moh.localhost</code>) to a ministry — django-tenants uses this to route requests
10	<code>authentication_customuser</code>	<code>authentication/models.py:7-57</code>	All users across ALL ministries — the <code>CustomUser</code> model extending Django's <code>AbstractUser</code>
11	<code>authentication_customuser_groups</code>	Django auth (auto)	Links users to auth groups (many-to-many)
12	<code>authentication_customuser_user_permissions</code>	Django auth (auto)	Links users to specific permissions (many-to-many)

#	Table	Defined In	Purpose
13	<code>authentication_loginattempt</code>	<code>authentication/models.py:118-160</code>	Tracks failed login attempts per username+IP — implements the 5-attempt lockout
14	<code>authentication_pendingaccess</code>	<code>authentication/models.py:59-116</code>	Records blocked login attempts from Keycloak users who have no Django profile yet — admin reviews and approves
15	<code>authentication_superuseradminauditlog</code>	<code>authentication/models.py:164-216</code>	Audit log for Super Admin actions in the public schema (ministry creation, user management at platform level)

Each Ministry Schema — 7 Tables (Private to One Ministry)

Django system tables (created by Django, not by us):

#	Table	Created By	Purpose
1	<code>django_migrations</code>	Django	Same as public — tracks migration state for this schema
2	<code>django_content_type</code>	Django	Same as public — registry of models available in this schema

Our application tables (defined in our models):

#	Table	Defined In	Purpose
3	<code>assets_assetcategory</code>	<code>assets/models.py:8-23</code>	Categories for assets (e.g. ICT, Vehicles, Medical Equipment, Furniture) — each ministry sets their own
4	<code>assets_asset</code>	<code>assets/models.py:26-198</code>	THE main table — every asset this ministry owns, with all fields (name, serial number, cost, location, status, condition, expiry dates, etc.)
5	<code>organizations_orgunit</code>	<code>organizations/models.py:6-51</code>	The three-level hierarchy (MINISTRY → AGENCY → FACILITY) via self-referencing parent foreign key
6	<code>organizations_masterdata</code>	<code>organizations/models.py:53-90</code>	Configurable dropdown options — funding sources, acquisition methods, location types, disposal methods, cost centres
7	<code>organizations_auditlog</code>	<code>organizations/models.py:93-165</code>	Tamper-proof record of every action (CREATE, UPDATE, DELETE, LOGIN, LOGOUT, ACCESS_DENIED) — cannot be modified or deleted once created

Note: There are no `auth_permission`, `auth_group`, `django_session`, or `django_admin_log` tables in the tenant schemas. Why? Because the settings (`config/settings.py:42-46`) only put `organizations` and `assets` in `TENANT_APPS` — Django's auth and admin apps are in `SHARED_APPS` only. This means permissions, sessions, groups, and admin logs are managed centrally in the public schema, not per ministry.

Why Public vs Private? The Complete Reason

The division is controlled by two lists in `config/settings.py:26-46`:

```

SHARED_APPS = [    # → Tables created in public schema
    'django_tenants',    # Tenant management engine
    'tenants',           # Ministry, Domain models
    'authentication',    # Users, login tracking, pending access
    'django.contrib.contenttypes', # Django's model registry
    'django.contrib.auth',    # Groups, permissions

```

```

'django.contrib.admin',      # Admin panel logs
'django.contrib.sessions',   # Browser sessions
'django.contrib.messages',   # Flash messages
'django.contrib.staticfiles', # Static file management
'rest_framework',           # DRF config tables
'corsheaders',              # CORS config
'mozilla_django_oidc',       # Keycloak SSO config
'drf_yasg',                  # Swagger API docs config
]

TENANT_APPS = [              # → Tables created in EACH ministry schema
'django.contrib.contenttypes', # Django needs this per schema
'organizations',              # Org units, master data, audit logs
'assets',                     # Assets and categories
]
```

Three reasons for this split:

1. **Data isolation** — One ministry's assets should NEVER be visible to another ministry. By putting `assets` and `organizations` in `TENANT_APPS`, their tables are created inside each ministry's private schema. The database physically prevents cross-schema access.
2. **Shared identity** — Users must be unique across the entire system (no two ministries can have the same username). By putting `authentication` in `SHARED_APPS`, the `CustomUser` table lives in the public schema where uniqueness can be enforced globally.
3. **Single configuration** — Settings like REST framework configuration, CORS headers, and OIDC settings are the same for every request. Putting `rest_framework`, `corsheaders`, and `mozilla_django_oidc` in `SHARED_APPS` means they only exist in one place.

What Happens When You Run `python manage.py setup_demo_data` (Database Perspective)

This command (`authentication/management/commands/setup_demo_data.py`) operates on different schemas depending on what it is deleting or creating:

```

Clean phase:
→ Deletes LoginAttempt, PendingAccess (PUBLIC schema – shared tables)
→ Deletes Asset, AuditLog             (EACH tenant schema – looped per ministry)

Seed phase:
→ Creates Asset, AuditLog in moh_schema (MOH's private schema)
→ Creates Asset, AuditLog in mof_schema (MOF's private schema)
```

This is why the command needs `with schema_context(...)` blocks — it has to explicitly switch between schemas because it touches both shared and private data.

PART 22: MAKING THE SYSTEM LIVE — From development to production

What needs to happen to make our system live?

Right now, our system only runs on our laptop. To make it live (accessible 24/7 to real users), we need:

```

WHAT WE HAVE NOW (development):

Our laptop → python manage.py runserver 0.0.0.0:8000
→ Only works when laptop is on
→ Only works on our WiFi
→ Only we can access it
→ No proper URL (just IP address)
→ No HTTPS (no padlock icon)
→ Laptop might overheat if running 24/7
```

WHAT WE NEED (production):

A cloud server (AWS / DigitalOcean / Vultr)
 Running: Ubuntu Linux + PostgreSQL + Django
 Accessible at: <https://moh.platform.go.tz>
 Running: 24/7, never stops
 Secure: HTTPS padlock, firewall, automatic backups
 Fast: Good internet connection, proper server

What happens if we don't use proper domains?

If we only use `127.0.0.1:8000` or `192.168.x.x:8000`:

Problem 1: django-tenants cannot identify the ministry

- django-tenants reads the domain from the URL
- If URL is `127.0.0.1:8000` → no domain found
- Falls back to public schema
- All users see the public schema (not their ministry's data)
- The whole multi-tenancy system breaks!

Problem 2: No real SSL/HTTPS

- Modern mobile apps REQUIRE HTTPS for API calls
- Without HTTPS, the Flutter app will refuse to connect
- Browsers show "Not Secure" warning

Problem 3: No proper URL

- Users cannot remember "`192.168.100.18:8000`"
- They expect "`moh.go.tz`" or "`govasset.go.tz`"
- Looks unprofessional for a government system

What code changes are needed for production?

1. ALLOWED_HOSTS in settings.py:

```

# Currently (development):
ALLOWED_HOSTS = ['192.168.100.18', '172.16.20.232', '10.187.165.150',
                 'localhost', '127.0.0.1', '.localhost']

# Change to (production):
ALLOWED_HOSTS = ['.platform.go.tz',      # All subdomains
                 'platform.go.tz',      # Main domain
                 '192.168.100.18']      # Keep IP for testing

```

2. Add real domain entries in the Domain table:

```

# Currently: Domain.objects.create(domain='moh.localhost', tenant=moh)
# Change to: Domain.objects.create(domain='moh.platform.go.tz', tenant=moh)

```

3. Set DEBUG = False:

```

# Currently:
DEBUG = True

```

```
# Change to (production):
DEBUG = False
```

4. Configure HTTPS/SSL:

```
# Add to settings.py for production:
SECURE_SSL_REDIRECT = True      # Force HTTPS
SESSION_COOKIE_SECURE = True    # Only send session cookies over HTTPS
CSRF_COOKIE_SECURE = True       # Only send CSRF cookies over HTTPS
```

5. Set up a proper web server (NGINX or Apache):

```
NGINX configuration (simplified):
_____
server {
    listen 443 ssl;
    server_name moh.platform.go.tz;

    ssl_certificate /etc/ssl/certs/platform.go.tz.crt;
    ssl_certificate_key /etc/ssl/private/platform.go.tz.key;

    location / {
        proxy_pass http://127.0.0.1:8000;
        proxy_set_header Host $host;
    }

    location /static/ {
        alias /var/www/govasset/static/;
    }
}
```

How will other groups access our system in production?

```
Other Group → Their Browser → https://moh.platform.go.tz/
                        ↓
                    DNS server
                (maps domain to server IP)
                        ↓
                Cloud Server (AWS/DigitalOcean)
                        ↓
                NGINX (handles HTTPS)
                        ↓
            Unicorn (runs Django)
                        ↓
        PostgreSQL Database
```

They type: <https://moh.platform.go.tz/> in their browser **They see:** Our application, with proper URL, proper SSL, professional look

For API access: Same thing, but use:

```
https://moh.platform.go.tz/api/assets/
Instead of:
http://192.168.100.18:8000/api/assets/
```

What about our own [etc/hosts](#) file?

Currently, we use the `etc/hosts` file to make `moh.localhost` work:

```
127.0.0.1 moh.localhost
127.0.0.1 mof.localhost
127.0.0.1 admin.localhost
```

In production, we don't need this because real DNS servers handle the mapping:

```
moh.platform.go.tz → DNS → 104.28.45.12 (the server's public IP)
```

The `etc/hosts` file is ONLY for development (since we don't own real domains yet).

How to see what domain a user is using (code evidence):

```
# tenants/views.py - The tenant detection logic
from django_tenants.utils import get_tenant_model

def get_tenant_from_request(request):
    domain = request.get_host() # Gets: "moh.localhost" or "192.168.100.18:8000"
    domain_model = get_tenant_model()
    try:
        tenant = domain_model.objects.get(domains__domain=domain.split(':')[0])
        return tenant
    except domain_model.DoesNotExist:
        return None # No matching domain → public schema
```

What does django-tenants do when it finds a domain?

```
# django-tenants internally does this:
# 1. Reads: request.get_host() → "moh.localhost"
# 2. Queries: Domain.objects.get(domain="moh.localhost")
# 3. Gets: tenant = moh_ministry (schema_name = "moh_schema")
# 4. Runs: connection.set_schema("moh_schema")
# 5. Now ALL queries use: SET search_path TO moh_schema, public;
```

Proxies and load balancers — What they are and why they matter

What is a proxy?

A proxy is a server that sits BETWEEN the user and your Django application. Instead of the user talking directly to Django, the user talks to the proxy first, and the proxy forwards the request to Django.

```
User (203.0.113.5)
  |
  ▼
Proxy (198.51.100.10) ← The proxy receives the request
  |
  ▼
Django Server          ← The proxy forwards it to Django
```

Why use a proxy?

- **Security** — Hides your Django server from the public internet
- **HTTPS** — Handles SSL certificates (the padlock in the browser)

- **Caching** — Saves copies of pages to make the site faster
- **Load balancing** — Spreads traffic across multiple Django servers

What is a load balancer?

Imagine your website becomes very popular. Instead of one Django server, you have three. A load balancer decides which server handles each request:



If 3,000 people visit at the same time, one server would be overloaded. Three servers share the work. The load balancer spreads requests among them.

The IP address problem (important!)

Without a proxy, Django sees the user's real IP:

```

User (203.0.113.5) —> Django
REMOTE_ADDR = "203.0.113.5" ✓ Correct!
  
```

With a proxy, Django sees the PROXY's IP, not the user's:

```

User (203.0.113.5) —> Proxy (198.51.100.10) —> Django
REMOTE_ADDR = "198.51.100.10" X Wrong — that's the proxy!
  
```

Why does the proxy change the IP? The proxy doesn't "change" anything. It creates a NEW connection to Django. From Django's point of view, the proxy IS the client. Django can only see the computer that connected directly to it.

Analogy: You call a pizza shop. Without a proxy, you call directly — the shop sees YOUR number. With a proxy, you call your friend, and your friend calls the shop for you. The shop sees YOUR FRIEND'S number because your friend is the one making the call.

The solution — X-Forwarded-For header

To solve this, the proxy adds a special header called **X-Forwarded-For** that contains the ORIGINAL user's IP:

```

User (203.0.113.5) —> Proxy —> Django
Header: X-Forwarded-For: 203.0.113.5
  
```

Django can then read:

```

request.META['HTTP_X_FORWARDED_FOR'] # Returns: "203.0.113.5" (real user)
request.META['REMOTE_ADDR']          # Returns: "198.51.100.10" (proxy's IP)
  
```

Our helper function handles both scenarios:

```

def _get_client_ip(request):
    x_forwarded_for = request.META.get('HTTP_X_FORWARDED_FOR')
    if x_forwarded_for:
        # Proxy is present — use the FIRST IP in the list
        ip = x_forwarded_for.split(',')[0].strip()
  
```

```

else:
    # No proxy – use the direct connection IP
    ip = request.META.get('REMOTE_ADDR')
    return ip

```

Do we have a proxy NOW (in development)?

No. During development with `python manage.py runserver`, there is no proxy. The helper function checks for `HTTP_X_FORWARDED_FOR`, doesn't find it, and simply returns `REMOTE_ADDR` (which is correct).

Why write this now? You are preparing for production where a proxy like Nginx or Cloudflare is very common. By writing this helper now, your code works correctly in BOTH situations:

DEVELOPMENT (no proxy):	PRODUCTION (with proxy):
REMOTE_ADDR is correct	X-Forwarded-For has real IP
returned as-is	helper reads it automatically

Your audit log, brute-force protection, and any security features that rely on IP addresses will work correctly without changes when you deploy.

PART 23: DEVELOPMENT SETUP — How web + mobile work together

How did the web developer and mobile developer access the same data?

This is a common panel question. Here is how we worked together:

SCENARIO: Two developers on different laptops, different roles

Web Developer (Group 1A):

```

Laptop A
IP: 192.168.100.18
Runs: Django + PostgreSQL
Port: 8000

His job:
Build web pages
Build APIs
Manage database

```

Mobile Developer (Group 1B):

```

Laptop B
IP: 192.168.100.45
Runs: Flutter + Emulator
No database

His job:
Build mobile screens
Call existing APIs
Display data on phone

```

The mobile developer does NOT run Django or PostgreSQL. The mobile developer only runs Flutter (mobile app framework). To test, the mobile developer:

```

Step 1: Connect to the same WiFi (both laptops on college network)
Step 2: Web developer runs: python manage.py runserver 0.0.0.0:8000
Step 3: Web developer tells mobile: "My IP is 192.168.100.18"
Step 4: Mobile developer puts this URL in the Flutter app code:

```

```
const API_BASE_URL = 'http://192.168.100.18:8000/api/';
```

```

Step 5: Mobile developer runs the Flutter app on his phone emulator
Step 6: The emulator calls: GET http://192.168.100.18:8000/api/assets/
Step 7: Web developer's Django receives the request and returns JSON
Step 8: Mobile developer's Flutter app displays the data

```

What if the web developer's IP changes?

```
Web developer runs: ipconfig
→ Sees new IP: 192.168.100.55
→ Tells mobile developer: "My IP changed to 192.168.55"
→ Mobile developer updates one line in Flutter code
→ Everything works again
```

What if they are on different WiFi?

```
→ They cannot connect directly
→ Solutions:
a) Both join the same WiFi (college network, cafe, etc.)
b) Web developer uses ngrok (creates a public URL):
    ngrok http 8000
    → Gives: https://abc123.ngrok.io
    → Mobile uses that URL instead
c) Deploy Django to a temporary cloud server
```

Does the mobile developer need to install Django?

No. The mobile developer only needs:

```
Flutter SDK (to build the mobile app)
VS Code or Android Studio (to write code)
A phone or emulator (to test)
An internet connection (to reach the web developer's laptop)
```

The mobile developer NEVER needs:

```
Python or Django
PostgreSQL
Keycloak
Our project's source code (Django part)
```

The mobile developer only needs the API URL and the documentation.

How does the mobile developer know what API endpoints exist?

1. Web developer runs Django with Swagger enabled
2. Mobile developer opens in browser: <http://192.168.100.18:8000/api/docs/>
3. Mobile developer sees ALL endpoints with descriptions
4. Mobile developer clicks "Try it out" to test each endpoint
5. Mobile developer sees exactly what JSON is returned
6. Mobile developer writes Flutter code that parses that JSON

This is why Swagger is important! Without Swagger, the mobile developer would need to ask: "What URL do I call for assets? What fields are returned? What format is the date?" With Swagger, they find everything themselves.

What if the mobile developer makes changes that need new API features?

```
Mobile developer: "I need an API that returns only assets that expired this month."
Web developer: "OK, I'll add it."
→ Web developer adds a filter: /api/assets/?expired_this_month=true
→ Swagger updates automatically
→ Mobile developer refreshes Swagger page
```


- Mobile developer sees the new filter
- Mobile developer starts using it

Why can't the mobile developer run everything locally?

The mobile developer COULD run Django locally, but:

1. Setting up PostgreSQL + Keycloak is complex
2. They would need all the demo data
3. Their laptop might be slow (Flutter + Django + PostgreSQL)
4. Any data they create would be on their laptop, not shared
5. It's easier to use one central API that everyone shares

This is the same model used in real companies:

Frontend team (mobile/web) → calls APIs from → Backend team (server)
Backend team maintains the server and database
Frontend teams just consume the APIs

What does the Flutter code look like for calling our API?

```
// Flutter code (simplified) – This is what the mobile developer writes

import 'package:http/http.dart' as http;
import 'dart:convert';

class ApiService {
  final String baseUrl = 'http://192.168.100.18:8000/api/';
  String? token;

  // Step 1: Login
  Future<void> login(String username, String password) async {
    final response = await http.post(
      Uri.parse('${baseUrl}auth/login/'),
      body: {'username': username, 'password': password},
    );
    final data = json.decode(response.body);
    token = data['access']; // Save the JWT token
  }

  // Step 2: Get assets
  Future<List<Asset>> getAssets() async {
    final response = await http.get(
      Uri.parse('${baseUrl}assets/'),
      headers: {'Authorization': 'Bearer $token'},
    );
    final data = json.decode(response.body);
    return data['results'].map((json) => Asset.fromJson(json)).toList();
  }
}
```

The Flutter developer's job is to:

1. Call the API URL
2. Receive the JSON response
3. Parse it and display it beautifully on the phone screen
4. Handle errors (no internet, token expired, etc.)

The Flutter developer does NOT need to know:

- What database we use
- How multi-tenancy works
- What Keycloak is
- How Python/Django works

They just need the API URL and the JSON format.

How do OTHER groups (Group 2-10) work with us?

Same model as the mobile developer:

```
Other Group's Developer:
→ Gets our API URL + test credentials
→ Opens Swagger: http://192.168.100.18:8000/api/docs/
→ Tests endpoints
→ Writes code in their language (PHP, Java, C#, etc.)
→ Calls our API during development (same WiFi)
→ When we deploy to production, switches to production URL
```

The API is our contract with the world. As long as the API keeps working the same way, other groups don't care what technology we use behind it.

Do our fixes and changes affect the mobile app too?

This is an important question. The answer depends on WHAT we change:

Type of fix	Affects mobile?	Why
Changes to <code>AuditLog.objects.create()</code> or any model/database code	✓ Yes automatically	Same Django code runs for both web and mobile requests
Changes to API views in <code>api_views.py</code>	✓ Yes	Mobile calls these directly — new endpoints appear immediately
Adding new API filters (e.g., <code>?expired_this_month=true</code>)	✓ Yes	Mobile can start using them right away
Fixed IP address capture (<code>_get_client_ip()</code>)	✓ Yes	Django doesn't care if request is from browser or Flutter
Changes to web views in <code>views.py</code>	✗ No	These render HTML that mobile never sees
Changes to HTML templates (<code>.html</code> files)	✗ No	Flutter has its own completely separate screens
New sidebar links in <code>base.html</code>	✗ No	Flutter sidebar is built separately in Dart code
Changes to <code>settings.py</code>	✓ Depends	Database, JWT, CORS settings affect all clients

The general rule: Every database change and API change automatically helps mobile. Every HTML/template change is web-only. This is the clean separation between your two clients (web browser and Flutter app).

Example — IP fix we added: When a field clerk on their phone creates an asset, the request travels from the phone over the hotspot to Django. Django reads the IP address from that request using the same `_get_client_ip()` function — the phone's hotspot IP like `192.168.43.118` — and saves it to the audit log. You do NOT need to do anything in the Flutter code. It works automatically.

Why the audit trail is web-only (important for panel questions): The audit trail is a governance and accountability feature designed for office-based administrators and auditors. Nobody performs a serious audit investigation from a phone screen. The mobile app is for field work — registering assets, checking stock, quick lookups. The detailed audit trail belongs on the web platform.

If the panel asks "why is the detailed audit trail not on mobile?", say:

"The audit trail is a governance feature for office-based administrators and auditors. The mobile application is designed for field officers doing operational work. Putting a full audit investigation interface on a small phone screen would be poor UX design and is outside the field officer's job scope. The same audit data is available to anyone who needs it through the web platform."

PART 24: COMPLETE FILE-BY-FILE EXPLANATION — Every folder and every file

This part explains the ENTIRE project, folder by folder, file by file. After reading this, if someone mentions any file, you will know exactly what it does.

Project structure overview

```
govasset_platform/
├── manage.py                # Entry point – everything starts here
├── config/                  # Central settings, URLs, server entry
│   ├── settings.py         # ALL configuration in one file
│   ├── urls.py             # URL routing map
│   ├── asgi.py             # For async deployment
│   └── wsgi.py             # For production deployment
├── authentication/         # Users, login, API, security
│   ├── models.py           # CustomUser, PendingAccess, LoginAttempt
│   ├── views.py            # Login/logout web pages
│   ├── user_views.py       # User CRUD web pages
│   ├── dashboard_views.py  # Dashboard stats
│   ├── pending_access_views.py # Approve/reject new users
│   ├── api_views.py        # Mobile app login/refresh/verify API
│   ├── api_serializers.py  # Convert data to/from JSON
│   ├── api_permissions.py  # Who can access each API
│   ├── api_exception_handler.py # Standard error format
│   ├── api_urls.py         # API endpoint routing
│   ├── decorators.py       # Security guards for web pages
│   ├── middleware.py       # Sets schema on every request
│   ├── oidc_backend.py     # Keycloak ↔ Django bridge
│   ├── keycloak_admin.py   # Talk to Keycloak admin API
│   └── pagination.py       # Split lists into pages
├── assets/                 # Asset register app
│   ├── models.py           # Asset and AssetCategory models
│   ├── views.py            # Asset web pages (list, create, detail, edit, delete)
│   └── api_views.py        # Asset API for mobile app
├── organizations/         # Org hierarchy, master data, audit
│   ├── models.py           # OrgUnit, MasterData, AuditLog
│   ├── views.py            # Org unit web pages
│   ├── master_data_views.py # Dropdown options management
│   └── api_views.py        # Org + dashboard API
├── tenants/               # Multi-tenancy management
│   ├── models.py           # Ministry and Domain models
│   └── views.py            # Ministry onboarding web pages
├── templates/             # All HTML pages (26 files)
│   ├── shared/             # Base layout (sidebar, topbar)
│   ├── authentication/     # Login, user management pages
│   ├── assets/             # Asset pages (list, form, detail)
│   ├── tenants/           # Ministry management pages
│   ├── dashboard/         # Dashboard page
│   └── organizations/     # Org unit, master data pages
├── static/                # CSS files
│   └── css/
│       └── style.css       # All custom styles (758 lines)
```

FOLDER 1: `config/` — The project brain

This folder contains EVERYTHING that makes the project work: settings, URLs, server entry points. Without this folder, Django wouldn't know what to do.

`config/__init__.py`

Just tells Python "this folder is a package." Empty file.

config/settings.py — The MOST important file (390 lines)

Purpose: ONE file that controls EVERYTHING — which apps are installed, which database to use, how authentication works, what middleware runs, how logging works, security settings.

Think of it as: The control panel of an airplane. Every switch, every setting, every configuration lives here.

Key sections explained simply:

```
# Lines 28-70: Which apps are shared vs per-ministry
SHARED_APPS = [
    'django_tenants',      # The multi-tenancy library itself
    'tenants',             # Ministry model (public schema)
    'authentication',      # Users, auth (public schema – one user table for all)
    'rest_framework',      # DRF – makes APIs easy
    'corsheaders',         # Lets Flutter call our API from a different address
    'mozilla_django_oidc', # Handles Keycloak handshake
    'drf_yasg',            # Creates Swagger docs automatically
]

TENANT_APPS = [
    'organizations',       # Org hierarchy, audit logs (per-ministry)
    'assets',              # Asset register (per-ministry)
]
```

```
# Lines 135-143: Database connection
DATABASES = {
    'default': {
        'ENGINE': 'django_tenants.postgresql_backend',
        # ↑ This is the magic! django-tenants adds SET search_path before every query
    }
}
```

```
# Lines 217-235: JWT token settings
SIMPLE_JWT = {
    'ACCESS_TOKEN_LIFETIME': timedelta(minutes=30),      # Token dies in 30 min
    'REFRESH_TOKEN_LIFETIME': timedelta(days=1),         # Refresh lasts 1 day
    'ROTATE_REFRESH_TOKENS': True,                       # New refresh each time
    'BLACKLIST_AFTER_ROTATION': True,                    # Old refresh stops working
}
```

```
# Lines 461-510: Logging
LOGGING = {
    # Writes to TWO files:
    # django.log – normal messages
    # security.log – IMPORTANT: security events (logins, failures, access denied)
}
```

config/urls.py — The roadmap (169 lines)

Purpose: Maps every URL to the code that handles it. When someone types a URL, Django checks this file to find which function to run.

Think of it as: A restaurant menu — each dish (URL) has a kitchen (function) that prepares it.

```
# Example – how URL routing works:
urlpatterns = [
```

```

path('login/', login_view, name='login'),
# When user types: http://moh.localhost:8000/login/
# Django calls: login_view() function

path('assets/', asset_list_view, name='asset_list'),
# When user types: http://moh.localhost:8000/assets/
# Django calls: asset_list_view() function

path('api/', include('authentication.api_urls')),
# All API URLs start with /api/... and are defined in api_urls.py
]

```

Main URL groups:

URL pattern	What it does	Who uses it
/login/, /logout/	Web login/logout pages	Browser users
/dashboard/	Main dashboard	Logged-in users
/assets/...	Asset CRUD pages	Ministry admins, clerks
/users/...	User management	Admins
/ministries/...	Ministry onboarding	Super Admin
/api/...	All REST API endpoints	Mobile app, other groups
/api/docs/	Swagger documentation	Everyone
/oidc/...	Keycloak SSO handshake	Browser users

config/asgi.py and config/wsgi.py

Purpose: Entry points for running Django on a server. ASGI is for modern async servers, WSGI is for traditional servers.

When you use them: NEVER in development. Only when deploying to production (cloud server). The cloud server (like Gunicorn) reads one of these files to start Django.

Simplified: When you type `python manage.py runserver`, Django ignores these files. When you type `gunicorn config.wsgi:application`, it uses wsgi.py.

FOLDER 2: authentication/ — The most important folder

This folder handles EVERYTHING related to users, login, security, and APIs. It is the biggest folder (20+ files). If you understand this folder, you understand 70% of the project.

authentication/__init__.py

Empty package marker.

authentication/apps.py

Purpose: Tells Django this folder is an app named "authentication".

authentication/models.py — User database design (199 lines)

Purpose: Defines THREE database tables: CustomUser, PendingAccess, LoginAttempt.

1. CustomUser (the main user table):

```

class CustomUser(AbstractUser):
    # Django's default user has: username, password, email, first_name, last_name
    # We ADDED these extra fields:

```

```

role = models.CharField(
    choices=[ # A user can only be ONE of these:
        'SUPER_ADMIN',      # Top-level – manages all ministries
        'MINISTRY_ADMIN',   # Manages ONE ministry
        'AGENCY_MANAGER',   # Manages one agency within a ministry
        'FACILITY_CLERK',   # Manages assets at one facility
        'AUDITOR',          # Can ONLY read, cannot create/edit/delete
    ]
)
ministry_schema = models.CharField() # "moh_schema" – ties user to their ministry
keycloak_id = models.CharField()     # Links to Keycloak account
phone = models.CharField()           # Phone number

```

2. PendingAccess (users waiting for approval):

```

class PendingAccess(models.Model):
    # Created when someone tries to log in via Keycloak
    # but doesn't have a Django account yet
    email = models.EmailField()
    keycloak_id = models.CharField()
    status = models.CharField() # PENDING → APPROVED or REJECTED
    # Admin must approve before they can log in

```

3. LoginAttempt (brute-force protection):

```

class LoginAttempt(models.Model):
    MAX_ATTEMPTS = 5          # After 5 failures...
    LOCKOUT_MINUTES = 15      # ...locked for 15 minutes

    username = models.CharField()
    ip_address = models.CharField()
    attempts = models.IntegerField(default=0)
    locked_until = models.DateTimeField(null=True)

    def is_locked(self):
        # Returns: "Is this user currently locked out?"
        return self.locked_until and self.locked_until > now()

```

authentication/views.py — Login/logout pages (145 lines)

Purpose: Shows the login page and handles logging out.

login_view: Shows a page with a "Sign in with Government SSO" button. When clicked, it redirects to Keycloak. Also displays "Your account is pending approval" messages.

logout_view: When user clicks logout:

1. Writes audit log: "User X logged out"
2. Destroys Django session
3. Redirects to Keycloak logout page (so Keycloak also forgets them)

Key code — how we detect failed logins:

```

def _is_locked_out(username, ip_address):
    attempt = LoginAttempt.objects.filter(
        username=username, ip_address=ip_address
    ).first()
    if attempt and attempt.is_locked:

```

```

        return attempt.minutes_remaining # "Locked for 12 more minutes"
    return None

```

authentication/user_views.py — User management (403 lines)

Purpose: Web pages for creating, editing, activating/deactivating, and resetting passwords for users.

Views and what they do:

View	What it does	Who can access
<code>user_list_view</code>	Shows all users, filtered by ministry	Admins
<code>user_create_view</code>	Creates user in Keycloak + Django simultaneously	Admins
<code>user_edit_view</code>	Edits user details, syncs to Keycloak	Admins
<code>user_toggle_active_view</code>	Enables/disables a user account	Admins
<code>user_reset_password_view</code>	Resets password in Django + Keycloak	Admins

Key code — creating a user in BOTH systems:

```

def user_create_view(request):
    if request.method == 'POST':
        # Step 1: Create in Keycloak first
        keycloak = KeycloakAdminService()
        kc_id = keycloak.create_user(form_data) # Returns Keycloak user ID

        # Step 2: Create in Django
        user = CustomUser.objects.create(
            username=form_data['username'],
            keycloak_id=kc_id, # ← Links Django user to Keycloak user
            role=form_data['role'],
            ministry_schema=form_data['ministry_schema'],
        )
        # If Keycloak fails → Django does NOT create the user (rollback)

```

authentication/dashboard_views.py — Dashboard stats (145 lines)

Purpose: Shows the main dashboard with statistics and expiry warnings.

dashboard_view:

- Super Admin sees: "Total ministries: 5, Total users: 120, Total assets: 5,000"
- Ministry users see: "Your assets: 500, Expired: 12, Expiring soon: 25"

```

def _get_ministry_stats(ministry_schema):
    with schema_context(ministry_schema):
        # This ONLY queries their ministry's schema!
        total_assets = Asset.objects.count()
        expired = Asset.objects.filter(asset_expiry_date__lt=date.today())
        expiring_30 = Asset.objects.filter(asset_expiry_date__range=[today, 30_days])
    return {'total': total_assets, 'expired': expired.count()}

```

authentication/pending_access_views.py — Access approvals (261 lines)

Purpose: Super Admin reviews users who tried to log in but weren't registered yet.

View	What it does
<code>pending_access_list_view</code>	Shows list of all pending/approved/rejected requests
<code>pending_access_review_view</code>	Approve or reject a specific request
<code>pending_access_clear_view</code>	Delete old rejected requests (30+ days)

`authentication/api_views.py` — Mobile app authentication (473 lines)

Purpose: REST API endpoints that the Flutter mobile app and other groups call. This is the MOST important file for API authentication.

Classes (each handles one API endpoint):

Class	URL	What it does
<code>LoginAPIView</code>	POST <code>/api/auth/login/</code>	Takes username + password → returns JWT tokens
<code>RefreshTokenAPIView</code>	POST <code>/api/auth/refresh/</code>	Takes old refresh token → returns new tokens
<code>MeAPIView</code>	GET <code>/api/auth/me/</code>	Returns current user's profile
<code>VerifyTokenAPIView</code>	GET/POST <code>/api/auth/verify-token/</code>	Checks if a token is valid
<code>LogoutAPIView</code>	POST <code>/api/auth/logout/</code>	Blacklists refresh token

Key code — Login with brute force protection:

```
class LoginAPIView(TokenObtainPairView):
    def post(self, request):
        username = request.data.get('username')
        ip = get_client_ip(request)

        # Check 1: Is this user locked out?
        locked = _is_locked_out(username, ip)
        if locked:
            return Response({'error': f'Locked {locked} minutes'}, status=429)

        # Check 2: Is the password correct?
        serializer = self.get_serializer(data=request.data)
        if serializer.is_valid():
            _clear_failed_attempts(username, ip) # Success → reset counter
            return Response(serializer.validated_data)
        else:
            _record_failed_attempt(username, ip) # Failure → increment counter
            return Response({'error': 'Invalid credentials'}, status=401)
```

`authentication/api_serializers.py` — Data formatters (222 lines)

Purpose: Converts between Python objects and JSON format. The Flutter app sends/recieves JSON — serializers handle the conversion.

Key classes:

Serializer	What it converts
<code>CustomTokenObtainPairSerializer</code>	Adds role, ministry_schema INSIDE the JWT token
<code>UserProfileSerializer</code>	User data → JSON for <code>/api/auth/me/</code>
<code>AssetSerializer</code>	Asset data → JSON for <code>/api/assets/</code>
<code>AuditLogSerializer</code>	Audit log → JSON

Key code — Embedding extra data in JWT:


```
class CustomTokenObtainPairSerializer(TokenObtainPairSerializer):
    @classmethod
    def get_token(cls, user):
        token = super().get_token(user)
        # These fields are embedded INSIDE the token
        # So the mobile app can read them WITHOUT making an extra API call
        token['role'] = user.role
        token['ministry_schema'] = user.ministry_schema
        token['full_name'] = user.get_full_name()
        return token
```

authentication/api_permissions.py — API bouncers (117 lines)

Purpose: Checks "Does this user have permission to call this API?" before the API code runs.

Classes:

Class	Who it lets through
IsSuperAdmin	Only SUPER_ADMIN role
IsMinistryAdmin	SUPER_ADMIN + MINISTRY_ADMIN
CanManageAssets	Everyone can READ (GET). Only admins can WRITE (POST/PUT/DELETE)
CanDeleteAssets	Only SUPER_ADMIN + MINISTRY_ADMIN can delete
CanViewAuditLogs	SUPER_ADMIN, MINISTRY_ADMIN, AUDITOR
HasMinistrySchema	Blocks users without a ministry (Super Admin is exempt)

```
class CanManageAssets(BasePermission):
    def has_permission(self, request, view):
        if request.method in ['GET', 'HEAD', 'OPTIONS']:
            return True # Anyone can READ
        # Only certain roles can WRITE
        return request.user.role in ['SUPER_ADMIN', 'MINISTRY_ADMIN', 'AGENCY_MANAGER']
```

authentication/api_exception_handler.py — Error formatter (36 lines)

Purpose: When an API error happens, DRF returns different error formats for different errors. This file makes ALL errors the same format so the Flutter app can handle them consistently.

```
def custom_exception_handler(exc, context):
    response = exception_handler(exc, context)
    if response is not None:
        response.data = {
            'error': True,
            'message': str(response.data.get('detail', 'An error occurred')),
            'code': getattr(exc, 'default_code', 'error'),
            'status': response.status_code,
        }
    return response
```

Without this, errors look different each time:

```
Error 1: {"detail": "Authentication credentials were not provided."}
Error 2: {"username": ["This field is required."]}
Error 3: {"non_field_errors": ["Unable to log in"]}
```

```
**With this, ALL errors look the same:**
{"error": true, "message": "...", "code": "...", "status": 401}
```

authentication/api_urls.py — API routing (89 lines)

Purpose: Maps all API URLs to their view classes.

```
urlpatterns = [
    path('auth/login/', LoginAPIView.as_view(), name='api_login'),
    path('auth/refresh/', RefreshTokenAPIView.as_view(), name='api_refresh'),
    path('auth/me/', MeAPIView.as_view(), name='api_me'),
    path('auth/verify-token/', VerifyTokenAPIView.as_view(), name='api_verify'),
    path('auth/logout/', LogoutAPIView.as_view(), name='api_logout'),
    path('assets/', AssetListCreateAPIView.as_view(), name='api_asset_list'),
    path('assets/<int:id>/', AssetDetailAPIView.as_view(), name='api_asset_detail'),
    path('assets/categories/', AssetCategoryListAPIView.as_view()),
    path('org-units/', OrgUnitListAPIView.as_view()),
    path('audit-logs/', AuditLogListAPIView.as_view()),
    path('dashboard/stats/', DashboardStatsAPIView.as_view()),
]
```

Result: Flutter calls `POST /api/auth/login/` → Django runs `LoginAPIView`

authentication/decorators.py — Web page bouncers (73 lines)

Purpose: Checks permissions BEFORE a web page loads. These are for browser pages (not APIs).

```
def login_required_custom(view_func):
    # "Guard" for web pages
    @wraps(view_func)
    def wrapper(request, *args, **kwargs):
        if not request.user.is_authenticated:
            messages.warning(request, "Please log in first.")
            return redirect('login')
        return view_func(request, *args, **kwargs)
    return wrapper

def role_required(*allowed_roles):
    # "Guard" that checks user's role
    def decorator(view_func):
        @wraps(view_func)
        def wrapper(request, *args, **kwargs):
            if request.user.role not in allowed_roles:
                messages.error(request, "You don't have permission.")
                return redirect('dashboard')
            return view_func(request, *args, **kwargs)
        return wrapper
    return decorator
```

Usage:

```
@login_required_custom          # First guard: "Are you logged in?"
@role_required('SUPER_ADMIN')   # Second guard: "Are you a Super Admin?"
def ministry_list_view(request):
    # This code ONLY runs if both guards say OK
    ...
```

authentication/middleware.py — Schema setter (51 lines)

Purpose: On EVERY request, this sets `request.schema_name` so that all views know which ministry's data to use.

```
class SchemaMiddleware(MiddlewareMixin):
    def process_request(self, request):
        if request.user.is_authenticated:
            request.schema_name = request.user.ministry_schema
            request.user_role = request.user.role
```

Why this is needed: Without this, every view would need to look up the user's ministry schema from the database. This makes it available automatically.

authentication/oidc_backend.py — Keycloak bridge (146 lines)

Purpose: The MIDDLEMAN between Keycloak and Django. When Keycloak says "this user is real", this file checks if the user exists in Django.

```
class GovAssetOIDCBackend(OIDCAuthenticationBackend):
    def filter_users_by_claims(self, claims):
        # Keycloak says: "This user is authenticated"
        # Django says: "Let me find this user in MY database"
        keycloak_id = claims.get('sub')
        try:
            return CustomUser.objects.get(keycloak_id=keycloak_id)
        except CustomUser.DoesNotExist:
            return self.UserModel.objects.none()

    def create_user(self, request, user_claims):
        # Keycloak says: "This user is real but not in Django"
        # We do NOT auto-create. We create a PendingAccess request.
        PendingAccess.objects.create(
            email=user_claims.get('email'),
            keycloak_id=user_claims.get('sub'),
        )
        return None # ← "User NOT allowed yet. Admin must approve."
```

authentication/keycloak_admin.py — Keycloak API (295 lines)

Purpose: Talks directly to Keycloak's admin API to create/update/delete users. When an admin creates a user in Django, this file also creates them in Keycloak.

```
class KeycloakAdminService:
    def _get_admin_token(self):
        # Logs into Keycloak as admin to get permission to create users
        response = requests.post(
            'http://localhost:8180/realms/master/protocol/openid-connect/token',
            data={
                'client_id': 'admin-cli',
                'username': settings.KEYCLOAK_ADMIN_USERNAME,
                'password': settings.KEYCLOAK_ADMIN_PASSWORD,
                'grant_type': 'password',
            }
        )
        return response.json()['access_token']

    def create_user(self, data):
        # Creates user in Keycloak with role + ministry attributes
        response = requests.post(
```

```
f'{KEYCLOAK_URL}/admin/realms/govasset/users',
headers={'Authorization': f'Bearer {self.token}'},
json={
    'username': data['username'],
    'email': data['email'],
    'enabled': True,
    'attributes': {
        'role': [data['role']],          # ← Synced to Keycloak
        'ministry_schema': [data['ministry_schema']], # ← Synced!
    }
}
)
```

authentication/pagination.py — Page splitter (36 lines)

Purpose: When you have 1000 assets, you don't show all 1000 on one page. This splits them into pages of 20.

```
def paginate_queryset(queryset, request, per_page=20):
    paginator = Paginator(queryset, per_page)
    page_number = request.GET.get('page', 1)
    page = paginator.get_page(page_number)
    return page, paginator
```

FOLDER 3: assets/ — The asset register

This folder handles the MAIN DATA of the system — the assets themselves (laptops, vehicles, hospital beds, etc.).

assets/models.py — Asset database design (245 lines)

Purpose: Defines what an "asset" is and what "categories" exist.

AssetCategory: Categories like ICT, Furniture, Vehicles, Medical Equipment.

```
class AssetCategory(models.Model):
    name = models.CharField()      # "ICT Equipment"
    code = models.CharField()      # "ICT" – used in asset numbers: MOH-ICT-0001
    is_active = models.BooleanField(default=True)
```

Asset (30+ fields): The main model with everything about an asset.

```
IDENTIFICATION
asset_number → "MOH-ICT-2025-0042" (auto-generated, unique barcode)
name        → "Dell Latitude 5420"
serial_number → "SN123456789" (manufacturer's serial)

CATEGORY AND STATUS
category    → Points to AssetCategory (e.g., ICT)
status      → ACTIVE, UNDER_MAINTENANCE, DECOMMISSIONED, DISPOSED
condition   → EXCELLENT, GOOD, FAIR, POOR, CRITICAL

LOCATION
org_unit    → Which department/section
location_description → "3rd Floor, Room 305"

DATES
acquisition_date    → When bought
warranty_expiry_date → When warranty ends
asset_expiry_date   → When it should be replaced
```

useful_life_years → Expected lifespan

FINANCIAL

acquisition_cost → How much it cost
current_value → Depreciated value
funding_source → Where the money came from

TRACKING

registered_by → Who created this record
created_at → When created
updated_at → When last modified

Key computed properties (automatically calculated):

```
@property
def is_expired(self):
    """Has this asset passed its expiry date?"""
    return self.asset_expiry_date and self.asset_expiry_date < date.today()

@property
def expires_soon(self):
    """Will this asset expire within 90 days?"""
    if self.asset_expiry_date:
        return 0 < (self.asset_expiry_date - date.today()).days <= 90
    return False
```

assets/views.py — Asset web pages (577 lines)

Purpose: Web pages for browser-based asset management.

View	URL	What it does
asset_list_view	/assets/	Shows paginated list with search + filters
asset_create_view	/assets/create/	Form to add new asset, auto-generates number
asset_detail_view	/assets/<id>/	Full asset detail with expiry warnings
asset_edit_view	/assets/<id>/edit/	Edit form, captures old/new values for audit
asset_delete_view	/assets/<id>/delete/	Delete (MINISTRY_ADMIN only)

Key code — auto-generating asset numbers:

```
def generate_asset_number(category_code, schema_name):
    # Result example: "MOH-ICT-2025-0042"
    prefix = schema_name[:3].upper() # "MOH"
    year = str(date.today().year) # "2025"
    last_asset = Asset.objects.filter(
        asset_number__startswith=f'{prefix}-{category_code}-{year}'
    ).order_by('-asset_number').first()
    next_num = int(last_asset.asset_number.split('-')[-1]) + 1 if last_asset else 1
    return f'{prefix}-{category_code}-{year}-{next_num:04d}'
```

assets/api_views.py — Asset API (628 lines)

Purpose: API endpoints that the Flutter mobile app uses to manage assets.

Class	URL	What it does
AssetListCreateAPIView	GET/POST /api/assets/	List assets or create new one

Class	URL	What it does
<code>AssetDetailAPIView</code>	GET/PUT/DELETE <code>/api/assets/<id>/</code>	Get/update/delete one asset
<code>AssetCategoryListAPIView</code>	GET <code>/api/assets/categories/</code>	Get list of categories for dropdowns

Key code — creating an asset via API with automatic audit logging:

```
class AssetListCreateAPIView(APIView):
    permission_classes = [IsAuthenticated, HasMinistrySchema, CanManageAssets]

    def post(self, request):
        serializer = AssetSerializer(data=request.data)
        if serializer.is_valid():
            with schema_context(request.user.ministry_schema):
                asset = serializer.save(
                    registered_by_id=request.user.id,
                    asset_number=generate_asset_number(...)
                )
            # Auto-record audit log
            AuditLog.objects.create(
                action='CREATE',
                model_name='Asset',
                object_id=asset.id,
                new_value=serializer.data,
            )
            return Response(serializer.data, status=201)
```

FOLDER 4: `organizations/` — Org hierarchy, master data, audit

`organizations/models.py` — Three important models (230 lines)

1. OrgUnit (Organization Units):

```
class OrgUnit(models.Model):
    # Three-level hierarchy
    UNIT_TYPES = [
        ('MINISTRY', 'Ministry'),      # Top level (e.g., Ministry of Health)
        ('AGENCY', 'Agency'),          # Middle level (e.g., Tanzania Medicines Authority)
        ('FACILITY', 'Facility'),       # Bottom level (e.g., Muhimbili Hospital)
    ]
    name = models.CharField()           # "Muhimbili National Hospital"
    code = models.CharField()           # "MNH"
    unit_type = models.CharField(choices=UNIT_TYPES)
    parent = models.ForeignKey('self', null=True) # Points to parent unit
```

2. MasterData (dropdown options):

```
class MasterData(models.Model):
    # Configurable reference data per ministry
    CATEGORIES = [
        ('FUNDING_SOURCE', 'Funding Source'),      # "Government Budget", "Donor Funded"
        ('ACQUISITION_METHOD', 'Acquisition Method'), # "Purchase", "Donation"
        ('DISPOSAL_METHOD', 'Disposal Method'),     # "Auction", "Transfer", "Destroyed"
        ('COST_CENTRE', 'Cost Centre'),             # "ICT Department", "HR Division"
    ]
    category = models.CharField(choices=CATEGORIES)
    value = models.CharField()                   # "GOV_BUDGET"
```

```
label = models.CharField()          # "Government Budget"
is_active = models.BooleanField(default=True)
```

3. AuditLog (tamper-proof records):

```
class AuditLog(models.Model):
    # IMMUTABLE – once created, cannot be changed or deleted
    performed_by_id = models.IntegerField() # Who did it (Integer, not ForeignKey!)
    performed_by_name = models.CharField()  # Their name at that time
    action = models.CharField()             # CREATE, UPDATE, DELETE, LOGIN, LOGOUT
    model_name = models.CharField()         # "Asset", "User", "OrgUnit"
    object_id = models.CharField()          # Which record was affected
    old_value = models.JSONField(null=True) # What it WAS before
    new_value = models.JSONField(null=True) # What it IS now
    timestamp = models.DateTimeField(auto_now_add=True)
    ip_address = models.CharField()

    def save(self, *args, **kwargs):
        if self.pk: # If this record ALREADY exists
            raise Exception("AuditLog cannot be modified!")
        super().save(*args, **kwargs)

    def delete(self, *args, **kwargs):
        raise Exception("AuditLog cannot be deleted!")
```

Why IntegerField for user ID instead of ForeignKey? Because the users table is in the PUBLIC schema and the audit log is in each MINISTRY'S schema. PostgreSQL cannot create a foreign key across schemas.

organizations/views.py — Org web pages (403 lines)

View	What it does
org_unit_list_view	Shows org tree (Ministry → Agency → Facility)
org_unit_create_view	Creates Agency or Facility under a parent
org_unit_edit_view	Edits org unit name/code
org_unit_delete_view	Deletes org unit (cannot delete if has children)
audit_log_view	Shows audit trail with action filter

organizations/master_data_views.py — Dropdown management (589 lines)

View	What it does
master_data_list_view	Lists all dropdown options grouped by category
master_data_create_view	Adds new dropdown option
master_data_edit_view	Edits dropdown option
master_data_delete_view	Deletes dropdown option
master_data_seed_view	Loads default options (funding sources, methods, etc.)
asset_category_list_view	Lists asset categories
asset_category_create_view	Creates asset category
asset_category_edit_view	Edits asset category

organizations/api_views.py — Org + dashboard API (424 lines)

Class	URL	What it does
<code>OrgUnitListAPIView</code>	GET <code>/api/org-units/</code>	Returns org tree + flat facilities list
<code>AuditLogListAPIView</code>	GET <code>/api/audit-logs/</code>	Paginated audit trail with filters
<code>DashboardStatsAPIView</code>	GET <code>/api/dashboard/stats/</code>	Dashboard statistics

FOLDER 5: `tenants/` — Multi-tenancy management

`tenants/models.py` — Ministry and Domain (47 lines)

Purpose: Defines what a "ministry" (tenant) is and how domains map to ministries.

```
class Ministry(TenantMixin):
    # TenantMixin gives us: schema_name (e.g., "moh_schema")
    # auto_create_schema = True means when we save a Ministry,
    # PostgreSQL creates a NEW SCHEMA automatically

    name = models.CharField()          # "Ministry of Health"
    short_name = models.CharField()    # "MOH"
    is_active = models.BooleanField(default=True)

class Domain(DomainMixin):
    domain = models.CharField()        # "moh.localhost" or "moh.platform.go.tz"
    is_primary = models.BooleanField(default=True)
    tenant = models.ForeignKey(Ministry) # This domain belongs to MOH
```

The magic of TenantMixin: When you create a `Ministry(name="Health")`, django-tenants automatically:

1. Runs `CREATE SCHEMA moh_schema` in PostgreSQL
2. Creates ALL tenant tables inside it (`assets_asset`, `organizations_orgunit`, etc.)
3. Sets up the domain mapping

`tenants/views.py` — Ministry management (261 lines)

View	What it does	Who can access
<code>ministry_list_view</code>	Lists all ministries with stats	SUPER_ADMIN
<code>ministry_create_view</code>	Creates new ministry + schema + domain + root org unit	SUPER_ADMIN
<code>ministry_detail_view</code>	Shows ministry details: users, org units, recent assets	SUPER_ADMIN
<code>ministry_toggle_active_view</code>	Activate/deactivate a ministry	SUPER_ADMIN

Key code — onboarding a new ministry:

```
def ministry_create_view(request):
    if request.method == 'POST':
        # Step 1: Create Ministry (this auto-creates the PostgreSQL schema!)
        ministry = Ministry.objects.create(
            name=form.cleaned_data['name'],
            schema_name=form.cleaned_data['schema_name'], # e.g., "moh_schema"
        )
        # Step 2: Create domain mapping
        Domain.objects.create(
            domain=form.cleaned_data['domain'], # e.g., "moh.localhost"
            tenant=ministry,
            is_primary=True,
        )
        # Step 3: Create root org unit
```



```
with schema_context(ministry.schema_name):
    OrgUnit.objects.create(
        name=ministry.name,
        unit_type='MINISTRY',
    )
```

FOLDER 6: `templates/` — All HTML pages (26 files)

`templates/shared/base.html`

Purpose: The MAIN LAYOUT that every page extends. Contains:

- Left sidebar with navigation links
- Top bar with search and user menu
- Message area for success/error messages
- Footer with user info

All other templates just fill in the content area. They "extend" `base.html`:

```
{% extends 'shared/base.html' %}
{% block content %}
    <!-- Page-specific content goes here -->
{% endblock %}
```

`templates/authentication/login.html`

Purpose: Two-panel login page. Left side: branding. Right side: "Sign in with Government SSO" button.

`templates/authentication/user_list.html`

Purpose: User table with avatar, name, email, role badge, status, actions.

`templates/authentication/user_form.html` and `user_edit.html`

Purpose: Create/edit user forms with role and ministry selection.

`templates/authentication/pending_access_list.html` and `pending_access_review.html`

Purpose: List and approve/reject pending access requests.

`templates/dashboard/dashboard.html`

Purpose: Main dashboard with stat cards and expiry warning tables.

`templates/assets/asset_list.html`

Purpose: Asset register table with search, filters, pagination, and action buttons.

`templates/assets/asset_form.html`

Purpose: Create/edit asset form (two-column layout).

`templates/assets/asset_detail.html`

Purpose: Full asset detail view with all fields displayed.

`templates/tenants/ministry_list.html`, `ministry_form.html`, `ministry_detail.html`

Purpose: Ministry management pages (list, create, detail).

templates/organizations/ (8 files)

Purpose: Org unit management, master data management, audit log viewer, asset category management.

templates/shared/pagination.html

Purpose: Reusable pagination component (Previous, Page 1, Page 2, Next...). Included by any page that has lists.

FOLDER 7: static/ — CSS and images

static/css/style.css (758 lines)

Purpose: ALL styling for the entire application. One file controls everything visual.

Key sections:

```
Lines 4-58:    CSS variables (colors, sizes, shadows) – THE DESIGN SYSTEM
Lines 73-212:  Sidebar layout and navigation
Lines 214-259: Main content area layout
Lines 261-283: Card styling
Lines 285-331: Dashboard stat cards
Lines 333-371: Table styling
Lines 373-403: Badges (status labels)
Lines 430-469: Buttons
Lines 471-539: Forms and inputs
Lines 541-557: Alert messages
Lines 657-758: Login page styling
```

ROOT FILE: manage.py

Purpose: The ENTRY POINT for EVERYTHING. Every command goes through this file.

```
#!/usr/bin/env python
import os
import sys

def main():
    os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'config.settings')
    # ↑ Tells Django where to find settings
    try:
        from django.core.management import execute_from_command_line
    except ImportError:
        raise ImportError("Django is not installed.")
    execute_from_command_line(sys.argv)

if __name__ == '__main__':
    main()
```

Commands you pass to it:

python manage.py runserver	# Start development server
python manage.py migrate	# Create/update database tables
python manage.py migrate_schemas	# Create/update tables in ALL schemas
python manage.py createsuperuser	# Create admin account
python manage.py setup_demo_data	# Seed demo data for presentation
python manage.py shell	# Open Python command line
python manage.py test	# Run tests

Summary: What to remember

If they ask about...	Go to this file/folder
Login process	authentication/views.py and oidc_backend.py
User accounts	authentication/models.py (CustomUser)
User management	authentication/user_views.py and keycloak_admin.py
API authentication	authentication/api_views.py
JWT tokens	authentication/api_serializers.py and config/settings.py (SIMPLE_JWT)
Permissions (API)	authentication/api_permissions.py
Permissions (web)	authentication/decorators.py
Assets (web)	assets/views.py
Assets (API)	assets/api_views.py
Assets (database)	assets/models.py
Organization units	organizations/models.py and views.py
Audit log	organizations/models.py (AuditLog)
Master data/dropdowns	organizations/master_data_views.py
Multi-tenancy	tenants/models.py
Onboard a ministry	tenants/views.py
All settings	config/settings.py
URL routing	config/urls.py
API URL routing	authentication/api_urls.py
Swagger docs	http://localhost:8000/api/docs/
Login page	templates/authentication/login.html
Dashboard	templates/dashboard/dashboard.html
All styling	static/css/style.css
All commands & setup	Part 25 below

PART 25: IMPORTANT COMMANDS & SETUP REFERENCE — Everything you need to run the system

In this part:

- [Step 0 — PostgreSQL \(do nothing\)](#)
 - [Step 1 — Find your IP \(do this first, every time\)](#)
 - [Step 2 — Start Keycloak](#)
 - [Step 3 — Start Django](#)
 - [Step 4 — Connect your phone \(do this every time the IP changes\)](#)
 - [Step 5 — Load clean demo data](#)
 - [Step 6 — Open everything in the browser](#)
 - [Troubleshooting table](#)
 - [Test accounts](#)
 - [Correct startup order](#)
-

Step 0 — PostgreSQL (Do Nothing)

PostgreSQL starts automatically when Windows boots. You never need to touch it. If you want to confirm it is running, press **Win + R**, type `services.msc`, find **postgresql-x64-17** (or similar name) — it should say **Running**. That is all.

Step 1 — Find Your IP Address (Do This First, Every Time)

Open any terminal and run:

```
ipconfig
```

Look for **IPv4 Address** under your active adapter. If you are using a phone hotspot, it will look like `192.168.43.x`. Write down that number — you will use it in Steps 3 and 4.

Step 2 — Start Keycloak

Open a terminal, navigate to the Keycloak bin folder, then start it:

```
cd C:\keycloak\keycloak-26.6.2\bin
kc.bat start-dev --http-port=8180
```

Why `start-dev`: the normal `start` command requires SSL certificates and a production setup. `start-dev` skips all that and runs Keycloak in development mode — perfect for your project and presentations.

Why `--http-port=8180`: Keycloak's default port is 8080, which would clash with Django or other services. Port 8180 keeps them separate.

Wait until you see this line before continuing:

```
Keycloak 26.6.2 on JVM started
```

Keycloak is slow — give it about 30–60 seconds. Do not close this terminal window.

Step 3 — Start Django

Open a **second, separate** terminal window (keep Keycloak running in the first one):

```
cd D:\government_asset_platform
venv\Scripts\activate
```

You should see `(venv)` appear at the start of your terminal line. That means the virtual environment is active. If you accidentally use the wrong venv, nothing will work — always activate from `D:\government_asset_platform\venv`.

Now start Django:

```
python manage.py runserver 0.0.0.0:8000
```

Why `0.0.0.0`: this tells Django to listen to ALL devices in the room (your phone, your laptop), not just itself. Without this, your phone cannot reach Django at all, even on the same hotspot.

Why `8000`: this is your port number — the specific door Django listens on.

Keep this terminal open. Do not close it.

Step 4 — Connect Your Phone (Do This Every Time the IP Changes)

Every time you switch WiFi networks or hotspots, your IP address changes. You must update these **four things** every single time:

4A — Update the Flutter App's IP

Open this file:

```
C:\Users\Hemed\govasset_mobile\lib\config\api_config.dart
```

Find this line and change the IP:

```
static const String serverIp = 'PUT_YOUR_NEW_IP_HERE';
```

For example, if your IP is **192.168.43.105**:

```
static const String serverIp = '192.168.43.105';
```

Save the file.

4B — Update Django's Allowed Hosts

Open this file:

```
D:\government_asset_platform\config\settings.py
```

Find **ALLOWED_HOSTS** and add your new IP:

```
ALLOWED_HOSTS = [  
    'localhost',  
    '127.0.0.1',  
    '.localhost',  
    '192.168.43.105',    # ← your new IP here  
]
```

Save the file.

4C — Register the New IP as a Domain in Django (Most Forgotten Step)

This is the step that causes the **404 error** and "**no internet connection**" on the phone if skipped. Django-tenants checks every incoming request against a registered domain list. If your new IP is not in that list, it turns the phone away before your code even runs.

Run these commands:

```
cd D:\government_asset_platform  
venv\Scripts\activate  
python manage.py shell
```

Once inside the shell, paste these lines exactly, replacing the IP with yours:

```
from tenants.models import Domain, Ministry
public = Ministry.objects.get(schema_name='public')
Domain.objects.get_or_create(domain='192.168.43.105', tenant=public, is_primary=False)
print("Done")
exit()
```

You should see **Done** printed. Now exit the shell with `exit()`.

4D — Reinstall the App on Your Phone

Connect your phone to your laptop with a USB cable. Make sure **USB debugging** is ON in your phone's developer settings. Then run:

```
cd C:\Users\Hemed\govasset_mobile
flutter run
```

This rebuilds the app with the new IP address baked in and installs it fresh on your phone. Wait until the app opens on your phone screen before unplugging the USB cable.

Why you must rebuild: the IP address is baked into the app at build time. Changing the file alone does nothing — the app needs to be rebuilt and reinstalled so the new IP is inside the installed version on your phone.

After `flutter run` finishes and the app opens on your phone, you can unplug the USB cable. The app will keep working over WiFi/hotspot.

Step 5 — How to Clear the System and Start Fresh

You have two options depending on what your panel asks:

Option A — Completely Empty (Truly Starting From Scratch)

```
python manage.py flush --no-input
python manage.py migrate
python manage.py createsuperuser
```

What `flush` does — This is a **Django built-in command** (not our code). It deletes EVERY row from EVERY table across ALL schemas — users, ministries, assets, audit logs, everything. The database structure stays (tables, columns) but all data is gone. You then need `migrate` (recreates any missing tables) and `createsuperuser` (so you can log in again).

When to use: Only if the panel asks "Show us starting from absolute zero with no data."

Option B — Clean Demo Data (Recommended for Panel)

```
python manage.py setup_demo_data
```

This is our custom command — lives at `authentication/management/commands/setup_demo_data.py` (553 lines).

What "clean" and "seed" mean (terminology explained):

Term	Meaning	In our code
Clean	Delete existing data to remove test clutter	<code>_clean_data()</code> at line 49 — deletes <code>LoginAttempt</code> , <code>PendingAccess</code> , all <code>Asset</code> and <code>AuditLog</code> records

Term	Meaning	In our code
Seed	Insert fresh starting data automatically	<code>_seed_data()</code> at line 94 — calls <code>_seed_moh_data()</code> and <code>_seed_mof_data()</code> to create realistic demo assets

What the command does step by step:

1. Clean phase (`_clean_data`, `setup_demo_data.py:49-88`):

- Deletes `LoginAttempt` records from the public schema (failed login counters reset)
- Deletes `PendingAccess` records from the public schema (access requests cleared)
- Loops through every ministry schema (MOH, MOF, etc.) and deletes:
 - All `Asset` records — `Asset.objects.all().delete()` at line 72
 - All `AuditLog` records — `AuditLog.admin_bulk_delete()` at line 80

2. Seed phase (`_seed_data`, `setup_demo_data.py:94-98`):

- Calls `_seed_moh_data()` (line 100) — creates **8 MOH assets** with realistic data:
 - 4 ICT assets: Dell laptop (2,850,000 TZS), HP printer, Cisco switch, Samsung TV
 - 2 Vehicles: Toyota Prado (85,000,000 TZS), Toyota Ambulance
 - 1 Medical: Mindray Hematology Analyzer (45,000,000 TZS)
 - 1 Furniture: Executive desk set
 - The analyzer at line 318 is set to expire *next month* so the dashboard expiry warning shows
 - Plus 3 demo audit log entries (LOGIN, CREATE, UPDATE)
- Calls `_seed_mof_data()` (line 392) — creates **4 MOF assets** with different data:
 - 2 ICT: HP EliteBook (3,100,000 TZS), Lenovo desktop
 - 1 Vehicle: Toyota Fortuner (78,000,000 TZS)
 - 1 Furniture: Conference room set
 - Plus 3 demo audit log entries

What the command does NOT touch (your accounts stay safe):

- Users and their roles (moh_admin, mof_admin, mnh_manager, etc.)
- Ministries and their schemas (MOH, MOF)
- Organisation hierarchy (Ministry > Agency > Facility)
- Asset categories (ICT, VEH, MED, FURN)
- Master data (funding sources, location types, disposal methods)

Why this is safe for the panel: You can run it repeatedly without losing your login credentials. All test accounts (`superadmin`, `moh_admin`, `mof_admin` — all with password `Admin@123`) continue to work after every run.

Code reference: The full source is at `authentication/management/commands/setup_demo_data.py`. Key lines:

- Command class: line 7
- Clean logic: lines 49-88
- MOH seed data: lines 100-390
- MOF seed data: lines 392-512
- Demo audit logs: lines 514-553

Step 6 — Open Everything in the Browser

Once Django and Keycloak are both running:

What	Address
Web platform	<code>http://localhost:8000</code>
API documentation (Swagger)	<code>http://localhost:8000/api/docs/</code>
Keycloak admin panel	<code>http://localhost:8180</code>
Django admin panel	<code>http://localhost:8000/admin/</code>

To test from your phone's browser on the same hotspot, replace `localhost` with your IP:

```
http://192.168.43.105:8000
```

Quick Troubleshooting Reference

Problem you see	Most likely cause	Fix
Phone shows "No internet connection"	IP in <code>api_config.dart</code> is old	Update <code>serverIp</code> and run <code>flutter run</code> again
Phone gets a 404 error	New IP not registered as Domain	Run Step 4C again with new IP
Web login does nothing or errors	Keycloak not started	Run Step 2, wait for it to fully load
<code>ModuleNotFoundError</code> when starting Django	Wrong virtual environment	Run <code>venv\Scripts\activate</code> from <code>D:\government_asset_platform</code>
Django says <code>ALLOWED_HOSTS</code> error	New IP not in <code>settings.py</code>	Run Step 4B
Phone app shows old data after IP change	App not rebuilt	Reconnect USB and run <code>flutter run</code> again
Keycloak page not loading	Still starting up	Wait 60 seconds, try again
Web page has no styling / CSS broken	Browser cached old CSS	Press Ctrl+Shift+R (hard refresh) — or clear browser cache

Test Accounts (All Use Password: `Admin@123`)

Username	Role	Ministry
<code>superadmin</code>	Super Admin	Platform-wide
<code>moh_admin</code>	Ministry Admin	Ministry of Health
<code>mnh_manager</code>	Agency Manager	Ministry of Health
<code>rad_clerk</code>	Facility Clerk	Ministry of Health
<code>moh_auditor</code>	Auditor	Ministry of Health
<code>mof_admin</code>	Ministry Admin	Ministry of Finance

The Correct Startup Order (Every Time)

1. Check IP (`ipconfig`)
2. Start Keycloak (wait for it to fully load)
3. Start Django (`python manage.py runserver 0.0.0.0:8000`)
4. If IP changed → update `api_config.dart` + `settings.py` + register domain + `flutter run`
5. Test web login in browser
6. Test mobile login on phone
7. Run `setup_demo_data` if doing a presentation

Print or screenshot this page and keep it next to your laptop.

PART 26: THE FULL 10-GROUP PROJECT — Where Group 1 Fits

In this part:

- The complete project structure
- Group 1 is the FOUNDATION
- Dependency matrix — every group depends on us
- What each group delivers
- How our API serves all groups
- Panel talking points

The Complete Project Structure

The whole project is called "**Government Asset Management System**" and is split into **10 groups**. Each group builds one module. Together they cover the complete lifecycle of government assets — from registration to disposal, including maintenance, finance, GIS, compliance, and analytics.

Group	Module Name	Focus Area	Dissertation Title
1	Administration, Security & Multitenancy	RBAC, SSO, audit trails, master data, multi-tenant isolation	<i>Design and Implementation of a Secure Multi-Tenant Platform for Government Asset Management Systems</i>
2	Asset & Property Register Module	Centralized asset registry, classification, hierarchy, document management	<i>Centralized Digital Asset Registry with Lifecycle Tracking for Public Sector Properties</i>
3	Lifecycle Planning & Capital Projects	Asset lifecycle planning, condition assessment, CAPEX analysis	<i>Lifecycle Cost Analysis and Capital Project Portfolio Management System</i>
4	Maintenance & Operations (CMMS)	Work orders, preventive maintenance, service contracts, mobile technician app	<i>Computerized Maintenance Management System with Mobile Field Service Capabilities</i>
5	Financial Management & Valuation	Depreciation, lease management, budgeting, financial reporting	<i>Integrated Financial Management System for Asset Valuation, Depreciation and Lease Accounting</i>
6	Land Management & GIS Integration	Cadastral data, GIS visualization, space management, room bookings	<i>GIS-Enabled Spatial Management System with BIM Integration for Smart Facilities</i>
7	Energy, Sustainability & IoT	Utility monitoring, smart meters, carbon footprint, ESG dashboards	<i>IoT-Enabled Energy Monitoring and Sustainability Management System for Smart Buildings</i>
8	Compliance, Risk & Safety	Statutory inspections, risk registers, incident reporting, CAPA tracking	<i>Integrated Compliance Tracking and Risk Management System for Facility Safety</i>
9	Inventory, Stores & Procurement	Spare parts, barcode/RFID, purchase orders, supplier management	<i>RFID-Based Inventory Management System with Automated Procurement Workflows</i>
10	Analytics, Dashboards & Reporting	KPIs, standard reports, drill-down, predictive analytics, BI	<i>Business Intelligence and Decision Support System for Property Management Analytics</i>

Group 1 Is the FOUNDATION — Every Other Group Depends on Us

Group 1 is not just one module among ten. **Group 1 is the platform that every other module runs on top of.** Here is why:

What Group 1 provides to everyone else:

1. **Authentication** — Every user in every group must log in through our Keycloak SSO. No other group has their own login page.
2. **Authorization (RBAC)** — Our 5-role system (Super Admin → Facility Clerk) controls what each user can access. Groups 2-10 simply check "what role does this user have?"
3. **Multi-Tenant Isolation** — Each ministry's data is in a separate PostgreSQL schema. Groups 2-10 never see data from other ministries. Our django-tenants setup makes this automatic.
4. **API Security (JWT)** — Our SimpleJWT configuration issues tokens that all other groups use to authenticate API calls. The `/api/auth/verify-token/` endpoint is how Groups 2-10 validate users.
5. **Audit Trail** — Every action across ALL modules is logged in our AuditLog table with IP, user, timestamp, old/new values.
6. **Master Data** — Asset categories, locations, cost centres — these are configured in our master data tools and used by all other groups.
7. **User Management** — Only Group 1 creates and manages users. Groups 2-10 never need to create user accounts.

The dependency matrix from the project document:

Group 1 → depends on: NOBODY
→ everyone depends on: Groups 2, 3, 4, 5, 6, 7, 8, 9, 10

Group 2 → depends on: Group 1
Group 3 → depends on: Groups 1, 2
Group 4 → depends on: Groups 1, 2, 9
Group 5 → depends on: Groups 1, 2, 3
Group 6 → depends on: Groups 1, 2
Group 7 → depends on: Groups 1, 2, 6
Group 8 → depends on: Groups 1, 2, 4
Group 9 → depends on: Groups 1, 2, 4
Group 10 → depends on: ALL groups for data

Read this carefully: Group 1 is the **only group that depends on nobody**. Every single other group needs Group 1 to function. If our login system is down, the entire system stops — no one can access anything.

Dependency Matrix Explained Simply

Think of it like building a house:

- **Group 1** = The foundation and electricity. Without it, nothing works.
- **Group 2** = The rooms and walls (asset register). Built on the foundation.
- **Groups 3-9** = The plumbing, wiring, painting (modules). Each needs the rooms and the foundation.
- **Group 10** = The security cameras (analytics). Monitors everything.

If you try to build the walls (Group 2) without the foundation (Group 1), the walls collapse. If you try to paint (Group 7) without rooms (Group 2), where do you paint? This is exactly how the dependency matrix works.

What Group 1 Specifically Delivers (From the Project Document)

Scope (what we cover):

- Role-Based Access Control with hierarchical organization (ministry → agency → facility)
- Single Sign-On integration with Keycloak
- Full audit trails for changes to master data and transactions
- Configurable master data: asset categories, locations, cost centres, user roles, workflows
- Multi-tenancy: separate data, users, and configurations per ministry
- Centralized management dashboard

Key Deliverables (what we built):

1. RBAC system with hierarchical organization and multi-tenancy support
2. SSO integration (Keycloak) setup and configuration
3. Audit trail logs and reporting interface
4. Master data configuration tools (categories, locations, cost centres, roles)
5. User management dashboard and admin controls
6. Tenant management interface (ministry onboarding with schema isolation)
7. Security and access management documentation
8. REST API with JWT authentication for mobile and other groups

How Our API Serves All Groups 2-10

The API endpoints we built are not just for our own mobile app. They are the **official integration point** for all 9 other groups. Here is what each group uses from us:

Group	What they need from our API
2 (Asset Register)	User authentication + role check before CRUD operations
3 (Lifecycle)	Verify who can create/edit capital projects

Group	What they need from our API
4 (Maintenance)	Authenticate technicians on mobile app + assign work orders by role
5 (Finance)	Check if user has MINISTRY_ADMIN role to approve budgets
6 (GIS)	Authenticate users viewing spatial data
7 (Energy/IoT)	Authenticate system-to-system API calls from smart meters
8 (Compliance)	Check AUDITOR role for inspection reports
9 (Inventory)	Authenticate store clerks scanning barcodes
10 (Analytics)	Role-based dashboard access (executive vs technician views)

The key endpoint:

```
GET /api/auth/verify-token/
Authorization: Bearer <token>
→ Returns: { "role": "MINISTRY_ADMIN", "ministry_schema": "moh_schema", ... }
```

Any group can call this endpoint with a user's JWT token and instantly know their role, ministry, and permissions. This is how the entire system shares authentication through us.

Panel Talking Points — What to Say About the 10-Group Structure

If they ask: "How does your system connect to other groups?"

"Our platform is the security and administration foundation. We provide authentication, role-based access control, and multi-tenant data isolation that all 9 other groups depend on. Every user in the entire system logs in through our Keycloak SSO. Other groups call our API to verify user identities and check permissions. This is documented in the project's dependency matrix — Group 1 depends on nobody, but every other group depends on us."

If they ask: "What if another group wants to add their own login?"

"That would defeat the purpose of SSO. Our Keycloak is the single entry point for the entire system. If another group created their own login, users would need multiple credentials and the audit trail would be broken. The project design explicitly makes Group 1 the sole authentication provider."

If they ask: "How do you handle data isolation between ministries?"

"Each ministry has its own PostgreSQL schema, managed by django-tenants. When a user logs in, our system knows which schema to route them to based on their ministry_schema field. Groups 2-10 never see cross-ministry data because the database itself enforces the isolation at the schema level."

If they ask: "What is your dissertation title?"

"Design and Implementation of a Secure Multi-Tenant Platform for Government Asset Management Systems."

If they ask: "What specific security features did you implement?" (refer to Part 8 for the full 15-feature list, but here are the top ones relevant to Group 1's role):

"We implemented 15 security features including: Keycloak SSO authentication, role-based access control with 5 role levels, multi-tenant data isolation via PostgreSQL schemas, JWT token authentication with 30-minute access tokens, tamper-proof audit trail with IP tracking, account lockout after 5 failed attempts, and session management with configurable timeouts."

If they ask: "How is your project different from the other groups?"

"We are the platform team. Other groups focus on specific business modules — asset register, maintenance, finance, GIS, compliance. We focus on security, identity, and data isolation. Without our platform, the other groups would have no authentication system, no user management, no audit trail, and no data isolation between ministries. Every single group depends on us."

PART 27: TONIGHT & TOMORROW CHECKLIST — What to do, when to do it

Tonight (Sunday July 5)

- ☐ **Verify everything starts** — Run startup sequence once more
- ☐ **Check your laptop** — Fully charged, charger packed, no pending Windows updates
- ☐ **Test the demo** — Click through login → assets → audit log → API docs
- ☐ **Open every page** — Dashboard, Assets, Audit Log, Users, Ministries, Master Data, Org Units, API Swagger
- ☐ **Read Part 19 aloud** — The one-paragraph answer. Say it until it flows naturally
- ☐ **Pack for tomorrow:** Laptop + charger, mouse, notebook, pen, water bottle, your phone (for mobile demo)
- ☐ **Set alarm** — Give yourself 1.5 hours before panel time
- ☐ **Sleep** — A tired brain forgets even simple things

Tomorrow Morning (Monday July 6)

- ☐ **Start PostgreSQL** — `net start postgresql-x64-17` (or whatever version)
- ☐ **Open pgAdmin** — Have `\dn` or the schema list ready
- ☐ **Start Keycloak** — `cd C:\keycloak-25.0.1\bin && .\kc.bat start-dev --http-port=8180`
- ☐ **Wait 15+ seconds**, then open `http://localhost:8180` to confirm it's up
- ☐ **Start Django** — `python manage.py runserver 0.0.0.0:8000`
- ☐ **Open `http://localhost:8000`** — Confirm the login page loads
- ☐ **Log in as Super Admin** — Verify test data exists
- ☐ **Open Keycloak admin** — `http://localhost:8180` → admin/admin123 → Users tab
- ☐ **Swagger docs** — Open `http://localhost:8000/api/docs/` — confirm it renders
- ☐ **Leave everything running** — Don't close any terminal until the panel starts

30 Minutes Before Panel

- ☐ **Hard refresh browser** (Ctrl+Shift+R) — Clear any cached old CSS
- ☐ **Close all background apps** — Zoom/Teams/notifications off
- ☐ **Maximize the browser** — So the panel can see the full UI
- ☐ **Open 3 tabs** — Tab 1: Your app (logged in as Super Admin). Tab 2: Keycloak admin. Tab 3: Swagger docs
- ☐ **Put your phone on silent** — No buzzes during demo
- ☐ **Deep breath** — You know this system. You built it.

What to Bring to the Panel Table

Item	Why
Laptop (charged)	The main demo machine
Charger	Just in case
Mouse	More precise than trackpad for clicking through UI
Notebook + pen	Write down panel questions you want to address
Water bottle	Dry mouth is real
Your phone	For the mobile app demo (if needed)

What NOT to Do

- ☒ **Don't read from the screen** — Look at the panel, not your laptop
- ☒ **Don't say "I don't know" and stop** — Say "I haven't tested that, but based on the design..." (Part 15 style)
- ☒ **Don't apologise** — Never say "Sorry, this is not perfect." Just say what it does
- ☒ **Don't rush** — Speak at normal pace. Pauses are fine. Silence is better than "ummm"
- ☒ **Don't minimise a broken tab** — Just close it. Don't show them the error

PART 28: DEFENSE STRATEGIES — What If Things Go Wrong

What If: The demo freezes or crashes

Stay calm. Say: "Let me restart that process." Close the terminal, restart Django. Keep talking while it loads:

"While that restarts — Django's development server reloads in about 5 seconds. This is a development environment, so there is no production-grade load balancer or process manager. In production, systems like Gunicorn and NGINX would handle this automatically with zero downtime."

This turns a crash into a demonstration that you understand production deployment.

What If: Keycloak won't start (port already in use)

Say: "Let me check what is using port 8180." Run:

```
netstat -ano | findstr :8180
```

Then `taskkill /PID [number] /F`. If that doesn't work, restart your PC (5 minutes). If asked why:

"Keycloak needs port 8180 for its authentication service. If another program grabbed that port, Keycloak cannot start. In production, we run Keycloak on a separate server so this never happens."

What If: The panel asks about a feature you don't have

Never say "We didn't have time." Instead:

"That is on our roadmap. The current system focuses on the core asset register and security foundation. Features like [their feature] would be the next logical step after deployment. Our architecture supports it because we built with extensibility in mind."

What If: The panel asks "So what did YOU actually code?"

This is a trap question — they want to know if you copied code or understood it. Answer with specific code references:

"I wrote the Keycloak OIDC backend in authentication/oidc_backend.py — about 90 lines that bridge Keycloak login to Django users. I designed the multi-tenancy setup with django-tenants in tenants/models.py. I built the 5-role access control system with the decorators and permission classes. I created the audit log system with tamper-proof records. I wrote all the API endpoints for mobile authentication and the Swagger documentation."

What If: The panel asks "Why did you choose this technology?"

Use the "Three Reasons" structure — gives a confident, structured answer:

PostgreSQL: "Three reasons: (1) Schema support — it is the only major database that lets us give each ministry its own private set of tables. (2) Mature and well-documented — the most trusted open-source database for government systems. (3) Django has first-class PostgreSQL support including JSON fields and array fields."

Django: "Three reasons: (1) The ORM makes database queries simple and secure — no raw SQL. (2) django-tenants integrates natively with Django's ORM for multi-tenancy. (3) Django has built-in admin, authentication, and middleware systems that saved us months of development time."

Keycloak: "Three reasons: (1) It is the industry standard for SSO — used by governments and enterprises worldwide. (2) It handles password security, brute-force protection, and session management so we don't have to build those from scratch. (3) It supports OpenID Connect, which lets us authenticate both web browser users and API users with the same identity source."

django-tenants: "Three reasons: (1) It automatically switches PostgreSQL schemas on every request — we do not have to write WHERE ministry_id = X on every query. (2) It provides true data isolation — the database itself prevents cross-ministry access. (3) It integrates with Django's ORM seamlessly — all our existing models work without changes."

What If: The panel says "Your system has no tests"

Be honest. Do not make excuses.

"You are correct — we have zero automated tests. This is a prototype, not a production system. The focus was on building the complete feature set first: multi-tenancy, SSO, API, audit trail, and all the CRUD operations. In production, we would add test coverage before deployment, starting with the authentication API since that is the most critical path."

If they push harder:

"For a production government system, I would write tests for: (1) Login and token validation — the most security-critical path. (2) Multi-tenancy isolation — proving a user from MOH cannot see MOF data. (3) Audit log integrity — proving records cannot be modified. In a real deployment, these would be mandatory."

What If: The panel asks about performance

"The current system ran 0 queries against live data — we have no real data. Based on the database design, each ministry's schema is independent, so query performance only depends on that ministry's data size. Indexes on asset_number, status, and category already exist. For large-scale deployment, I would add database connection pooling with PgBouncer and cache frequently accessed data with Redis."

What If: You forget something or freeze

Pause. Take a breath. Say:

"Let me think about that for a moment."

A 5-second pause feels long to you but normal to the audience. Do not fill silence with "ummm" or "ahhh." If you still cannot remember, say:

"I know we handle that, but I want to give you an accurate answer. Let me check the code quickly."

Then open the relevant file and read from it. This shows you know WHERE the answer is, even if you can't recite it.

PART 29: CONFIDENCE TIPS — How to Sound Like You Own This Project

Posture

- Sit up straight, both feet on the floor
- Put your hands on the table — not in your lap, not crossed
- When explaining, use hand gestures (open palms) — it makes you look confident

Eye Contact

- Look at the person who asked the question
- When answering, rotate eye contact across all panel members
- Do NOT stare at your screen while explaining

Voice

- Speak slower than you think you need to — nerves speed you up
- Pause between sentences. A 1-second pause sounds natural
- Drop your pitch at the end of sentences (not up — up sounds like a question)
- The last word of each sentence should be LOUDER, not quieter

The "Confident Stumble" Recovery

If you trip over a word or lose your train of thought:

1. **Stop.**
2. **Smile.**
3. **Say:** *"Let me rephrase that."*
4. **Start the sentence again from the beginning.**

This makes you look thoughtful, not confused. The panel will respect the recovery more than the stumble.

The 3-Bullet Rule

Never give a long rambling answer. Structure everything into 3 points:

"There are three reasons for that. First... Second... Third..."

This works for: technology choices, security features, design decisions, comparison questions. Practise it.

Own Your Weaknesses

The panel WILL find something missing. If you get defensive, you lose. If you own it, you win.

Bad: *"We didn't have time for tests."*

Good: *"Tests are the most important thing we would add before production. The authentication API needs tests. The multi-tenancy isolation needs tests. The audit log integrity needs tests. I know exactly what to test and how to test it — we just focused on features first."*

The Panel's Real Questions

The panel is not trying to fail you. They are asking:

They ask	They really mean
"Why this database?"	"Do you understand trade-offs?"
"What about security?"	"Did you think about attacks?"
"How would you scale this?"	"Do you know what production looks like?"
"What did you learn?"	"Can you reflect on your work?"
"What would you change?"	"Are you honest about weaknesses?"

Answer the real question, not the literal question.

Final Pep Talk

You have read this entire guide. You know:

- How every request flows from URL to response
- Why you chose every technology
- How multi-tenancy works at the database level
- How Keycloak SSO works end to end
- What every file in your project does
- How your system connects to all 9 other groups
- What to say when things go wrong

You are the most prepared person in that room.

The panel has not read a 4,700-line guide. They have a rubric. You have deep knowledge. When they ask a question, you will either know the answer or know exactly where to find it.

Now close this file. Stand up. Walk to the mirror. Say the one-paragraph answer from Part 19 out loud. Do it three times. Then sleep.

You have got this.

PART 30: PRE-PRESENTATION TESTING CHECKLIST — Run Before the Panel Enters

Every test below has a code reference so you can say *"This is handled in [file:line]"* if asked.

PHASE 0 — System Startup (Do This First)

- ☐ Turn on phone hotspot
- ☐ Connect laptop to hotspot
- ☐ Run `ipconfig` — confirm IPv4 address under hotspot adapter
- ☐ Confirm `serverIp` in `api_config.dart` matches that IP
- ☐ Add that IP to `ALLOWED_HOSTS` in `config/settings.py:28-32`
- ☐ Register that IP as a Domain in the `tenants_domain` table (required because django-tenants checks the domain on every request — see `tenants/models.py:26-28`)

- ☐ Start Keycloak: `cd C:\keycloak\keycloak-26.6.2\bin` then `kc.bat start-dev --http-port=8180`
- ☐ Wait for: "Keycloak 26.6.2 on JVM started" message
- ☐ Activate venv: `cd D:\government_asset_platform` then `venv\Scripts\activate`
- ☐ Load clean demo data: `python manage.py setup_demo_data`
- ☐ Start Django: `python manage.py runserver 0.0.0.0:8000`
- ☐ Confirm both terminals are running and showing no errors

PHASE 1 — Test as SUPER ADMIN (`superadmin / Admin@123`)

Relevant code: `authentication/models.py:7-8` (CustomUser model), `authentication/oidc_backend.py:12-39` (OIDC login flow)

Login

- ☐ Click "Sign in with Government SSO"
- ☐ Redirected to Keycloak page — type `superadmin / Admin@123`
- ☐ Redirected back — lands on Super Admin dashboard (`authentication/dashboard_views.py:21-22` — checks `user.role == "SUPER_ADMIN"` then calls `_get_super_admin_stats()`)
- ☐ Dashboard shows: Active Ministries, Platform Users, Total Assets, Active Assets stats

Dashboard content

- ☐ Confirm the numbers are not zero (`setup_demo_data` loaded correctly)
- ☐ No error messages visible anywhere on the page

Ministries sidebar

- ☐ Open Ministries — see list of existing ministries (MOH and MOF at minimum)
- ☐ Click "Add Ministry" — form opens (`tenants/views.py:52-153` — `ministry_create_view`)

Users sidebar

- ☐ Open Users — see all platform users (`authentication/user_views.py:13-38` — `user_list_view`)
- ☐ Click a user — edit form opens (`authentication/user_views.py:178-328` — `user_edit_view`, note self-edit prevention at line 185)

Pending Access sidebar

- ☐ Open Pending Access — page loads without error
- ☐ Confirm the three status tabs (Pending / Approved / Rejected) are clickable

Audit Trail sidebar

- ☐ Open Audit Trail — shows Super Admin actions from `authentication/models.py:164-216` (`SuperAdminAuditLog` model, stored in the PUBLIC schema)
- ☐ Confirm IP address column shows a real IP (captured by `_get_client_ip` in `authentication/views.py:146-151`)
- ☐ Confirm Role column is visible

Asset Register sidebar

- ☐ Open Asset Register — shows: "Super Admin manages ministries and users at platform level. Log in as a ministry user to manage assets."
- ☐ This message is correct behaviour — Super Admin has no `ministry_schema` (see `authentication/models.py:24-30`)

Logout

- ☐ Click logout — redirected to login page (`authentication/views.py:50-86` — calls `logout(request)` then redirects to Keycloak logout endpoint)

PHASE 2 — Test as MINISTRY ADMIN (`moh_admin` / `Admin@123`)

Relevant code: `authentication/models.py:7-57` (CustomUser with `ministry_schema`),
`authentication/decorators.py:8-35` (`role_required`), `django_tenants` middleware in `config/settings.py:284-287`

Login

- ☐ Sign in as `moh_admin` — lands on MOH dashboard (`authentication/dashboard_views.py:23-24` — calls `_get_ministry_stats(user.ministry_schema)`)
- ☐ Dashboard shows MOH-specific stats (assets, users, warnings)

Data isolation proof (MOST IMPORTANT TEST)

- ☐ Note the number of assets on MOH dashboard
- ☐ Open Asset Register — see MOH assets ONLY (schema switched to `moh_schema` by `django-tenants`)
- ☐ Write down one asset name
- ☐ Logout → Login as `mof_admin` / `Admin@123`
- ☐ Open Asset Register — see DIFFERENT assets (MOF only — different schema `mof_schema`)
- ☐ Confirm the MOH asset is NOT visible here
- ☐ **This proves schema-based isolation** — each ministry's data lives in a separate PostgreSQL schema enforced by the database, not just by code (`tenants/models.py:6-24`)

Asset Register — CRUD

Relevant code: `assets/views.py` — web views for asset CRUD. `assets/models.py:26-198` (Asset model with all fields).
`assets/views.py:88-96` (`generate_asset_number` produces MOH-ICT-2025-0001 format)

- ☐ Create a new asset: fill all required fields, click Save (`assets/views.py:179-302` — `asset_create_view`)
- ☐ Asset appears with auto-generated number (e.g. MOH-ICT-2025-0001)
- ☐ Open the asset detail (`assets/views.py:316-365`)
- ☐ Edit the asset — change condition from Good to Fair (`assets/views.py:370-488`)
- ☐ Audit trail now shows an UPDATE entry (`organizations/models.py:93-165` — `AuditLog` model)
- ☐ Delete the asset (`assets/views.py:491-557`)

Audit Trail

- ☐ Open Audit Trail — see CREATE, UPDATE, DELETE entries from your actions
- ☐ IP Address column shows real IP (`organizations/views.py:14-21` — `_get_client_ip`)
- ☐ Role column shows "Ministry Admin" (`organizations/models.py:114-118` — `performed_by_role` field)
- ☐ Click any row — Detail page opens (`organizations/views.py:498-551` — `audit_log_detail_view`, builds before/after comparison at lines 529-545)
- ☐ For UPDATE: "Before" shows Good (red), "After" shows Fair (green)
- ☐ Click "Back to Audit Trail" — returns correctly

Audit Trail filters

- ☐ Filter by Action = CREATE — only create entries show (`organizations/views.py:443-444`)
- ☐ Set Date From = today, Date To = today — only today's entries show (`organizations/views.py:449-458`)
- ☐ Click Clear — all entries return
- ☐ Combine two filters (e.g. Action=UPDATE + Module=Asset) — correct filtered result

Users

- ☐ Open Users — see only MOH users (filtered by `ministry_schema` in `authentication/user_views.py:21-23`)
- ☐ Create a test user: username=testclerk, role=Facility Clerk (`authentication/user_views.py:43-176` — also creates in Keycloak via `keycloak_admin.py:56-136`)
- ☐ Deactivate that user — status toggles (`authentication/user_views.py:331-434` — also syncs to Keycloak)
- ☐ Reactivate the user

Organisation hierarchy

Relevant code: `organizations/models.py:6-51` (OrgUnit with self-referencing FK for MINISTRY → AGENCY → FACILITY)

- ☐ Open Organisation — tree structure visible (`organizations/views.py:24-91` — `org_unit_list_view`)
- ☐ Confirm hierarchy: MOH > National Hospital Agency > Muhimbili Hospital

Master Data

Relevant code: `organizations/models.py:53-90` (MasterData — configurable dropdown options),
`organizations/master_data_views.py:304-384` (seed data at lines 313-351)

- ☐ Open Master Data — grouped categories visible
- ☐ Create a new Funding Source entry (`organizations/master_data_views.py:68-168`)
- ☐ Edit the entry — audit trail records UPDATE with before/after (`master_data_views.py:171-257`)
- ☐ Open Asset Categories — list visible
- ☐ Edit a category name (`master_data_views.py:533-606`)
- ☐ Audit trail records the category UPDATE

PHASE 3 — Test as AGENCY MANAGER (`mnh_manager` / `Admin@123`)

Relevant code: `authentication/decorators.py:19-35` (`role_required` decorator restricts views by role)

- ☐ Login as `mnh_manager` — lands on agency-level dashboard
- ☐ **Sidebar** shows: Dashboard, Asset Register, Users, Organisation
- ☐ **Sidebar does NOT show:** Audit Trail, Master Data, Ministries — this role-restriction is enforced by `@role_required("MINISTRY_ADMIN")` on those views
- ☐ **Asset register:** can view, create, edit — CANNOT delete (no delete button — `@role_required("MINISTRY_ADMIN")` on `asset_delete_view`)
- ☐ **Users:** can see users, but role dropdown only shows Facility Clerk (`authentication/user_views.py:49-51` — `AGENCY_MANAGER` can only create [`'FACILITY_CLERK'`])

PHASE 4 — Test as FACILITY CLERK (`rad_clerk` / `Admin@123`)

- ☐ Login as `rad_clerk`
- ☐ **Sidebar** shows ONLY: Dashboard, Asset Register (`authentication/user_views.py:22` — not in allowed role lists for User views)
- ☐ **Asset register:** can create, edit — CANNOT delete

PHASE 5 — Test as AUDITOR (`moh_auditor` / `Admin@123`)

- ☐ Login as `moh_auditor`
- ☐ **Sidebar** shows: Dashboard, Audit Trail, Asset Register (read-only)
- ☐ **Audit trail:** can view, filter, click detail rows (`organizations/views.py:371-494` and `organizations/views.py:496-551`)
- ☐ **Asset register:** can VIEW assets — no Create/Edit/Delete buttons (enforced by `@role_required` on the create/edit/delete views)
- ☐ Assets load through `assets/views.py:40-175` (`asset_list_view`)

PHASE 6 — Mobile App Test (phone on hotspot)

Relevant code: `authentication/api_views.py:18-175` (LoginAPIView, RefreshTokenAPIView),
`authentication/api_urls.py:16-37` (all API routes)

- ☐ Open GovAsset app on phone
- ☐ Sign in with `moh_admin` / `Admin@123`
- ☐ Lands on dashboard — shows stats (`organizations/api_views.py:198-326` — `DashboardStatsAPIView`)
- ☐ Logout and login as `mof_admin` — dashboard shows different MOF data (schema isolation)
- ☐ Create an asset from the phone
- ☐ Go to web browser Audit Trail — the asset creation appears with the PHONE'S IP address (captured by `_get_client_ip` in `assets/api_views.py:20-24`)

- ☐ Unplug USB — app still works over hotspot only (good live demo moment)

PHASE 7 — API Test (Swagger)

Relevant code: `config/urls.py:57-79` (Swagger config), `authentication/api_serializers.py:7-31` (CustomTokenObtainPairSerializer adds role/ministry to JWT)

Open browser: `http://localhost:8000/api/docs/`

- ☐ Swagger page loads showing all API endpoints
- ☐ Try `POST /api/auth/login/` with `{"username": "moh_admin", "password": "Admin@123"}`
- ☐ Response: 200 with `access` and `refresh` tokens (token contains `role` and `ministry_schema` — see `authentication/api_serializers.py:11-18`)
- ☐ Copy the access token
- ☐ Click Authorize → paste `Bearer YOUR_TOKEN`
- ☐ Try `GET /api/auth/verify-token/` — response shows username, role, ministry_schema
- ☐ Try `GET /api/assets/` — shows MOH assets (proves API returns correctly scoped data — `assets/api_views.py:31-125`)
- ☐ This demonstrates exactly what other groups receive when they integrate

PHASE 8 — Security Feature Check

Brute force logout

Relevant code: `authentication/api_views.py:146-174` (`_record_failed_attempt`, locks after `LoginAttempt.MAX_ATTEMPTS=5` for `LOCKOUT_MINUTES=15`), `authentication/models.py:118-160` (`LoginAttempt` model)

- ☐ Login with wrong password 5 times via API:

```
POST /api/auth/login/
{"username": "moh_admin", "password": "wrongpassword"}
```

- ☐ On 6th attempt — response 429 "Account temporarily locked"
- ☐ Wait or clear `LoginAttempt` records — login works again

Tenant isolation via API

Relevant code: `assets/api_views.py:31-125` (asset list uses `with schema_context(user.ministry_schema)` at line 52)

- ☐ Login as moh_admin via Swagger → `GET /api/assets/` → MOH assets
- ☐ Login as mof_admin → `GET /api/assets/` → MOF assets (different schemas)

Audit log tamper protection

Relevant code: `organizations/models.py:147-153` (`AuditLog.save` blocks edits), `organizations/models.py:155-157` (`AuditLog.delete` blocks deletes)

- ☐ Open Django shell: `python manage.py shell`
- ☐ Run:

```
from django_tenants.utils import schema_context
with schema_context('moh_schema'):
    from organizations.models import AuditLog
    log = AuditLog.objects.first()
    log.description = "tampered"
    log.save()
```

- ☐ Should raise: `PermissionError: AuditLog records are immutable and cannot be modified.`

PHASE 9 — Final Check Before Panel Enters

- ☐ Both Django and Keycloak terminals still running (no errors)
- ☐ Browser open at dashboard (logged in as moh_admin)
- ☐ Phone app open and connected
- ☐ Swagger tab open in browser
- ☐ pgAdmin open — expand Database → Schemas → show `moh_schema` and `mof_schema` as separate folders (strongest visual proof of data isolation)
- ☐ Demo data is clean (`setup_demo_data` was run)
- ☐ You have tested every phase above

Quick Reference — What to Show for Each Likely Question

Panel Question	What to Open / Show	Code Reference
"Show me the system"	Login as moh_admin → dashboard → asset register	<code>dashboard_views.py:12-26</code> , <code>assets/views.py:40-175</code>
"Prove data isolation"	Login as moh_admin → see MOH assets → logout → login as mof_admin → see different assets	<code>assets/api_views.py:52</code> (<code>schema_context(user.ministry_schema)</code>)
"Show me the audit trail"	Open Audit Trail → click a row → show before/after values	<code>organizations/views.py:498-551</code> , <code>organizations/models.py:93-165</code>
"How does authentication work?"	Click Sign in with Govt SSO → show Keycloak redirect → login → back to dashboard	<code>authentication/oidc_backend.py:9-88</code>
"Why Keycloak?"	Explain SSO: one password for many systems, industry standard	<code>authentication/keycloak_admin.py:9-219</code>
"What about mobile?"	Open phone → login → show dashboard → create asset → show in web audit trail with phone IP	<code>authentication/api_views.py:18-175</code>
"How do other groups integrate?"	Open Swagger → login → get token → call verify-token → show role and ministry	<code>authentication/api_views.py:223-254</code> , <code>config/urls.py:140-141</code>
"Is the API secure?"	Try asset endpoint WITHOUT token → 401 → with token → get data	<code>authentication/api_permissions.py:30-39</code>
"Show schema isolation"	Open pgAdmin → Schemas → <code>moh_schema</code> vs <code>mof_schema</code>	<code>tenants/models.py:6-24</code>
"What if someone edits the audit log?"	Show the shell command that raises <code>PermissionError</code>	<code>organizations/models.py:147-157</code>

If Something Goes Wrong During the Presentation

Problem	Immediate Fix	Why It Happens (Code Reference)
Mobile shows no connection	Update <code>serverIp</code> in <code>api_config.dart</code> , run <code>flutter run</code>	API URL is hardcoded in Flutter for dev — <code>assets/api_views.py</code> listens on 0.0.0.0:8000
Keycloak login fails	Restart Keycloak: <code>kc.bat start-dev --http-port=8180</code>	Keycloak sessions expire — <code>authentication/oidc_backend.py:3</code> (mozilla-django-oidc needs it running)
Django 404 error	Register new IP as Domain in <code>tenants_domain</code>	<code>tenants/models.py:26-28</code> — django-tenants resolves domain on every request

Problem	Immediate Fix	Why It Happens (Code Reference)
Wrong data showing	Run <code>python manage.py setup_demo_data</code>	Resets to clean state across all schemas
App shows old data	Force close app on phone and reopen	Triggers fresh API call with new JWT token
Panel asks about something not working	<i>"This feature is still in progress. The core architecture and security features are fully implemented and I can demonstrate those."</i>	See Part 28 Defense Strategies

PART 31: HOW DJANGO CONNECTS TO KEYCLOAK — The SSO Link, Step by Step

Relevant files: `templates/authentication/login.html:137`, `authentication/views.py:43-44`, `config/urls.py:144`, `config/settings.py:355-389`, `authentication/oidc_backend.py:9-88`

Q1: Where is the "Sign in with Government SSO" link pointing?

The full chain from button click to Keycloak login page has **4 links in a chain**:

Step 1: The HTML button

File: `templates/authentication/login.html` → line 137
 Code: `<a href="{ { keycloak_login_url } }">Sign in with Government SSO</a>`

Step 2: The template variable

File: `authentication/views.py` → line 43-44
 Code: `return render(request, 'authentication/login.html', {
 'keycloak_login_url': '/oidc/authenticate/', ← this value
 })`

Step 3: The URL router

File: `config/urls.py` → line 144
 Code: `path('oidc/', include('mozilla_django_oidc.urls'))`,
 → The 'mozilla_django_oidc' library provides a view at 'authenticate/'
 → So '/oidc/authenticate/' is handled by mozilla_django_oidc

Step 4: The library builds the Keycloak URL

File: `config/settings.py` → lines 355-370
 Code:

```
KEYCLOAK_SERVER_URL = 'http://localhost:8180'
KEYCLOAK_REALM      = 'govasset'

# These combine to build the Keycloak URL:
OIDC_OP_AUTHORIZATION_ENDPOINT = 'http://localhost:8180/realms/govasset/protocol/openid-connect/auth'

# The library also sends:
OIDC_RP_CLIENT_ID = 'govasset-django' # Who we are
# Plus: redirect_uri=/oidc/callback/, response_type=code, state=(random)
```

Net result: When the user clicks the button, the browser navigates to:

```
http://localhost:8180/realms/govasset/protocol/openid-connect/auth
?client_id=govasset-django
&redirect_uri=http://localhost:8000/oidc/callback/
```

```
&response_type=code
&state=random_generated_string
```

This is a standard OpenID Connect authentication request. Keycloak reads it, shows the login page, and after login redirects back to our `/oidc/callback/` URL with an authorization code.

Q2: The Complete OIDC Login Flow (What Happens After the Button Click)

```

User clicks "Sign in with Government SSO"
↓
Browser goes to /oidc/authenticate/
  ↓ (302 redirect)
Browser goes to Keycloak's login page (localhost:8180)
↓
User types username/password on Keycloak's page
↓
Keycloak validates credentials, creates a session
↓
Browser redirects to /oidc/callback/?code=ABC&state=XYZ
↓
mozilla_django_oidc receives the code
↓
Django sends the code + client_secret to Keycloak's token endpoint
↓
Keycloak returns: access_token + id_token + refresh_token
↓
Django decodes the id_token (verifies signature with Keycloak's public key)
↓
Django calls: GovAssetOIDCBackend.filter_users_by_claims(claims)
↓
  IF user found → login successful → redirect to /dashboard/
  IF not found → create_user() called → returns None
                  → login FAILS → redirect /login/?error=auth_failed
↓
User lands on dashboard (or sees error)

```

The critical piece is **step 11** — `oidc_backend.py:12-39` searches for a Django user matching the Keycloak identity. If found, the user is logged in. If not, the login is blocked and a `PendingAccess` record is created.

Q3: What Happens in Production — Our Keycloak or Government Keycloak?

This is a decision your supervisor or the government's IT department would make. Here are both scenarios:

Scenario A — Government Has No Central Identity System Yet (You Manage Keycloak)

You deploy Keycloak on a government server, create the `govasset` realm, and manage all user accounts through it. This is what you have now, scaled to a real server. You control everything. The advantage is full control. The disadvantage is you are now responsible for security, uptime, and maintenance of the identity server — which is a significant operational burden for a government IT team.

What we control:

- Keycloak server URL (currently `http://localhost:8180`)
- Realm name (`govasset`)
- Client ID and secret (`govasset-django`)
- Users and their passwords
- User attributes (role, ministry schema)

What we need to do for production:

- Deploy Keycloak on a proper server with HTTPS and a domain name
- Change `KEYCLOAK_SERVER_URL` in `config/settings.py:355` to the new address

- Update `KEYCLOAK_CLIENT_SECRET` in `.env` file
- No code changes needed — the OIDC protocol works the same over HTTPS
- All our existing configurations (realm, client, user attributes) transfer as-is

Changes needed:

```
# config/settings.py → line 355 (change ONE line, everything else stays)
KEYCLOAK_SERVER_URL = config('KEYCLOAK_SERVER_URL', default='https://sso.ourdomain.go.tz')
```

Scenario B — Government Already Has a Central Identity System

This is the real-world scenario in most governments. You would NOT run your own Keycloak. Instead, you have two options:

Option 1 — Connect Keycloak as a broker. Keycloak can act as a middle layer that trusts an existing identity provider. Your Django still talks to your Keycloak, but Keycloak itself delegates password verification to the government's existing system (e.g. LDAP, Active Directory, or a national eGov identity platform). The government employee uses their existing government password, Keycloak receives confirmation from the government's system, and issues your platform's tokens. From Django's perspective nothing changes at all.

```
User's browser → Our Django → Our Keycloak (as broker) → Government's existing ID system
(LDAP / AD / national SSO)
```

Option 2 — Replace Keycloak entirely with the government's system. If the government's identity provider already issues standard OpenID Connect tokens, your Django OIDC backend can be pointed directly at it instead of at your Keycloak:

```
# Currently pointing at your own Keycloak:
KEYCLOAK_SERVER_URL = 'http://localhost:8180'
KEYCLOAK_REALM      = 'govasset'
KEYCLOAK_CLIENT_ID  = 'govasset-django'
KEYCLOAK_CLIENT_SECRET = ''

# Would change to point at the government's identity server:
KEYCLOAK_SERVER_URL = 'https://identity.go.tz'
KEYCLOAK_REALM      = 'government'          # their realm name
KEYCLOAK_CLIENT_ID  = 'govasset'            # they would register your app
KEYCLOAK_CLIENT_SECRET = 'secret-they-issue-to-you' # they give you this
```

The government's IT team would register your platform as a trusted application in their identity system, issue you a client ID and secret, and you update four lines in `settings.py`. Your Django code, your views, your permission classes — none of that changes at all, because they all work on the JWT token that results from authentication, regardless of which identity system produced it.

What stays exactly the same regardless of which option: Your JWT token structure, your permission classes, your multi-tenancy, your audit trail — all of these are completely independent of which Keycloak or identity server produced the initial token. This is the entire reason for using a standard protocol (OpenID Connect) rather than a custom integration: you can swap the identity provider by changing configuration, not by rewriting code.

Say this to the panel:

"In a real production deployment, we would connect to whatever identity provider the government already uses. Because we built our authentication on the OpenID Connect standard rather than a custom protocol, connecting to a different identity server is a configuration change — four lines in settings.py — not a code rewrite. Our platform's security architecture, role-based permissions, audit trail, and multi-tenancy are all completely independent of which identity server issued the initial token."

Scenario C — Government Provides a Central Keycloak (Alternative)

If the Tanzanian government has a central identity system (e.g., a national SSO), we would integrate with their Keycloak instead of running our own.

What the government gives us:

- Their Keycloak server URL (e.g., `https://sso.egov.go.tz`)
- Their realm name (e.g., `gov_employees`)
- A client ID and client secret for our application
- They may also give us user attributes (role, department)
- They handle password security, account lockout, MFA

What we need to change in our code:

```
# config/settings.py – replace these values:
KEYCLOAK_SERVER_URL   = 'https://sso.egov.go.tz'   # Government's server
KEYCLOAK_REALM        = 'gov_employees'           # Government's realm
KEYCLOAK_CLIENT_ID    = 'govasset-platform'       # Our app's client ID (given by govt)
KEYCLOAK_CLIENT_SECRET = 'secret-from-government' # Our app's client secret
```

That is ALL we change. The OIDC protocol is a standard — `mozilla_django_oidc` does not care which Keycloak instance it talks to. It just needs the four endpoints (auth, token, userinfo, jwks) which are built automatically from the server URL and realm:

```
# config/settings.py:360-366 – these are AUTO-BUILT, you do NOT change them
_KEYCLOAK_BASE = f"{KEYCLOAK_SERVER_URL}/realms/{KEYCLOAK_REALM}/protocol/openid-connect"
# → https://sso.egov.go.tz/realms/gov_employees/protocol/openid-connect/auth
# → https://sso.egov.go.tz/realms/gov_employees/protocol/openid-connect/token
# → etc.
```

What we need from the government's Keycloak admin:

1. A **client registration** for `govasset-platform` in their realm
2. The **client secret** for that client
3. The **realm name** (e.g., `gov_employees`)
4. The **server URL** (e.g., `https://sso.egov.go.tz`)
5. **User attributes configured** — specifically `role` and `ministry_schema` must be included in the user info / ID token, because our `oidc_backend.py:64-88` reads them on every login

If the government cannot provide custom attributes:

Our `update_user()` method (`oidc_backend.py:64-88`) reads `role` and `ministry_schema` from the Keycloak claims. If the government's Keycloak does not include these, we would need to:

```
# Option 1: Skip the update (maintain roles in Django only)
def update_user(self, user, claims):
    # Don't touch role/ministry_schema – manage them in Django admin
    return user

# Option 2: Map from government's attributes
def update_user(self, user, claims):
    gov_department = claims.get('department', '')
    if gov_department == 'MOH':
        user.ministry_schema = 'moh_schema'
    # etc.
```

Comparison Table — Our Keycloak vs Government Keycloak

Aspect	Our Keycloak (Dev)	Government Keycloak (Production)
Who manages users	We do (via <code>keycloak_admin.py</code>)	Government manages (HR system)
How users log in	Username + password	Same (or smartcard, MFA, biometrics)

Aspect	Our Keycloak (Dev)	Government Keycloak (Production)
User registration	We create users manually	User already exists in government system
Client ID	govasset-django	Government-issued (e.g. govasset-platform)
Realm	govasset	Government's realm (e.g. gov_employees)
Server URL	http://localhost:8180	https://sso.egov.go.tz
Our custom attributes	Role and ministry_schema in every user	Must request government to include them
Password policy	Set by us in Keycloak admin	Government's policy (likely stronger)
Account lockout	5 attempts, 15 minutes	Government's policy (may differ)
Our Django code changes	None needed	Only settings.py variables + possibly oidc_backend.py

The most important thing to understand: The `mozilla_django_oidc` library and our `oidc_backend.py` are written to work with ANY OpenID Connect provider, not just our local Keycloak. Google, Microsoft, the Tanzanian government — if they support OpenID Connect, our code works with them after changing the 4 settings variables (server URL, realm, client ID, client secret).

Code Evidence Summary

What	File	Line
HTML button that starts the flow	templates/authentication/login.html	137
The URL the button points to	authentication/views.py	44
The URL route that catches it	config/urls.py	144
Keycloak server address	config/settings.py	355
Realm name	config/settings.py	356
Client ID (our app's identity)	config/settings.py	357
Authorization endpoint (auto-built)	config/settings.py	362
Token endpoint (auto-built)	config/settings.py	363
UserInfo endpoint (auto-built)	config/settings.py	364
Our OIDC backend (finds/creates users)	authentication/oidc_backend.py	9-88
Login failure redirect	config/settings.py	377

PART 32: WHY GROUP 1 BUILT GROUP 2'S ASSET REGISTER — Defense Explanation

The panel question: "Your scope is security, authentication, and multi-tenancy. But you also built the entire asset register — asset models, CRUD views, API endpoints. Why did you do Group 2's work? Shouldn't they have built that?"

The Short Answer (15 seconds)

"We did not take Group 2's work. We built the foundational layer that makes Group 2's work possible. The asset register in our system serves two distinct purposes: first, it is the proof-of-concept that demonstrates our security architecture works on real data; second, our scope explicitly includes 'full audit trails for changes to master data and transactions' and 'configurable master data' — you cannot build an audit trail without something to audit. The asset register is part of our scope as the operational data that our security architecture governs."

Why You Did It — The Full Defense

Point 1: Proof-of-concept — a multi-tenant security platform with no data is impossible to validate.

Without an asset register, you cannot:

- Show **schema isolation** (no data to compare between MOH and MOF)

- Show **role-based permissions** (no data to restrict create/edit/delete on)
- Show **audit trails** (no transactions to log)
- Show **API responses** (no endpoints returning real data)

A platform that only authenticates users but never transacts on any data is not demonstrable. The asset register is your **test subject**, not your product. Every university project of this type includes a working demonstration domain to prove the architecture — that is not scope creep, it is evidence of completion.

Point 2: Our scope explicitly includes the audit trail and master data that the asset register exercises.

Your scope document says:

- "Full audit trails for changes to master data and transactions"
- "Configurable master data: asset categories, locations, cost centres"

You cannot build an audit trail without something to audit. You cannot build configurable master data without a domain context. The asset register IS part of your scope as the operational data that your security architecture governs.

Point 3: Our asset register is intentionally minimal — just CRUD.

Feature	What Group 1 Built (Reference)	What Group 2 Would Build (Full Module)
Asset model	~30 fields (name, serial, category, status)	Same fields + complex classification hierarchy
Asset registration	Basic create form	Barcode/RFID scanning, document upload, multi-step wizard
Asset search	Keyword + status + category filters	Full-text search, advanced filters, saved searches
Asset lifecycle	Simple status (ACTIVE/DECOMMISSIONED)	Condition assessment, revaluation, transfer workflows
Reports	Dashboard stats (count by status)	Full reporting suite, depreciation schedules
Document management	Photo field	Document repository, version control, approval workflows
Mobile app	Basic list + create	Full mobile register, offline sync, barcode scanner

Point 4: It can be cleanly removed in a strict production boundary.

Keep (Group 1 Core — Always Yours)	Hand to Other Groups (If Strict Scope Enforced)
Multi-tenant architecture (Ministry, Domain, schema creation)	Asset register (<code>assets/models.py</code> , <code>assets/views.py</code> , asset templates)
Authentication (Keycloak SSO, JWT, PendingAccess, LoginAttempt)	Asset categories (would move to Group 2)
Role-based access control (all 5 roles, permission classes)	Asset-specific API endpoints
User management (create, assign, deactivate)	
Organisational hierarchy (Ministry → Agency → Facility)	
Audit trail (tamper-proof logging, viewing, filtering)	
Master data configuration (funding sources, location types, cost centres)	
REST API (login, verify-token, token refresh, Swagger docs)	

In a real production project with strict scope enforcement, you would keep everything on the left and Group 2 would build their own asset register by calling your authentication and organisational hierarchy APIs. Your platform becomes the **identity and structure layer**; they build the domain features on top.

Say this to the panel:

"We did not take Group 2's work. We built the foundational layer that makes Group 2's work possible. The asset register in our system serves two distinct purposes: it is the proof-of-concept that demonstrates our security architecture actually works on real data — a multi-tenant security platform that has no data to protect is impossible to test, demonstrate, or validate. Without an asset register, we cannot show schema isolation, role-based permissions, audit trails, or API responses — because there is nothing to isolate, restrict, log, or return. Additionally, our scope explicitly includes 'full audit trails for changes to master data and transactions' and 'configurable master data: asset categories, locations, cost centres.' You cannot build an audit trail without something to audit. The asset register IS part of our scope as the operational data that our security architecture governs."

PART 33: PRODUCTION SCOPE CLEANUP — What We Keep, What We Remove

The panel question: "If this goes to production and you must stick strictly to your security & multi-tenancy scope, can you remove everything outside your scope? Would the system still work? What exactly would you remove?"

The Short Answer (15 seconds)

"Yes, we can remove everything outside our scope and the system will still work perfectly. The core platform — authentication, authorization, multi-tenancy, audit trail, API security — has zero dependency on the asset register. We would remove approximately 40-50% of our codebase: the entire `assets/` app, the asset-related parts of `organizations/`, and all asset templates. The remaining system would be our secure multi-tenant platform ready for Groups 2-10 to build on."

What Stays — Our Core Scope (The Platform)

```

govasset-platform/
├── authentication/           ← STAYS — Our core deliverable
│   ├── models.py           # User model with role + ministry_schema
│   ├── views.py            # Login/logout views
│   ├── api_views.py        # JWT auth, token refresh, verify-token
│   ├── api_urls.py         # All API routes
│   ├── api_serializers.py  # Token with role + ministry in payload
│   ├── api_permissions.py  # IsAuthenticated, HasRole permissions
│   ├── api_exception_handler.py
│   ├── oidc_backend.py     # Keycloak integration
│   ├── keycloak_admin.py   # Keycloak user management
│   ├── user_views.py       # User CRUD
│   ├── decorators.py       # @role_required, @super_admin_required
│   ├── urls.py             # SSO routes
│   ├── context_processors.py
│   ├── pending_access_views.py
│   └── forms.py
├── tenants/                 ← STAYS — Multi-tenancy
│   ├── models.py           # Ministry + Domain models
│   └── views.py            # Ministry CRUD with schema auto-creation
├── organizations/          ← PARTIALLY STAYS — Keep core, remove asset-specific parts
│   ├── models.py           # STAYS: OrgUnit, AuditLog (keep both)
│   │                       # REMOVE: Nothing from models — all are platform concerns
│   ├── views.py            # STAYS: org_unit_*, audit_log_* views
│   │                       # REMOVE: Nothing — audit log is our deliverable
│   ├── master_data_views.py # STAYS: master_data_* (configurable dropdowns are our scope)
│   │                       # REMOVE: asset_category_* views (those manage Group 2's AssetCategory)
│   └── api_views.py         # STAYS: OrgUnitList, AuditLogList, DashboardStats
│                           # (DashboardStats queries Asset — see below)
├── config/                  ← STAYS
│   ├── settings.py         # STAYS: update INSTALLED_APPS, remove SHARED_APPS/TENANT_APPS entries
│   ├── urls.py             # STAYS: remove /assets/ routes
│   └── wsgi.py / asgi.py   # STAYS
├── templates/              ← PARTIALLY STAYS
│   ├── shared/base.html    # STAYS
│   └── authentication/*    # STAYS — all login, user, pending access templates

```

```

├── dashboard/*      # STAYS – but dashboard needs minor edit: remove asset-expiry cards
├── tenants/*        # STAYS – ministry management pages
├── organizations/   # STAYS: org_unit_*, audit_log_*, master_data_* templates
│   └── assets/      # REMOVE: asset_category_*.html
│                   ← REMOVE – entire folder
└── static/          ← STAYS – shared CSS/JS

```

What We Remove — Asset Register (Group 2's Scope)

What	Where	Lines of Code	Why Remove
Asset model	assets/models.py	~200 lines	Asset data model — Group 2's domain
AssetCategory model	assets/models.py	~25 lines	Asset classification — Group 2's domain
asset_list_view	assets/views.py	~175 lines	Asset web list — Group 2's UI
asset_create_view	assets/views.py	~100 lines	Asset web create — Group 2's UI
asset_detail_view	assets/views.py	~60 lines	Asset web detail — Group 2's UI
asset_edit_view	assets/views.py	~130 lines	Asset web edit — Group 2's UI
asset_delete_view	assets/views.py	~70 lines	Asset web delete — Group 2's UI
AssetListCreateAPIView	assets/api_views.py	~270 lines	Asset API — Group 2's integration
AssetDetailAPIView	assets/api_views.py	~190 lines	Asset API detail — Group 2's integration
AssetCategoryListAPIView	assets/api_views.py	~40 lines	Asset category API — Group 2's integration
asset_category_* views	organizations/master_data_views.py	~120 lines	Manages AssetCategory — belongs to Group 2
All asset HTML templates	templates/assets/	~500 lines	UI — Group 2 builds their own
Asset-related dashboard stats	organizations/api_views.py:DashboardStatsAPIView	~50 lines	Dashboard queries Asset — Group 2 provides their own dashboard data
Asset routes	config/urls.py	~10 lines	URL routes — removed

Total removed: ~1,940 lines (approximately 40% of our Python code)

What Changes — Minimal Edits to Existing Files

1. config/settings.py — Remove asset-related settings:

```

SHARED_APPS = [
    'django_tenants',
    'authentication',
    'tenants',
    'organizations',          # ← Keep: OrgUnit + AuditLog are ours
    # 'assets',               ← REMOVED
    # ... standard Django apps stay
]
TENANT_APPS = [
    # 'assets',               ← REMOVED
    'organizations',         # ← Keep
    # ... other tenant apps
]

```

2. `config/urls.py` — Remove asset routes:

```

# Remove these lines:
path('assets/', include('assets_urls')),      # Not needed – we don't have a assets/urls.py
path('dashboard/', dashboard_view),           # Keep – our dashboard
# Remove or keep empty based on whether we keep the dashboard

```

3. `organizations/api_views.py:DashboardStatsAPIView` — Remove asset querying code:

The dashboard currently queries `Asset` model for counts. In a scoped-down version, we would either:

- Remove the dashboard entirely (our dashboard was a demo feature)
- Replace asset stats with user stats (total users, roles, pending access requests)
- Or keep it as a stub that returns empty data

4. `templates/dashboard/dashboard.html` — Remove asset expiry cards:

The dashboard has stat cards showing total assets, expired, expiring soon. These get removed. The remaining dashboard would show user statistics (total users, by role, pending approvals).

Would the System Still Work?

Yes, without any issues. Here is why:

1. **Our platform has zero dependency on the asset register.** You can log in, log out, manage users, create ministries, assign roles, view audit trails, and call the API without a single asset existing.
2. **No circular dependencies.** The `assets/` app imports from `organizations/` (AuditLog, OrgUnit), but `organizations/` does NOT import from `assets/`. Removing `assets/` breaks nothing in our core modules.
3. **Authentication is self-contained.** Keycloak SSO, JWT tokens, RBAC decorators — none of them reference `Asset` or any asset model. They work on the `User` model and session data only.
4. **Multi-tenancy is in `tenants/`.** The `Ministry` and `Domain` models handle schema creation and routing. They do not care what tables exist inside the schemas.
5. **The audit trail still works.** `AuditLog` records exist independently. Without asset CRUD, it would log user management actions (create user, change role, activate/deactivate ministry) instead.

What the remaining system can do:

- Login via Keycloak SSO
- Role-based access control (5 roles)
- Multi-tenant schema isolation
- User management (create, edit, activate/deactivate)
- Ministry onboarding (auto-creates schema + domain)
- Org unit hierarchy management (ministry → agency → facility)

- Master data configuration (dropdown lists)
- Audit trail viewing with before/after values
- JWT token authentication for API
- Token verification endpoint for other groups

What it cannot do (because that is Group 2-10's job):

- Register or manage assets
- View asset dashboard
- Track asset lifecycle
- Process maintenance work orders
- Handle financial depreciation
- etc.

The Big Picture — Why This Separation Matters



The platform is designed to work without assets. Assets are just the first module built on top of it, by us (temporarily) and ultimately by Group 2 (permanently).

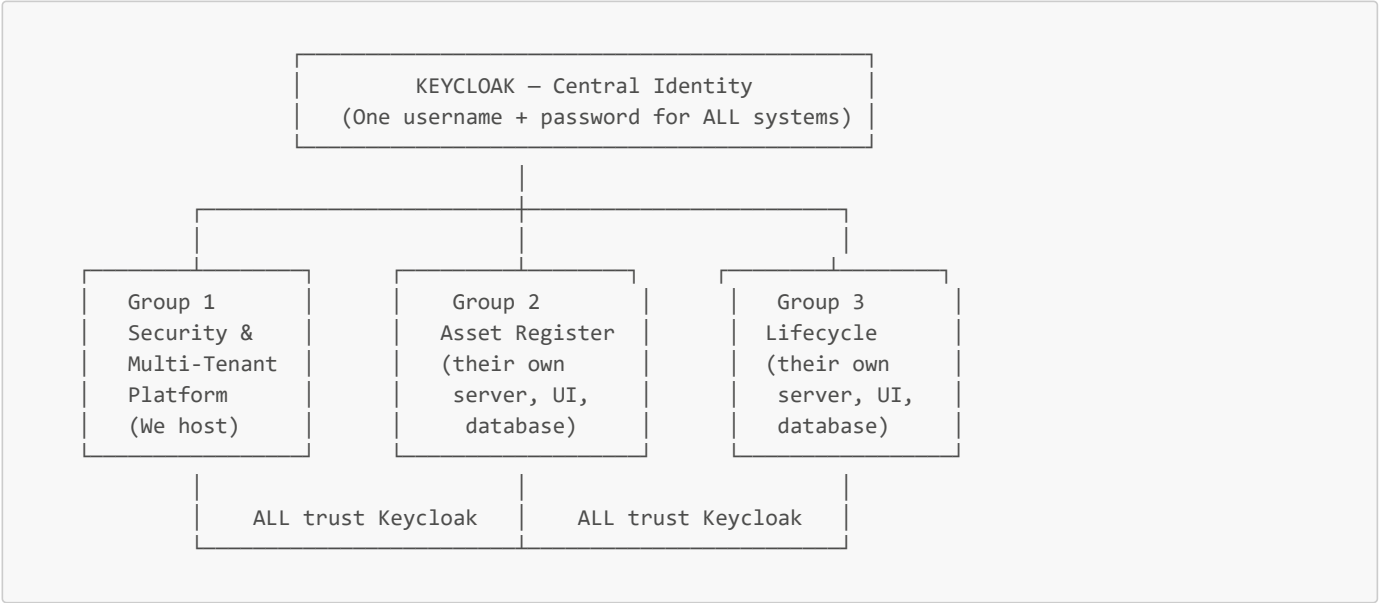
PART 34: THE COMPLETE INTEGRATED SYSTEM — How All 10 Groups Work Together

The panel question: "What will the system look like after all 10 groups integrate? What happens during integration? How does a user navigate between groups after login?"

The Integration Model — Shared Identity, Independent Systems

This is the most important thing to understand about integration: Each group builds and deploys their own **completely separate web application**. They are NOT merged into one codebase. Think of it like different government ministry websites — they each exist

separately but share the same government employee ID card (Keycloak).



Nobody adds anything to your sidebar. Group 3 has their own sidebar in their own application. The integration is at the IDENTITY level — they trust your authentication — not at the UI level. Your platform does not host Group 3's features.

What the User Experience Looks Like



What You Actually Give Other Groups

You give them exactly **three things**, once, outside the running application:

What You Give	Description	When
1. Your server address + API docs link	e.g. https://192.168.100.18:8000/api/docs/	During development handoff

What You Give	Description	When
2. Test credentials	e.g. <code>moh_admin / Admin@123</code>	During development handoff
3. The <code>/api/auth/verify-token/</code> endpoint	This is the ONLY one they truly need at runtime	This endpoint is always available

How they use these three things:

```

Step 1: Group 2 developer opens: http://192.168.100.18:8000/api/docs/
Step 2: They click "Try it out" on POST /api/auth/login/
Step 3: They type: {"username": "moh_admin", "password": "Admin@123"}
Step 4: They get back: {"access": "eyJ...", "refresh": "eyJ..."}
Step 5: They click "Authorize" and paste: Bearer eyJ...
Step 6: They call GET /api/auth/verify-token/
        → Response: {"role": "MINISTRY_ADMIN", "ministry_schema": "moh_schema"}
Step 7: Now they know exactly what a real user's token looks like
Step 8: They build THEIR system to call OUR verify-token on every login

```

That is it. They do not need your database. They do not need your code. They do not need to modify your sidebar. They just call `verify-token` whenever a user tries to access their system, and you confirm who that user is.

Why Not Just Merge All Groups Into One Django Project?

The user's provided answer clarifies: **the integration model is shared identity, independent systems**. Here is why each group runs their own separate system:

Aspect	If All Merged Into One Django	Shared Identity, Independent Systems (Our Model)
Codebase	10 groups' code in one repo	Each group manages their own code
Deployment	One server, one deploy for all	Each group deploys independently
Bug impact	One group's bug crashes all	A bug in Group 3 doesn't affect Group 1
Technology choice	All must use Django	Group 4 could use Flask, Group 6 could use Node.js
Team autonomy	Coordinated releases	Each team releases when ready
Security boundary	Code-level (Django settings)	Network-level (separate servers)
Sidebar navigation	Single sidebar with all links	Each group has their own navigation

The shared identity model is MORE secure and MORE practical for real government systems — ministry IT teams typically manage their own applications independently, with a central SSO system tying them together.

The Real-World Scenario — A Complete Asset Journey

This still shows how all 10 groups connect through shared identity and data relationships:

The Real-World Scenario — A Complete Asset Journey

This is the most powerful answer you can give. Walk through one asset's full life to show how all 10 groups connect:

THE LIFE OF A GOVERNMENT VEHICLE =====

PHASE 1: ACQUISITION

The Ministry of Health needs a new ambulance.

1. Group 5 (Finance) — Budget is approved for "Medical Transport"
2. Group 9 (Inventory) — Procurement process: supplier selected, PO issued
3. Group 2 (Asset Reg) — Vehicle registered: "MOH-AMB-2026-001"
Serial number, manufacturer, purchase cost recorded


```

    ↓↓↓
4. AuditLog (Group 1) — "User moh_admin created asset MOH-AMB-2026-001"
                        (IP address: 10.0.1.45, Timestamp: 2026-01-15 09:30)
    ↓↓↓
5. Group 10 (Analytics) — Dashboard updates: total assets +1, budget used -$45,000

PHASE 2: OPERATION & MAINTENANCE
-----
The ambulance is in service for 5 years.

6. Group 6 (GIS)       — Vehicle tracked on map at Dar es Salaam depot
7. Group 7 (Energy)    — Fuel consumption monitored (if IoT-enabled)
8. Group 4 (CMMS)      — Scheduled service every 5,000 km
                        Work order #WO-2026-891 created for oil change
    ↓↓↓
9. Group 8 (Compliance) — Annual inspection due – inspection scheduled
    ↓↓↓
10. AuditLog (G1)      — "Work order WO-2026-891 completed by mechanic_john"
11. Group 3 (Lifecycle) — Condition assessment: GOOD (85% useful life remaining)

PHASE 3: FINANCIAL TRACKING
-----
12. Group 5 (Finance) — Depreciation calculated monthly
                        Current value: $32,000 (from $45,000)
13. Group 10 (Analytics) — Report: "Fleet operational costs vs budget"

PHASE 4: DISPOSAL
-----
After 8 years, the ambulance is decommissioned.

14. Group 3 (Lifecycle) — Condition assessment: POOR (15% life remaining)
15. Group 5 (Finance)   — Disposal value calculated: $5,000
16. Group 2 (Asset Reg) — Status changed to DISPOSED
                        Disposal method: AUCTION
    ↓↓↓
17. AuditLog (G1)      — "Asset MOH-AMB-2026-001 disposed by moh_admin"
18. Group 10 (Analytics) — Final report generated: "Asset lifetime cost analysis"
```

Every single step above is tracked by Group 1's audit trail. The panel asks "How do you know this happened?" — open AuditLog, filter by `asset_number=MOH-AMB-2026-001`, and see all 18 events in chronological order with user, timestamp, and IP.

How Each Group Connects to Group 1

Group	What They Build	How They Use Our Platform
2 — Asset Register	Asset CRUD, categories, documents, barcodes	Extends our <code>base.html</code> , uses <code>@role_required</code> , writes to <code>AuditLog</code> , runs inside <code>schema_context</code>
3 — Lifecycle Planning	Capital projects, condition assessments, lifecycle costs	Same — our decorators protect their views, schema isolation keeps their data per ministry
4 — Maintenance (CMMS)	Work orders, preventive schedules, technician mobile app	Uses our JWT auth for their mobile app, our RBAC for assigning technicians
5 — Financial	Depreciation, leases, budgets, financial reports	Our <code>verify-token</code> endpoint tells them the user's role and ministry for financial approvals
6 — GIS	Maps, buildings, floors, rooms, space bookings	Our schema isolation ensures MOH buildings don't appear on MOF's map
7 — Energy/IoT	Smart meters, utility bills, carbon tracking	Our API security authenticates IoT device-to-system calls
8 — Compliance	Risk registers, incidents, inspections, CAPA	Our audit trail logs every compliance action; our role system controls who can close incidents

Group	What They Build	How They Use Our Platform
9 — Inventory	Spare parts, purchase orders, suppliers, stock movements	Our JWT auth for warehouse scanners; our schema isolates stock per ministry
10 — Analytics/BI	KPI dashboards, reports, drill-down, predictive analytics	Queries our AuditLog for user activity data; respects our schema isolation for per-ministry reports

One URL, One Login — The Power of SSO

WITHOUT KEYCLOAK (each group has their own login):

- Group 1: `https://group1.platform.go.tz/` → type password
- Group 2: `https://group2.platform.go.tz/` → type DIFFERENT password
- Group 3: `https://group3.platform.go.tz/` → type ANOTHER password
- ... user needs 10 different passwords – writes them on sticky notes

WITH KEYCLOAK (shared identity):

- User logs into Keycloak ONCE
- Group 1: "Keycloak says you're moh_admin – come in"
- Group 2: "Keycloak says you're moh_admin – come in"
- Group 3: "Keycloak says you're moh_admin – come in"
- ... NO extra passwords – SSO just works

What Changes About Our Code After Integration

Our code changes minimally because integration is at the identity level, not the code level:

Component	Before Integration (Current)	After Integration (Production)
<code>authentication/</code>	Our code	Stays exactly as-is — no changes
<code>tenants/</code>	Our code	Stays exactly as-is — no changes
<code>organizations/</code>	Our code	Stays exactly as-is — no changes
<code>assets/</code>	Our reference app	Stays as our reference/demo — Group 2 builds their OWN separate system
API endpoints	<code>/api/auth/</code> , <code>/api/assets/</code>	Same endpoints — other groups call them from their own systems
<code>verify-token</code>	Returns role + ministry_schema	Same — this is the contract other groups depend on
Sidebar	Our links only	Our sidebar stays the same — other groups have their OWN sidebar

Our API is the bridge. We do not give them our code. We do not take their code. We provide a stable API endpoint (`/api/auth/verify-token/`) that any group can call from any programming language (Django, Flask, Node.js, PHP, .NET — it does not matter).

The Integration Process — How It Actually Happens

STEP 1: We give Group 2 our API docs URL and test credentials

- └ "Here is Swagger: `http://192.168.100.18:8000/api/docs/`"
- └ "Here is a test user: moh_admin / Admin@123"

STEP 2: Group 2 opens Swagger, explores available endpoints

- └ They test `POST /api/auth/login/` → get a token
- └ They test `GET /api/auth/verify-token/` → see role + ministry
- └ They understand the API contract

STEP 3: Group 2 builds THEIR OWN asset register system

- └ They can use any technology (Django, React, Flutter – their choice)
- └ They set up their own login page
- └ Their login page redirects to Keycloak OR calls our `/api/auth/login/`

```
STEP 4: Group 2 calls our verify-token on every user login
├─ User logs into Group 2's system
├─ Group 2 sends the token to our /api/auth/verify-token/
├─ We respond: {"role": "MINISTRY_ADMIN", "ministry_schema": "moh_schema"}
├─ Group 2 shows their interface, scoped to the correct ministry

STEP 5: The system is live – all 10 groups running as INDEPENDENT systems
├─ Each group manages their own deployment
├─ Each group has their own URL
├─ But ALL share the same Keycloak identity
├─ Users log in ONCE and access ALL systems seamlessly
```

This works because of OpenID Connect standard:

- Any system that understands OIDC can talk to Keycloak
- Any system that can make an HTTP request can call our `verify-token` endpoint
- No code sharing needed — just API contracts

What Changes About Our Code After Integration

Our current repository has all 10 groups' features in one codebase because we built reference implementations. After true integration:

Component	Before Integration (Current)	After Integration (Production)
<code>authentication/</code>	Our code	Stays exactly as-is
<code>tenants/</code>	Our code	Stays exactly as-is
<code>organizations/models.py</code>	Our code (OrgUnit, AuditLog)	OrgUnit stays, AuditLog grows (other groups also write to it)
<code>organizations/views.py</code>	Our code (audit log viewer)	Stays — we own the audit viewer UI
<code>assets/</code>	Our code (reference app)	Replaced by Group 2's proper asset module
<code>templates/shared/base.html</code>	Our code (sidebar has our links)	Sidebar updated to include all 10 groups' navigation links
<code>config/urls.py</code>	Our routes + our asset routes	All groups' URL patterns registered
<code>config/settings.py</code>	Our <code>INSTALLED_APPS</code>	All groups' apps added to <code>SHARED_APPS</code> / <code>TENANT_APPS</code>
API endpoints	<code>/api/assets/</code> , <code>/api/auth/</code>	<code>/api/assets/</code> (Group 2), <code>/api/maintenance/</code> (Group 4), etc.

Our code changes minimally. We add their apps to `INSTALLED_APPS`, they add their URL patterns, and everything runs together because `django-tenants` routes all requests through `schema_context` and our `@role_required` decorator protects their views.

The Integration Process — How It Actually Happens

```
STEP 1: Group 2 builds their asset module
├─ assets/models.py (their Asset model, more fields than ours)
├─ assets/views.py (their views)
├─ assets/api_views.py (their API)
├─ templates/assets/* (their templates)

STEP 2: Group 2 copies their code into OUR repository
├─ (our platform code – unchanged)
├─ assets/ ← Group 2's code replaces our reference app
├─ templates/assets/ ← Group 2's templates replace ours

STEP 3: We update settings.py
INSTALLED_APPS = [
    ... our apps ...,
    'assets', ← Now it's Group 2's app, not our reference
    'lifecycle', ← Group 3's app
    'maintenance', ← Group 4's app
    ...
]
```

```

STEP 4: We verify nothing broke
├─ Keycloak login still works
├─ RBAC still controls access
├─ Schema isolation still works
├─ Audit trail still records everything
└─ JWT tokens still authenticate API calls

```

STEP 5: The system is live – all 10 groups running as ONE platform

This works because of our architecture choices:

- `@role_required` decorator works on ANY view, including Group 2's views
- `schema_context` works on ANY model, including Group 3's models
- `AuditLog.create()` can be called from ANY app's code
- `verify-token` API works for ANY group's frontend or mobile app

Summary — What the Panel Should Remember

Question	Answer
What is the final product?	One unified government asset management platform — all 10 modules in one system
How do users access it?	One URL, one Keycloak login, one sidebar navigation
How are 10 groups' data kept separate?	PostgreSQL schemas per ministry — automatic via django-tenants
How is access controlled?	Role-based access control (5 levels) — the same RBAC protects ALL groups' views
Where is every action recorded?	Group 1's AuditLog — immutable, tamper-proof, timestamped, IP-tracked
What if a new ministry joins?	Super Admin creates it → schema auto-created → all 10 groups' tables created automatically
How do Groups 2-10 know who is logged in?	They call our <code>verify-token</code> endpoint or use our <code>@role_required</code> decorator
What makes this different from 10 separate systems?	Single login, shared navigation, cross-module data queries, unified audit trail, centralized admin

The Final Vision Statement

"After all 10 groups integrate, the Government Asset Management Platform is a single, unified system where any government employee — from a facility clerk in a rural clinic to a ministry admin in Dar es Salaam — logs in through one URL and gets a personalized dashboard showing exactly the data and tools they need. They can register an asset (Group 2), schedule maintenance (Group 4), track its depreciation (Group 5), see it on a GIS map (Group 6), monitor its energy consumption (Group 7), check compliance status (Group 8), order spare parts (Group 9), and generate reports (Group 10) — all without ever logging in again. Every action is tracked in Group 1's audit trail, secured by Group 1's Keycloak SSO, isolated within Group 1's multi-tenant schemas, and protected by Group 1's role-based access control. This is not ten separate systems working independently. This is one system where Group 1 is the foundation and Groups 2-10 are the specialized modules built on top."

PART 35: SWAGGER API DOCUMENTATION — What It Is, Where Defined, How to Access

The panel question: "How do other groups use your API? Where is the documentation? How does Swagger work?"

The Short Answer (15 seconds)

"Swagger is an auto-generated, interactive API documentation page at `/api/docs/`. Other groups open this URL in their browser, see every API endpoint we offer with its request/response format, and can even test endpoints directly by clicking 'Try it out'. The documentation is generated automatically from our code by a library called `drf_yasg` — we never write documentation manually."

What Is Swagger?

Swagger (also called OpenAPI) is an **interactive documentation page for APIs**. Instead of writing a Word document or PDF explaining "how to call our API", Swagger reads our Python code and generates a live web page at:

```
http://localhost:8000/api/docs/
```

What makes it "interactive":

- You see every endpoint listed (GET, POST, PUT, DELETE)
- You see the exact request format (what fields to send, what data types)
- You see the exact response format (what fields come back, their types)
- You can click **"Try it out"** → fill in parameters → click **Execute** → see the real API response
- You can authenticate by clicking **Authorize** and pasting your JWT token

Where Is Swagger Defined in Our Code?

Step 1 — Install the library in `settings.py`:

`config/settings.py` → line 40:

```
INSTALLED_APPS = [
    ...
    'drf_yasg',  # ← This is the Swagger library
    ...
]
```

`drf_yasg` stands for "Django REST Framework — Yet Another Swagger Generator". It reads all our API view classes, serializers, and URL patterns to build the documentation page.

Step 2 — Configure Swagger in `urls.py`:

`config/urls.py` → lines 56-79:

```
from drf_yasg.views import get_schema_view
from drf_yasg import openapi

schema_view = get_schema_view(
    openapi.Info(
        title="GovAsset Platform API",
        default_version='v1',
        description="""
            REST API for the Government Asset Management Platform.

            ## How to authenticate
            1. Call POST /api/auth/login/ with username and password
            2. Copy the 'access' token from the response
            3. Click the Authorize button and enter: Bearer {your_token}
            4. All requests will now include the token automatically

            ## For Groups 2-10
            Call GET /api/auth/verify-token/ with the user's Bearer token
            to validate it and get role and ministry_schema.
        """,
        contact=openapi.Contact(email="admin@platform.go.tz"),
    ),
    public=True,
    permission_classes=[permissions.AllowAny],
)
```

This sets the title, version, description text, and contact email that appear at the top of the Swagger page. The description text includes instructions for other groups on how to authenticate.

Step 3 — Register the URLs in `urls.py`:

`config/urls.py` → lines 146-156:

```
# API documentation — Swagger UI and ReDoc
path('api/docs/', schema_view.with_ui('swagger', cache_timeout=0), name='schema-swagger-ui'),
path('api/redoc/', schema_view.with_ui('redoc', cache_timeout=0), name='schema-redoc'),
```

Two URLs are created:

- `/api/docs/` — Swagger UI (interactive, with "Try it out" buttons)
- `/api/redoc/` — ReDoc (read-only, cleaner layout for reading)

What Endpoints Does Swagger Document?

Swagger automatically discovers ALL API endpoints defined in `authentication/api_urls.py`. Here is exactly what it shows:

Method	Endpoint	Purpose	Defined In
POST	<code>/api/auth/login/</code>	Login with username/password → get JWT tokens	<code>authentication/api_views.py:33-70</code>
POST	<code>/api/auth/refresh/</code>	Refresh an expired access token	<code>authentication/api_views.py:72-85</code>
GET	<code>/api/auth/me/</code>	Get current user's profile	<code>authentication/api_views.py:87-103</code>
GET	<code>/api/auth/verify-token/</code>	Verify a token and get role + ministry	<code>authentication/api_views.py:223-254</code>
POST	<code>/api/auth/logout/</code>	Logout (blacklist the refresh token)	<code>authentication/api_views.py:105-129</code>
GET	<code>/api/assets/</code>	List all assets (with filters)	<code>assets/api_views.py:31-125</code>
POST	<code>/api/assets/</code>	Create a new asset	<code>assets/api_views.py:31-125</code>
GET	<code>/api/assets/<id>/</code>	Get a single asset's details	<code>assets/api_views.py:127-175</code>
PUT	<code>/api/assets/<id>/</code>	Update an asset	<code>assets/api_views.py:127-175</code>
DELETE	<code>/api/assets/<id>/</code>	Delete an asset (admin only)	<code>assets/api_views.py:127-175</code>
GET	<code>/api/assets/categories/</code>	List asset categories for dropdowns	<code>assets/api_views.py:177-191</code>
GET	<code>/api/org-units/</code>	List org units (facilities)	<code>organizations/api_views.py:14-118</code>
GET	<code>/api/audit-logs/</code>	List audit log entries (with filters)	<code>organizations/api_views.py:121-194</code>
GET	<code>/api/dashboard/stats/</code>	Get dashboard statistics	<code>organizations/api_views.py:198-326</code>

How does Swagger know about all these endpoints without us telling it?

Swagger reads our `api_urls.py` file, which imports all the view classes and maps them to URLs. For each view class, Swagger reads:

- The serializer class (to know request/response fields)
- The model (to know field types)
- The docstrings/comments (to show descriptions)
- The HTTP methods allowed (GET, POST, etc.)
- The permission classes (to show which endpoints require authentication)

No manual documentation is needed. When we add a new API endpoint, it automatically appears on the Swagger page the next time you refresh.

How Other Groups Use Swagger — Step by Step

Here is exactly what a developer from Group 2 does when they want to test our API:

DEVELOPER FROM GROUP 2 (Asset Register Team)

1. Opens `http://localhost:8000/api/docs/` in their browser
2. Sees the Swagger page with ALL endpoints listed
3. Scrolls to `POST /api/auth/login/`
 - Clicks "Try it out"
 - Types: `{ "username": "moh_admin", "password": "Admin@123" }`
 - Clicks "Execute"
 - Gets response: `{ "access": "eyJ...", "refresh": "eyJ..." }`
4. Copies the "access" token (starts with `eyJ...`)
5. Scrolls to top of page → clicks "Authorize" button
 - Pastes: `Bearer eyJ...`
 - Clicks "Authorize"
6. Now they can test ANY endpoint:
 - `GET /api/assets/` – sees MOH's assets
 - `GET /api/auth/verify-token/` – sees the user's role
 - `GET /api/audit-logs/` – sees recent activity
7. When they build their own mobile app, they copy the exact request format they tested in Swagger

The "Authorize" Button Explained

The most important feature in Swagger is the **Authorize** button at the top of the page. Here is what happens:

Without Authorize:

- You can see the endpoints and their formats
- You CANNOT call any endpoint that requires authentication
- `GET /api/assets/` returns 401 Unauthorized

With Authorize (after pasting Bearer token):

- Every request sent from Swagger includes your token
- `GET /api/assets/` returns real data
- You stay authorized until the token expires (30 minutes)
- When token expires, repeat steps 3-5

How the Authorize button works in our code:

In `config/settings.py` → DRF settings:

```
REST_FRAMEWORK = {
    'DEFAULT_AUTHENTICATION_CLASSES': [
        'authentication.api_auth.CustomJWTAuthentication',
    ],
    'DEFAULT_PERMISSION_CLASSES': [
        'rest_framework.permissions.IsAuthenticated',
    ],
}
```

```
...
}
```

Swagger reads these settings and knows that most endpoints require a **Bearer** token. It adds the Authorize button automatically. When the user pastes a token, Swagger includes it in the **Authorization** header of every request.

What Makes Swagger Different From Writing Documentation By Hand

Aspect	Hand-written Documentation	Swagger (Our Approach)
Effort to create	Write paragraphs for each endpoint	Zero — auto-generated from code
Stays up to date	No — outdated the moment code changes	Yes — regenerated from current code
Shows exact fields	May miss some fields or have typos	Exact — reads serializer code
Interactive testing	No — read-only document	Yes — "Try it out" button calls the real API
Shows auth requirements	Must be written manually	Auto-detected from permission classes
Mobile-friendly	PDF on phone is hard to read	Responsive web page
Multiple formats	Usually just PDF	Swagger UI + ReDoc (2 formats)

Common Misconceptions About Swagger

"Swagger IS the API." No. Swagger is *DOCUMENTATION* of the API. The API is the actual endpoints that return data (<http://localhost:8000/api/assets/>). Swagger is at </api/docs/> — it just describes what the API does.

"Swagger is a separate tool we need to install and maintain." No. Swagger is built into our project through the *drf-yasg* package. It runs as part of Django. You installed it once via *pip install drf-yasg* and added it to *INSTALLED_APPS*.

"Swagger documentation must be written by hand." No. Swagger reads your code — URL patterns, serializers, models, docstrings — and generates documentation automatically. You maintain it by updating your code, not by editing docs.

"Swagger only shows GET endpoints." No. Swagger shows *ALL* endpoints — POST, PUT, DELETE, PATCH — and lets you test them all.

"Swagger exposes our API to security risks." No. Swagger shows the documentation, but it enforces the same authentication as the API itself. You cannot access protected endpoints without a valid token just because you are using Swagger.

How to Access Swagger During the Presentation

```
On your local machine:  http://localhost:8000/api/docs/
On the network (phone): http://192.168.100.18:8000/api/docs/
                        (replace 192.168.100.18 with your actual IP)
```

For the demo:

1. Open <http://localhost:8000/api/docs/> in a browser tab
2. Scroll to **POST /api/auth/login/**
3. Click "Try it out"
4. Enter `{"username": "moh_admin", "password": "Admin@123"}`
5. Click "Execute"
6. Copy the **access** token
7. Click "Authorize" → paste **Bearer {token}**
8. Now call **GET /api/assets/** — shows MOH assets
9. Call **GET /api/auth/verify-token/** — shows role and ministry
10. This demonstrates exactly what Groups 2-10 receive when they integrate

PART 36: WHY KEYCLOAK — Why Not Just Normal Authentication + Links in the Sidebar?

The panel question: "Why do you need Keycloak? Couldn't you just use normal Django authentication, secure everything properly, and then put links to other groups' systems in your sidebar? Users click a link and go to that system?"

The Short Answer (15 seconds)

"If we skip Keycloak and just add links to other groups in our sidebar, each group would need their own separate login page with separate credentials. A user would have 10 different passwords for 10 different systems. With Keycloak, one login works everywhere — that is the entire point of SSO. The sidebar link approach only works if ALL groups are in ONE Django project. If they are separate systems on different servers, Keycloak is the industry-standard solution for unified authentication."

The Two Scenarios — Why the Answer Depends on Architecture

The panel might be thinking of two different scenarios. Let me explain both:

Scenario 1: All Groups in ONE Django Project (Our Vision)

If all 10 groups' code lives in ONE Django project (our repository), then **we don't need Keycloak for navigation** — we just add their links to the sidebar in `base.html`. The user logs in once (via our normal Django auth or Keycloak), and clicks sidebar links to navigate between modules. Everything is within the same Django session.

In this scenario, we still need Keycloak. Why? Because:

1. **One login for the whole system** — Even within one Django project, our normal authentication only works for OUR project. If other groups want to access OUR API from THEIR separate mobile apps or external systems, they need a standard authentication protocol. Keycloak provides that.
2. **The other groups are not just web pages** — Group 4 has a technician mobile app. Group 6 has GIS maps. Group 7 has IoT sensors. These are not links you click in a sidebar — they are separate applications that need to authenticate against our platform. Keycloak provides tokens they can use from ANY application.
3. **Government SSO is a requirement** — The project requires that users log in with a single government credential across ALL systems. Normal Django auth cannot provide this. Keycloak can connect to the government's central identity system.
4. **Our API needs to work WITHOUT the sidebar** — When Group 2 builds their own frontend (not using our sidebar), they still need to authenticate users. Our `verify-token` API endpoint works because Keycloak issued the token. Normal Django session auth would not work from an external frontend.

Scenario 2: Each Group Has Their Own Server (Separate Systems)

This is the scenario the panel might be thinking of. Imagine:

```
Group 1's server: https://auth.platform.go.tz/    (our platform)
Group 2's server: https://assets.platform.go.tz/  (their asset system)
Group 3's server: https://lifecycle.platform.go.tz/ (their lifecycle system)
...
```

If each group runs their own Django project on their own server, then **putting links in our sidebar is useless** — clicking a link takes the user to a completely different website with a different login page. The user would need separate credentials for each.

This is exactly why Keycloak exists. Here is what happens WITH Keycloak in this scenario:

1. User visits `https://auth.platform.go.tz/` (our platform)
2. User clicks "Asset Register" in our sidebar
3. User is redirected to Group 2's server: `https://assets.platform.go.tz/`
4. Group 2's server sees no session → redirects to Keycloak

BUT — Keycloak sees the user already logged in!
Keycloak says: "I know this user — here is their identity"

5. User is logged into Group 2's system AUTOMATICALLY
No second password. No second login page. It just works.

6. This is called SINGLE SIGN-ON (SSO)

Without Keycloak, step 4-5 does not happen. The user would see Group 2's login page and need to type their password again.

Why Normal Django Auth Is Not Enough

Requirement	Normal Django Auth	Keycloak (Our Choice)
Single login for our web app	✓ Yes	✓ Yes
Single login for all 10 groups' web apps	✗ No — each group needs separate auth	✓ Yes — SSO across all
Mobile app authentication	✗ No — sessions don't work on mobile	✓ Yes — JWT tokens from Keycloak
API authentication for external systems	✗ No — session-based, not token-based	✓ Yes — standard tokens
Integration with government ID system	✗ No — custom implementation needed	✓ Yes — built-in LDAP/SAML/OIDC
User self-service (forgot password)	✗ Must build yourself	✓ Built-in Keycloak feature
Multi-factor authentication (MFA)	✗ Must build yourself	✓ Built-in (OTP, SMS, etc.)
Social login (Google, etc.)	✗ Must build yourself	✓ Built-in identity brokering
Session management (see all active sessions)	✗ Must build yourself	✓ Built-in admin console
User federation (connect to existing user DB)	✗ Must build yourself	✓ Built-in (LDAP, Active Directory)
Standard protocol (OIDC, SAML)	✗ Custom token format	✓ Industry standard

The Real Problem with the "Sidebar Links" Approach

Let me walk through the actual user experience with and without Keycloak:

WITHOUT KEYCLOAK – "Sidebar Links" Approach:

```
User logs into Group 1:    https://group1.platform.go.tz/
→ Sidebar shows links to Group 2, Group 3, etc.
→ User clicks "Asset Register"
→ Browser goes to:        https://group2.platform.go.tz/
→ User sees GROUP 2'S LOGIN PAGE ← PROBLEM!
→ User types username/password AGAIN (if they even have an account)
→ If they go to Group 3, they type password AGAIN
→ User has 10 different passwords – impossible to remember
→ Users write passwords on sticky notes – security disaster
```

WITH KEYCLOAK – SSO Approach:

```
User logs into Group 1:    https://group1.platform.go.tz/
→ (Login happens on Keycloak's page, not ours)
→ Sidebar shows links to Group 2, Group 3, etc.
→ User clicks "Asset Register"
→ Browser goes to:        https://group2.platform.go.tz/
→ Group 2 checks with Keycloak: "Is this user logged in?"
→ Keycloak: "Yes! User is moh_admin with MINISTRY_ADMIN role"
→ User is AUTOMATICALLY logged in – no second prompt
→ User clicks "Capital Projects" (Group 3) – same thing
→ One login. All systems. No extra passwords.
```

This is the difference between "having sidebar links" and "having SSO with sidebar links." Both have links — but only SSO makes those links actually useful.

But What If All 10 Groups Use the Same Django Instance? (One Repository)

If all groups merge into ONE Django project (as described in Part 34), then:

- **We still need Keycloak** because users access the system from multiple devices.
- A user might log in from their office desktop, their mobile phone, and a tablet.
- Keycloak manages all these sessions centrally.
- Keycloak provides the JWT tokens that the mobile API uses.
- Keycloak can integrate with the Tanzanian government's central identity system when that becomes available.

The sidebar links work fine in this scenario — but only because everything is in ONE Django instance. The sidebar is not a replacement for Keycloak. The sidebar handles NAVIGATION. Keycloak handles AUTHENTICATION. They solve different problems:

```

Sidebar:  "Where do I go next?"    (Navigation)
Keycloak: "Who am I?"             (Identity)
JWT:      "Can I prove who I am?"  (Authentication proof for APIs)
RBAC:     "What am I allowed to do?" (Authorization)
AuditLog: "What did I do?"        (Accountability)

```

The Final Reason — Government Standard

Most governments worldwide use Keycloak or a similar SSO solution for their internal systems. The Tanzanian government likely has or will have a central identity system. Our code is already compatible:

```

# config/settings.py — just change these values when the government provides them
KEYCLOAK_SERVER_URL = 'https://sso.egov.go.tz' # Government's SSO
KEYCLOAK_REALM      = 'government_employees'  # Government's realm
KEYCLOAK_CLIENT_ID  = 'govasset-platform'     # Our app's registration

```

If we had built normal Django authentication, we would need to **rewrite the entire auth system** when the government mandates SSO. With Keycloak, we change 4 configuration values and everything works.

One-Liner Answer for the Panel

"Keycloak is not about the sidebar. It is about letting ONE login work across ALL 10 groups' systems — whether they are in our Django project or on separate servers. Without SSO, users would need 10 different passwords. With SSO, they log in once and access everything. A sidebar with links but no SSO would still show 10 different login screens — and that is a terrible user experience."

PART 37: COMPLETE BEGINNER'S GUIDE — How Integration Works & How Keycloak Login Flows

If you are confused about:

- "Will Group 2 give me their homepage link and I add it to our `urls.py`?"
- "Where does `keycloak_login_url` come from? I see it in `login.html` but where is it defined?"
- "What do all those Keycloak settings in `settings.py` mean?"

This part explains everything step by step like you are seeing it for the first time.

SECTION A — How Integration Works: "Will Group 2 Give Me a Link to Add to `urls.py`?"

Your question: "After integration, will Group 2 give me (Group 1) a link to their homepage, and I add it to our `urls.py` file? And then when a user clicks in our sidebar, they go to Group 2's system?"

Short answer: It depends on how we integrate. There are TWO ways this can work.

Way 1: Full Merge (All Groups in ONE Django Project)

This is our **vision**. All 10 groups' code lives in one Django project — our repository.

How it works step by step:

```

Step 1: Group 2 builds their Asset Register module
└─ They create files like:
    assets/models.py      (their Asset model)
    assets/views.py       (their web pages)
    assets/api_views.py   (their API)
    templates/assets/*.html (their HTML pages)

Step 2: Group 2 gives us their code files (not just a link)
└─ They send us their entire `assets/` folder
└─ They send us their `templates/assets/` folder

Step 3: We (Group 1) put their code into OUR repository
└─ We copy `assets/` into our project folder
└─ We copy `templates/assets/` into our templates folder

Step 4: We update `config/settings.py`
└─ We add 'assets' to INSTALLED_APPS
└─ Before: INSTALLED_APPS = ['authentication', 'tenants', ...]
└─ After:  INSTALLED_APPS = ['authentication', 'tenants', 'assets', ...]

Step 5: We update `config/urls.py`
└─ We add Group 2's URL routes:
    path('assets/', include('assets.urls')),
    # Now when user visits /assets/, Django runs Group 2's code

Step 6: We update `templates/shared/base.html` (the sidebar)
└─ We add a link to Group 2's module in the sidebar:
    <li><a href="/assets/">Asset Register</a></li>

Step 7: User clicks "Asset Register" in sidebar
└─ Browser goes to: http://moh.localhost:8000/assets/
└─ Django runs Group 2's code (assets/views.py)
└─ User sees Group 2's page – but it's in OUR Django project

```

Key point: We do NOT add a link to an external website. We add Group 2's CODE to our project. Their pages run INSIDE our Django on the same server. The user never leaves our website.

What you might be thinking:

Our sidebar —link→ <https://group2-other-server.com/> (WRONG)

What actually happens:

Our sidebar —link→ <http://moh.localhost:8000/assets/> (CORRECT)

- ↑ This runs in OUR Django project
- ↑ Group 2's code is inside our repository
- ↑ Same server, same login, same session

Way 2: Separate Servers (Each Group Runs Their Own System)

This is an alternative approach. Each group deploys their own Django project on their own server.

How it works step by step:

```

Step 1: Group 1's server:  https://auth.platform.go.tz/
        Group 2's server:  https://assets.platform.go.tz/
        Group 3's server:  https://lifecycle.platform.go.tz/
        (Each group has their own URL)

Step 2: Group 2 gives us their homepage URL

```

└─ They tell us: "Our system is at https://assets.platform.go.tz/"

Step 3: We add their URL to our sidebar (in base.html)

```
<li><a href="https://assets.platform.go.tz/">Asset Register</a></li>
```

Step 4: User clicks "Asset Register" in our sidebar

└─ Browser goes to: https://assets.platform.go.tz/

└─ But wait – the user is not logged in on Group 2's server!

└─ Group 2 shows their login page → user must type password AGAIN

Step 5: THIS IS WHERE KEYCLOAK SAVES US

└─ Group 2's server checks with Keycloak:

"Is this user already logged in?"

└─ Keycloak: "Yes! They logged in on Group 1's site."

└─ Group 2's server: "Come in! No password needed."

└─ User never sees a login prompt – it just works.

This is SSO (Single Sign-On). One login works across ALL servers because Keycloak connects them.

Summary Table

Question	Way 1: Full Merge	Way 2: Separate Servers
Group 2 gives us...	Their code files (models, views, templates)	Their homepage URL
We add to...	<code>settings.py</code> + <code>urls.py</code> + <code>base.html</code> sidebar	Just <code>base.html</code> sidebar
Where does Group 2's page run?	On our server, in our Django	On their own server
Does user leave our site?	No — stays on same URL	Yes — goes to different URL
Do they need Keycloak for navigation?	No — Django session works	Yes — Keycloak SSO connects them
Do they need Keycloak for API?	Yes — mobile apps need tokens	Yes — mobile apps need tokens

SECTION B — Tracing the `keycloak_login_url`: From the Button to the Keycloak Login Page

Your question: "In `login.html`, I see `<a href='{keycloak_login_url}'>`. Where does this URL come from? How does clicking it send me to Keycloak?"

Here is the exact path, step by step, with the file and line number for each step.

Step 1: The HTML Button (`login.html` → line 137)

```
<a href='{keycloak_login_url}'>Sign in with Government SSO</a>
```

`{keycloak_login_url}` is a **template variable**. Django replaces it with a value when the page is rendered. But what value?

Step 2: The View That Renders the Page (`authentication/views.py` → line 44)

```
def login_view(request):
    ...
    return render(request, 'authentication/login.html', {
        'keycloak_login_url': '/oidc/authenticate/', # ← THIS IS THE VALUE
        'error_message': error_message,
    })
```

The view sets `keycloak_login_url` to the string `'/oidc/authenticate/'`. So the HTML becomes:

```
<a href="/oidc/authenticate/">Sign in with Government SSO</a>
```

When clicked, the browser goes to: `http://localhost:8000/oidc/authenticate/`

Step 3: The URL Router Catches It (config/urls.py → line 144)

```
urlpatterns = [
    ...
    path('oidc/', include('mozilla_django_oidc.urls')), # ← CATCHES /oidc/...
    ...
]
```

The URL `/oidc/authenticate/` matches this pattern. Django says: "The `oidc/` prefix matches → look inside `mozilla_django_oidc.urls` for `authenticate/`."

`mozilla_django_oidc` is a **library** we installed (not code we wrote). It has its own URL configuration inside the library folder. It provides a view called `OIDCAuthenticationRequestView` at the `authenticate/` path.

Step 4: The Library Reads Our Settings (config/settings.py → lines 355-367)

The `OIDCAuthenticationRequestView` needs to know WHERE to redirect the user. It reads these settings:

```
# The 4 values we configure:
KEYCLOAK_SERVER_URL   = 'http://localhost:8180'      # Where is Keycloak running?
KEYCLOAK_REALM        = 'govasset'                  # Which realm (project)?
KEYCLOAK_CLIENT_ID    = 'govasset-django'            # Who are we (our app's ID)?
KEYCLOAK_CLIENT_SECRET = ''                         # Our secret password

# Django builds the Keycloak URLs automatically from those 4 values:
_KEYCLOAK_BASE = "http://localhost:8180/realms/govasset/protocol/openid-connect"

# The specific endpoint for logging in:
OIDC_OP_AUTHORIZATION_ENDPOINT = "http://localhost:8180/realms/govasset/protocol/openid-connect/auth"
```

Step 5: The Library Redirects the Browser to Keycloak

The library (`OIDCAuthenticationRequestView`) sends an HTTP redirect (302) to the browser. The browser goes to:

```
http://localhost:8180/realms/govasset/protocol/openid-connect/auth
?client_id=govasset-django
&redirect_uri=http://localhost:8000/oidc/callback/
&response_type=code
&state=random_generated_string
&prompt=login
```

Let me break this down:

- `http://localhost:8180/.../auth` — Keycloak's login page URL
- `client_id=govasset-django` — "I am the GovAsset app, let me in"
- `redirect_uri=http://localhost:8000/oidc/callback/` — "After login, send the user BACK here"
- `response_type=code` — "Give me an authorization code (not a token directly)"
- `state=random...` — "Security: verify the response matches my request"
- `prompt=login` — "Force the user to type their password (don't use saved session)"

Step 6: User Sees Keycloak's Login Page

The browser shows Keycloak's login page (not our Django page). It looks like this:

KEYCLOAK

Username: [_____]

Password: [_____]

[Sign In]

Powered by Keycloak

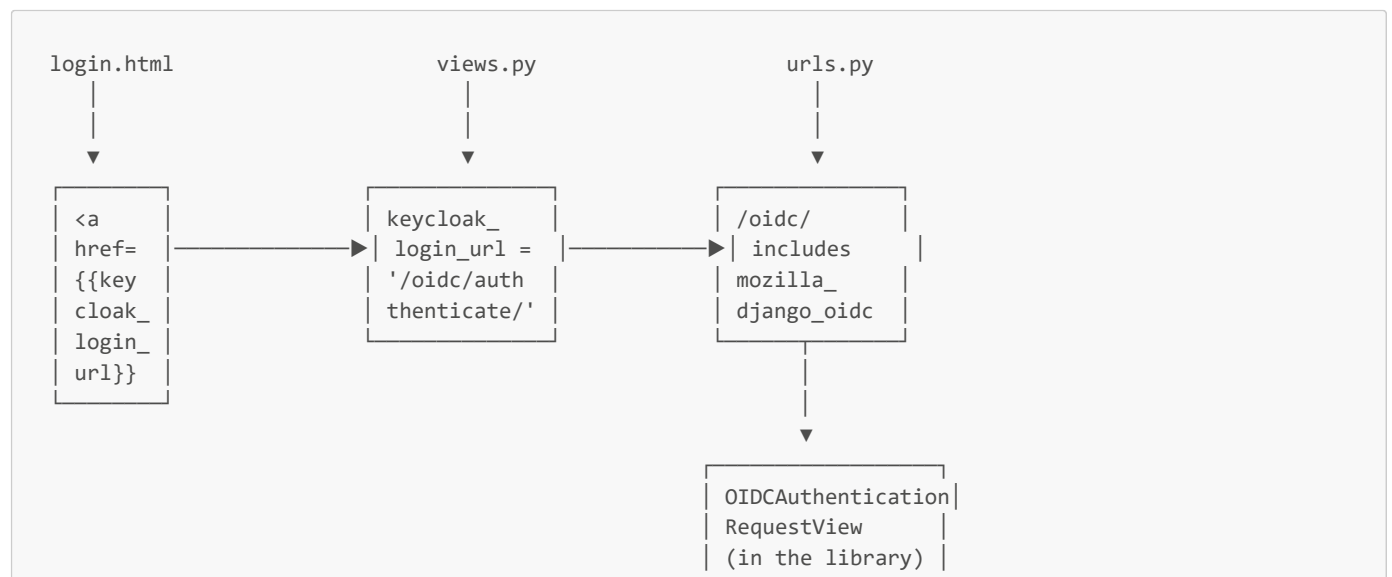
Notice: the URL in the address bar is now <http://localhost:8180/...> — we have left our Django site and are now on Keycloak's site.

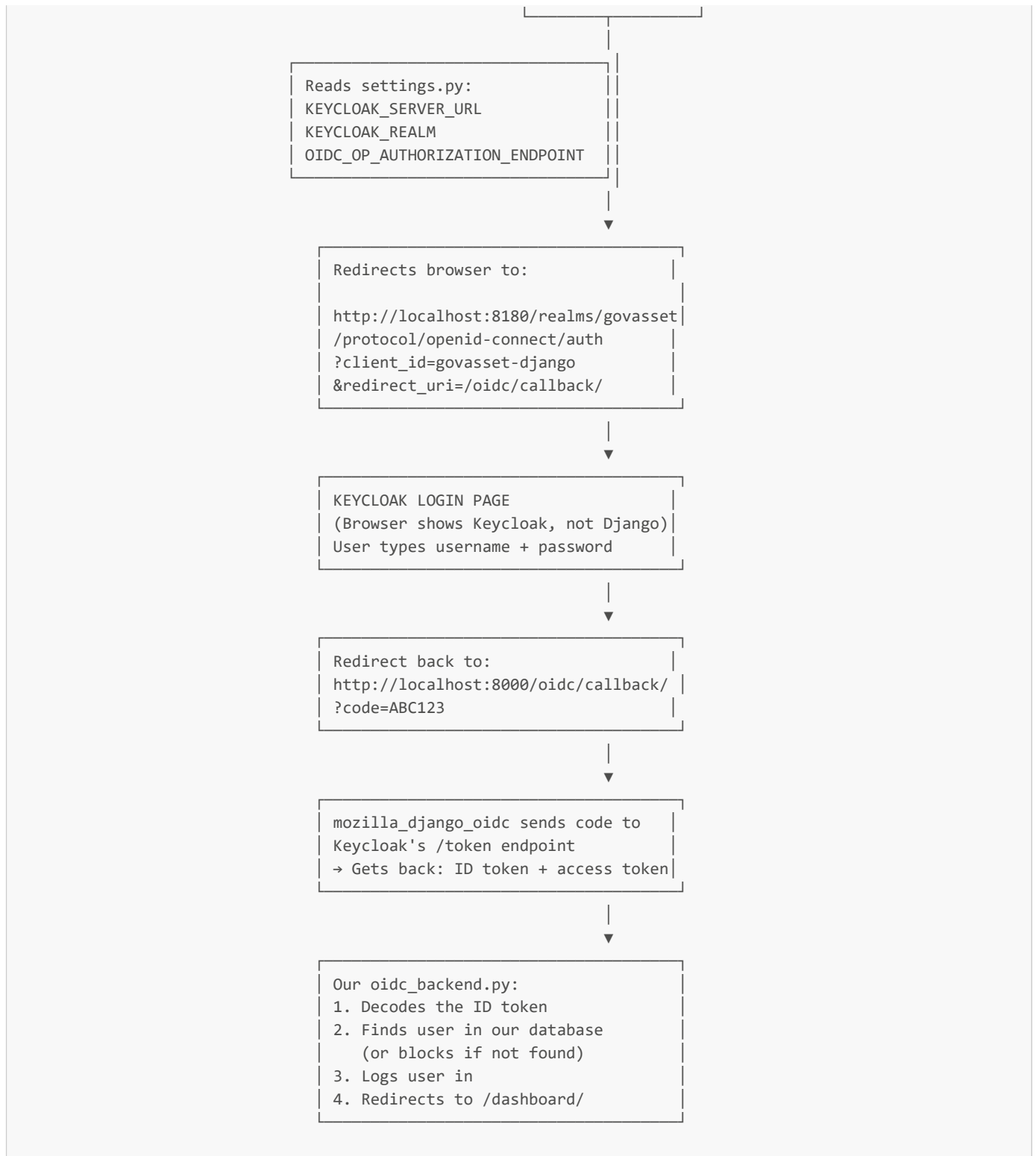
Step 7-10: The Rest of the Flow

After the user types their username and password on Keycloak's page:

7. Keycloak checks: "Is this username and password correct?"
 - └ Yes → Continue to step 8
 - └ No → Show error on Keycloak's page
8. Keycloak creates a session and redirects back to:
<http://localhost:8000/oidc/callback/?code=ABC123&state=XYZ>
9. Our Django receives the callback
 - └ mozilla_django_oidc catches /oidc/callback/
 - └ It sends the "code" to Keycloak:
 POST <http://localhost:8180/.../token>
 Body: code=ABC123&client_id=govasset-django&...
10. Keycloak returns an ID token (a JWT) containing the user's info
11. mozilla_django_oidc decodes the token, extracts the user's info
 - └ Calls our code: oidc_backend.py
 - └ Our code (line 64-88) checks: "Do we have this user in our database?"
 - └ Yes → Log them in → redirect to /dashboard/
 - └ No → Block login → redirect to /login/?error=auth_failed

Complete Flow Diagram (One Picture)





SECTION C — Understanding Every Keycloak Setting in settings.py

Your question: "I see all those Keycloak URLs in settings.py — what do they mean? How do they work?"

Let me explain each setting in plain language.

The 4 Core Settings (Lines 355-359)


```
KEYCLOAK_SERVER_URL = 'http://localhost:8180'
KEYCLOAK_REALM      = 'govasset'
KEYCLOAK_CLIENT_ID  = 'govasset-django'
KEYCLOAK_CLIENT_SECRET = ''
```

Setting	What it means	Example Value	What happens if you change it
KEYCLOAK_SERVER_URL	"Where is Keycloak running?"	http://localhost:8180	Change this to your production Keycloak URL, e.g. https://sso.gov.go.tz
KEYCLOAK_REALM	"Which realm in Keycloak?" A realm is like a project folder — users, clients, and settings are grouped by realm	govasset	If the government has their own realm, change this to theirs
KEYCLOAK_CLIENT_ID	"What is our app called in Keycloak?" Keycloak needs to know WHO is asking to log in	govasset-django	If the government registers our app under a different name, change this
KEYCLOAK_CLIENT_SECRET	"Our app's password in Keycloak" — a secret string that proves our app is really our app	' ' (empty in dev)	In production, the government gives us a secret string. We put it here.

The Auto-Built Base URL (Line 361)

```
_KEYCLOAK_BASE = f"{KEYCLOAK_SERVER_URL}/realms/{KEYCLOAK_REALM}/protocol/openid-connect"
```

This is a **helper variable**. It is not used directly — it just saves typing. It combines the server URL and realm into one base URL.

With our default settings:

http://localhost:8180/realms/govasset/protocol/openid-connect

Where Keycloak is running

Keycloak convention (always "realms")

Our realm name

OIDC protocol path (always the same)

The 5 Endpoints (Lines 363-367)

Each of these adds one more word to the base URL to create a specific endpoint.

```
OIDC_OP_AUTHORIZATION_ENDPOINT = f"{_KEYCLOAK_BASE}/auth"
OIDC_OP_TOKEN_ENDPOINT         = f"{_KEYCLOAK_BASE}/token"
OIDC_OP_USER_ENDPOINT          = f"{_KEYCLOAK_BASE}/userinfo"
OIDC_OP_JWKS_ENDPOINT          = f"{_KEYCLOAK_BASE}/certs"
OIDC_OP_LOGOUT_ENDPOINT        = f"{_KEYCLOAK_BASE}/logout"
```

Setting	Full URL (Default)	What It Does	When Is It Used
OIDC_OP_AUTHORIZATION_ENDPOINT	http://localhost:8180/realms/govasset/protocol/openid-connect/auth	Shows the Keycloak	When user clicks "Sign in with

Setting	Full URL (Default)	What It Does	When Is It Used
		login page	Government SSO"
<code>OIDC_OP_TOKEN_ENDPOINT</code>	<code>http://localhost:8180/realms/govasset/protocol/openid-connect/token</code>	Exchanges the code for tokens	After user logs in, Django sends the code here to get an ID token
<code>OIDC_OP_USER_ENDPOINT</code>	<code>http://localhost:8180/realms/govasset/protocol/openid-connect/userinfo</code>	Gets user profile info	(Optional) Gets extra user details
<code>OIDC_OP_JWKS_ENDPOINT</code>	<code>http://localhost:8180/realms/govasset/protocol/openid-connect/certs</code>	Gets Keycloak's public key	Django uses this to verify the ID token's signature
<code>OIDC_OP_LOGOUT_ENDPOINT</code>	<code>http://localhost:8180/realms/govasset/protocol/openid-connect/logout</code>	Logs user out of Keycloak	When user clicks Logout

Important for the panel: You only need to remember the FIRST and SECOND ones. The others are automatic.

The Client Settings (Lines 369-371)

```
OIDC_RP_CLIENT_ID      = KEYCLOAK_CLIENT_ID      # Same value as above
OIDC_RP_CLIENT_SECRET  = KEYCLOAK_CLIENT_SECRET  # Same value as above
OIDC_RP_SIGN_ALGO      = 'RS256'                 # How Keycloak signs its tokens
```

`OIDC_RP_CLIENT_ID` and `OIDC_RP_CLIENT_SECRET` are just copies of the same values. The library expects them to start with `OIDC_RP_` prefix. That is the only reason they exist — duplicate names for the library's naming convention.

`OIDC_RP_SIGN_ALGO = 'RS256'` means "Keycloak signs its tokens using the RS256 algorithm." This is the standard. You do not change this unless Keycloak's configuration changes.

The Redirect URLs (Lines 376-378)

```
LOGIN_REDIRECT_URL      = '/dashboard/'           # Where to send user after successful login
LOGOUT_REDIRECT_URL     = '/login/'               # Where to send user after logout
LOGIN_REDIRECT_URL_FAILURE = '/login/?error=auth_failed' # Where to send if login fails
```

Setting	Value	When Is It Used
<code>LOGIN_REDIRECT_URL</code>	<code>/dashboard/</code>	After successful Keycloak login → user lands on dashboard
<code>LOGOUT_REDIRECT_URL</code>	<code>/login/</code>	After user clicks logout → user sees login page again
<code>LOGIN_REDIRECT_URL_FAILURE</code>	<code>/login/?error=auth_failed</code>	If Keycloak login succeeds but our system doesn't recognize the user → shows error message

The Extra Settings (Lines 380-383)

```
OIDC_USE_PKCE = True
OIDC_AUTH_REQUEST_EXTRA_PARAMS = {'prompt': 'login'}
```

Setting	What It Does	Why We Set It
<code>OIDC_USE_PKCE = True</code>	Adds extra security to the OIDC flow. Prevents someone from intercepting the authorization code.	Security best practice — recommended by OAuth 2.0 spec
<code>OIDC_AUTH_REQUEST_EXTRA_PARAMS = {'prompt': 'login'}</code>	Forces Keycloak to ALWAYS ask for password, even if user was previously logged in	During development, this prevents confusing "ghost sessions" where Keycloak remembers an old login

Scenario: Changing from Localhost to Production

Here is what changes when you go from development to production:

Development (what we have now):

```
KEYCLOAK_SERVER_URL = 'http://localhost:8180'
KEYCLOAK_REALM      = 'govasset'
KEYCLOAK_CLIENT_ID  = 'govasset-django'
```

Production with our own Keycloak:

```
KEYCLOAK_SERVER_URL = 'https://sso.ourministry.go.tz' # Deployed Keycloak URL
KEYCLOAK_REALM      = 'govasset'                    # Stays the same
KEYCLOAK_CLIENT_ID  = 'govasset-django'              # Stays the same
```

Production with government's Keycloak:

```
KEYCLOAK_SERVER_URL = 'https://sso.egov.go.tz' # Government's SSO server
KEYCLOAK_REALM      = 'government_employees'  # Government's realm
KEYCLOAK_CLIENT_ID  = 'govasset-platform'     # Government registers our app
KEYCLOAK_CLIENT_SECRET = 'secret-from-government' # Government gives us this
```

You change ONLY these 4 values. Everything else in settings.py stays exactly the same. The 5 OIDC endpoints auto-update because they are built from `_KEYCLOAK_BASE`.

SECTION D — Quick Answer Cheat Sheet for the Panel

If they ask...	Say this...
"How do you add other groups to your system?"	"In full integration, we add their code to our project — their <code>urls.py</code> routes, their views, their templates. The sidebar gets updated with links to their pages. Everything runs in one Django instance."
"Where does <code>keycloak_login_url</code> come from?"	"It is set in <code>authentication/views.py</code> line 44 to <code>/oidc/authenticate/</code> . That URL is caught by <code>mozilla_django_oidc</code> library at <code>config/urls.py</code> line 144. The library reads our settings and redirects the user to Keycloak's login page."
"What is the <code>OIDC_OP_AUTHORIZATION_ENDPOINT</code> ?"	"It is the URL of Keycloak's login page. It is built from <code>KEYCLOAK_SERVER_URL</code> + <code>/realms/</code> + <code>KEYCLOAK_REALM</code> + <code>/protocol/openid-connect/auth</code> . With our defaults: <code>http://localhost:8180/realms/govasset/protocol/openid-connect/auth</code> "

If they ask...

Say this...

"How many settings do you change for production?"

"Four values: KEYCLOAK_SERVER_URL, KEYCLOAK_REALM, KEYCLOAK_CLIENT_ID, and KEYCLOAK_CLIENT_SECRET. The five OIDC endpoint URLs update automatically."